QUT Centre for Robotics



ENN583 – Foundations of Robotic Vision

Week 6 – Sem 2 – 2026

Prof Niko Suenderhauf

Welcome to Week 6!



CRICOS No.00213J

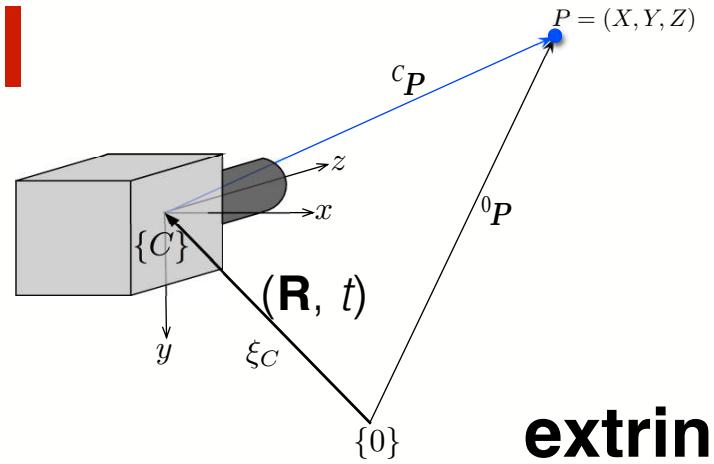


Recap

Corke, P. I Reproduced with permission from Springer Science, & Business Media

Complete camera model

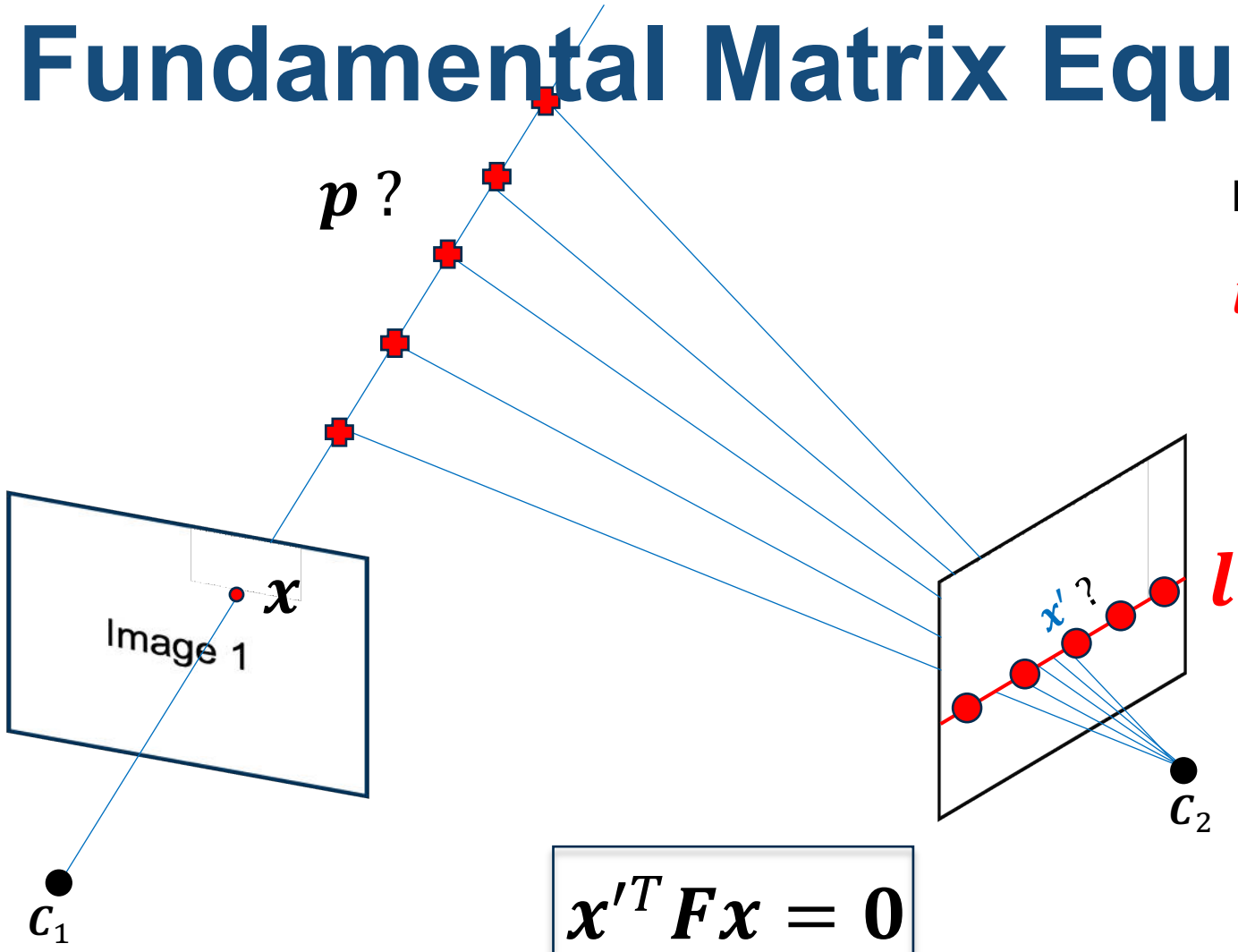
$$\begin{pmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{z} \end{pmatrix} = \begin{pmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}$$



$$\begin{pmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{pmatrix} = \underbrace{\underbrace{\begin{pmatrix} \frac{1}{\rho_u} & 0 & u_0 \\ 0 & \frac{1}{\rho_v} & v_0 \\ 0 & 0 & 1 \end{pmatrix}}_{\text{intrinsic parameters}} \begin{pmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}}_{\mathbf{K}} \underbrace{\begin{pmatrix} \mathbf{R} & t \\ \mathbf{0}_{1\times 3} & 1 \end{pmatrix}^{-1}}_{\text{extrinsic parameters}} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}}_{\mathbf{C}} \quad \ominus \boldsymbol{\xi}_C$$

camera matrix

The Fundamental Matrix Equation



F is the **Fundamental Matrix** (3x3)

l is called the **Epipolar Line** of x.

$$l = Fx$$

x' must be on l!

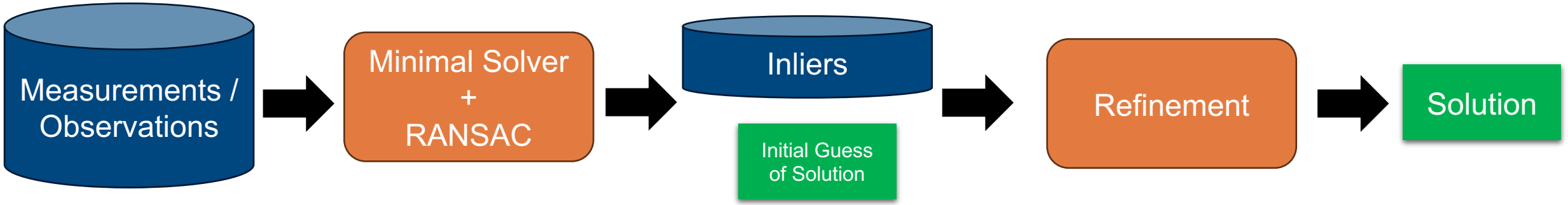
$$x'l = 0$$

Remember from the discussion of homogeneous coordinates!

$$x'^T Fx = 0$$

This is true for **any x** and corresponding **x'**!

A very typical pipeline in Computer Vision



- Typically a lot of data but contains outliners
- *e.g. matched features*

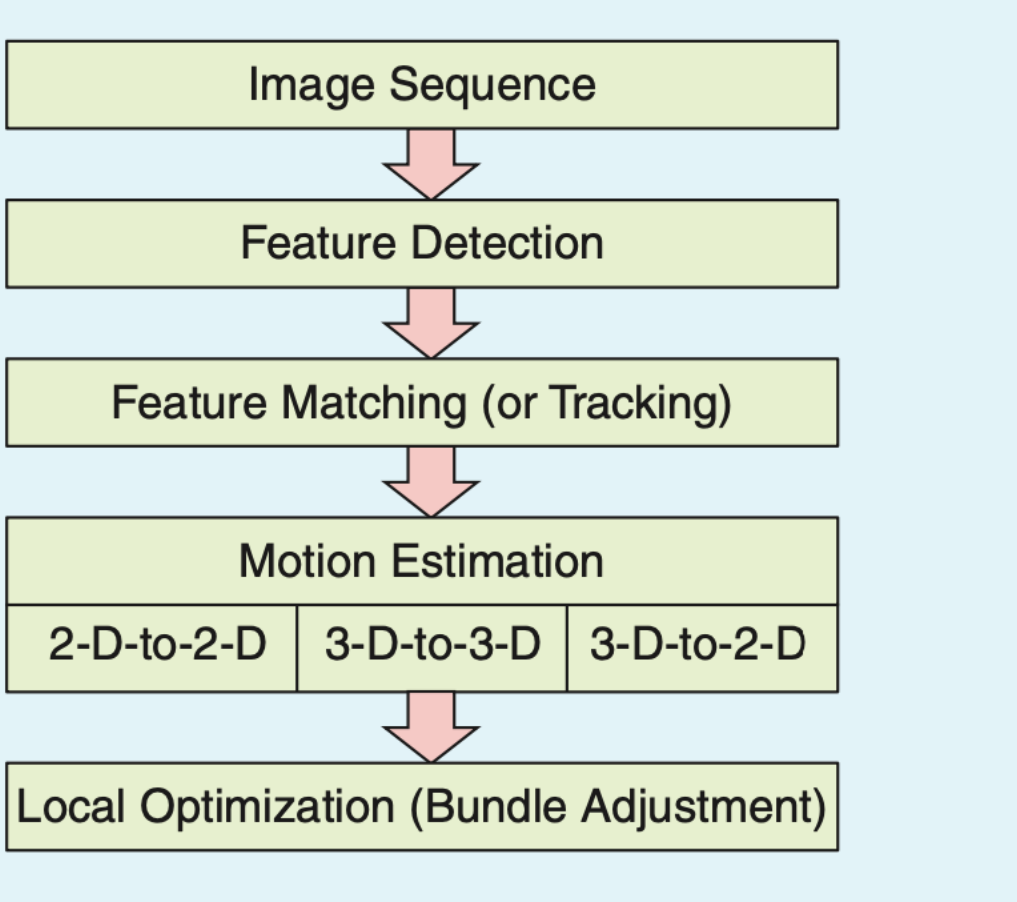
Identifies outliers

- Reduced set of inliers, guaranteed* to be free of outliers
- Solution based only on minimal set
- *e.g. 8 point pairs for F, 3 point pairs for P3P*

- Typically iterative (nonlinear) Least Squares
- Requires initial guess
- Very vulnerable to outliers

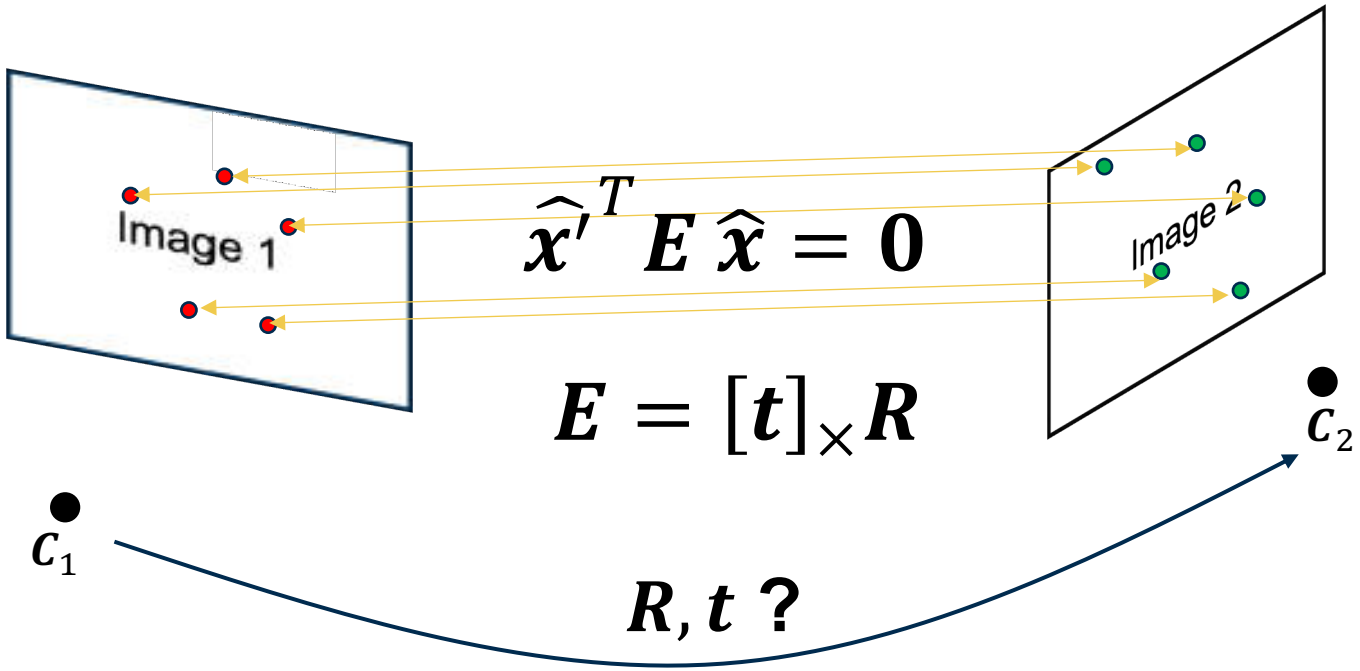
CRICOS No.00213J

Recap – What we learned so far



CRICOS No.00213J

Motion from 2D-2D Correspondences



- Find interest points in both images.
- Match them.
- Calculate Essential Matrix using `findEssentialMat()`
- Find **R** and **t** using `recoverPose(E, points1, points2)`

- **R** is correct, but **t** is only the direction of the translation!
- The translation distance cannot be recovered without additional information.

CRICOS No.00213J

Recover Scale from Known 3D Points

Component	Example Scale recovery
Parameters	s
Prediction	projected pixel $\boldsymbol{K}(\boldsymbol{R} \cdot \boldsymbol{p}_i + s \cdot \boldsymbol{t})$
Residual / Error	Reprojection Error $\boldsymbol{x}_i^{k+1} - \boldsymbol{K}(\boldsymbol{R} \cdot \boldsymbol{p}_i + s \cdot \boldsymbol{t})$
Objective	sum of squared reprojection errors $\sum_i \| \cdots \|^2$

$$s* = \text{argmin}_s \sum_i \left\| \boldsymbol{x}_i^{k+1} - \boldsymbol{K}(\boldsymbol{R} \cdot \boldsymbol{p}_i + s \cdot \boldsymbol{t}) \right\|^2$$

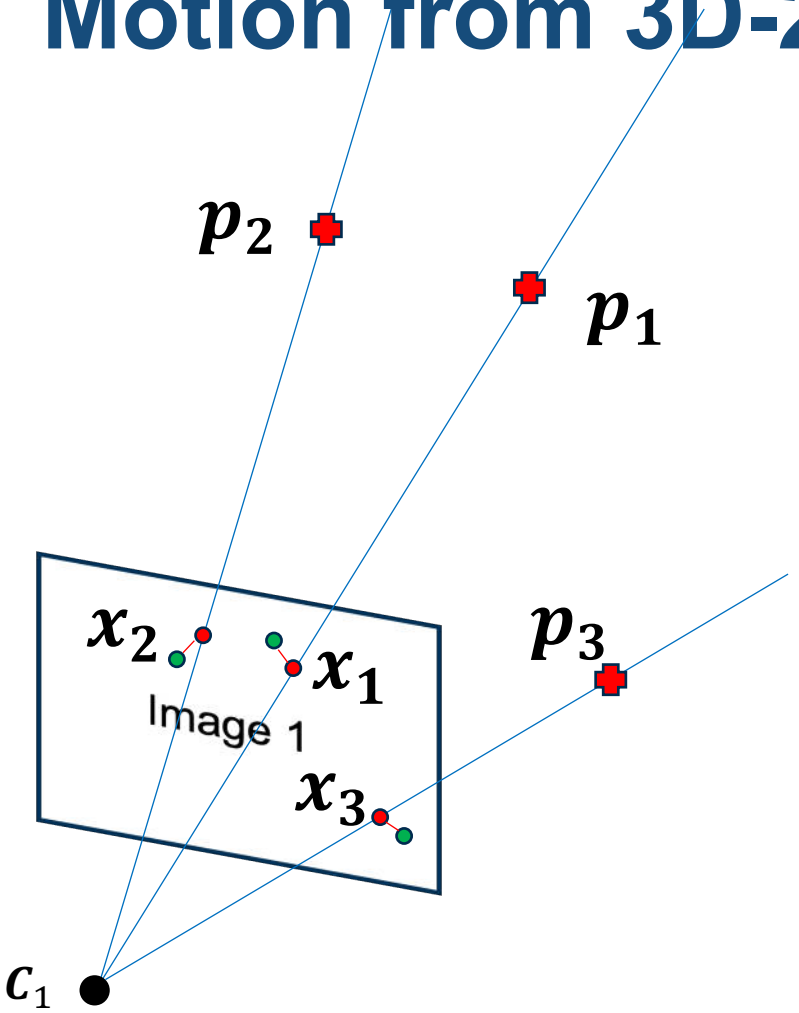
- Idea: We know the 3D coordinates $\boldsymbol{p}_i$ for some image points $\mathbf{x}_i^k$.
- We also know where that 3D point should project into the second camera, if we had the correct scale s:

$$\boldsymbol{K}(\boldsymbol{R} \cdot \boldsymbol{p}_i + s \cdot \boldsymbol{t})$$

- But if we have the wrong scale, we will observe the point at a different pixel.
- The difference is called the **reprojection error**
- Solution: Find the scale that minimises the reprojection error:

CRICOS No.00213J

PnP: Motion from 3D-2D Correspondences



Component	PnP
Parameters	$\boldsymbol{R}, \boldsymbol{t}$
Prediction	projected pixel $\boldsymbol{K}(\boldsymbol{R} \cdot \boldsymbol{p}^{i}{}_{\tau-1} + \boldsymbol{t})$
Residual / Error	Reprojection Error $\boldsymbol{x}_{\tau}^{i} - \boldsymbol{K}(\boldsymbol{R} \cdot \boldsymbol{p}^{i}{}_{\tau-1} + \boldsymbol{t})$
Objective	sum of squared reprojection errors $\sum_{i} \| \cdots \|^{2}$

$$\mathrm{argmin}_{\boldsymbol{R},\boldsymbol{t}} \sum_{i} \left\| \boldsymbol{x}_{\tau}^{i} - \boldsymbol{K}(\boldsymbol{R} \cdot \boldsymbol{p}^{i}{}_{\tau-1} + \boldsymbol{t}) \right\|^{2}$$

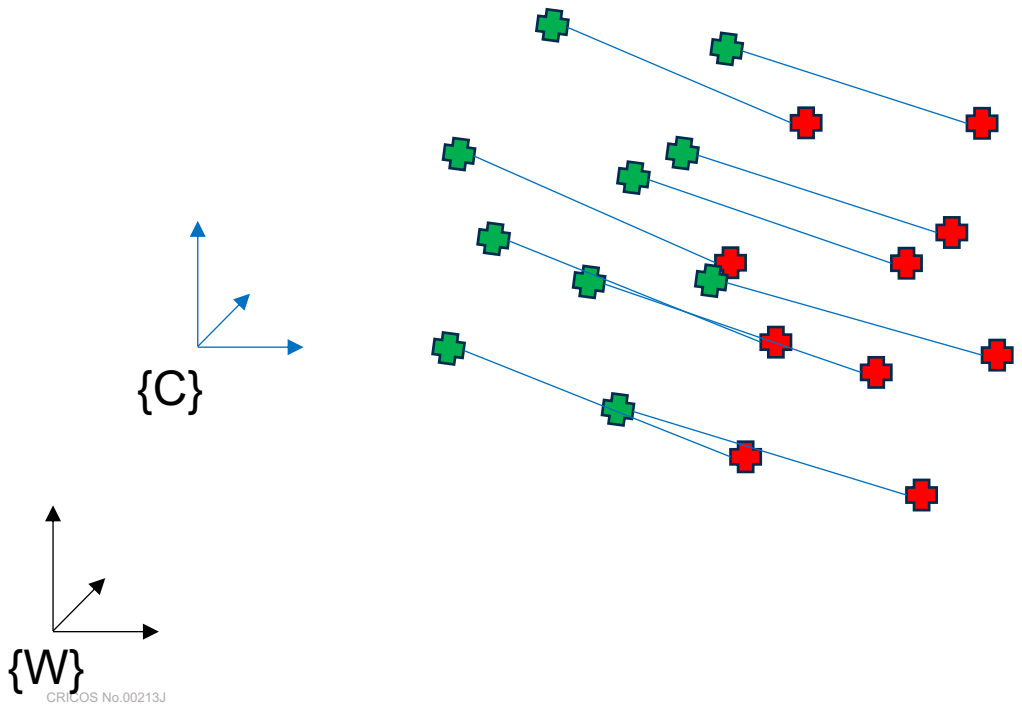
CRICOS No.00213J

Motion from 3D-3D Correspondences

$$\text{argmin}_{\boldsymbol{R},\boldsymbol{t}} \sum_i \left\| \boldsymbol{x}^i{}_{\tau} - (\boldsymbol{R} \cdot \boldsymbol{x}^i{}_{\tau-1} + \boldsymbol{t}) \right\|^2$$

Idea:

- Find the **R** and **t** that mininise the Euclidean distance between both sets of corresponding 3D points.

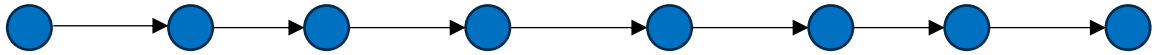


Component	3D-3D Motion Estimation
Parameters	$\boldsymbol{R}, \boldsymbol{t}$
Prediction	Transformed 3D point $\boldsymbol{R} \cdot \boldsymbol{x}^i{}_{\tau-1} + \boldsymbol{t}$
Residual / Error	Euclidean Distance in 3D $\boldsymbol{x}^i{}_{\tau} - (\boldsymbol{R} \cdot \boldsymbol{x}^i{}_{\tau-1} + \boldsymbol{t})$
Objective	sum of squared errors $\sum_i \| \cdots \|^2$

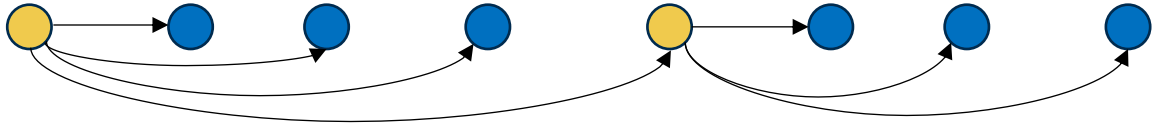
CRICOS No.00213J

Keyframes

- Don’t estimate the motion between every consecutive image pair ${}^{C_{t-1}}\boldsymbol{T}_{C_t}$



- Instead, estimate the motion relative to the last keyframe: ${}^{K_i}\boldsymbol{T}_{C_t}$

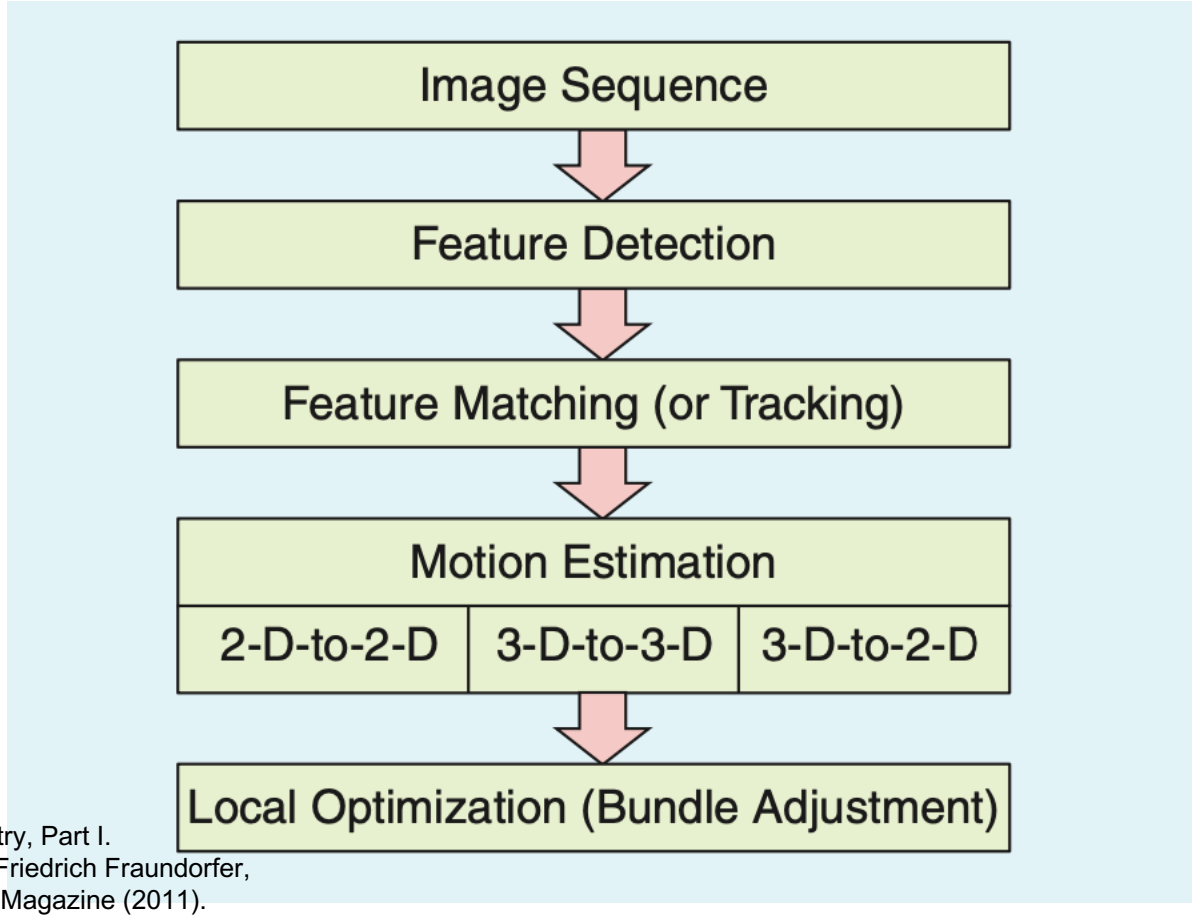




A Case Study

Continued from last week …

Case Study: Monocular Visual Odometry with PTAM



Tutorial on Visual Odometry, Part I.
Davide Scaramuzza and Friedrich Fraundorfer,
Robotics and Automation Magazine (2011).



Parallel Tracking and Mapping for Small AR Workspaces

Georg Klein* David Murray†

Active Vision Laboratory
Department of Engineering Science
University of Oxford

ABSTRACT

This paper presents a method of estimating camera pose in an unknown scene. While this has previously been attempted by adapting SLAM algorithms developed for robotic exploration, we propose a system specifically designed to track a hand-held camera in a small AR workspace. We propose to split tracking and mapping into two separate tasks, processed in parallel threads on a dual-core computer: one thread deals with the task of robustly tracking erratic hand-held motion, while the other produces a 3D map of point features from previously observed video frames. This allows the use of computationally expensive batch optimisation techniques not usually associated with real-time operation: The result is a system that produces detailed maps with thousands of landmarks which can be tracked at frame-rate, with an accuracy and robustness rivalling that of state-of-the-art model-based systems.

1 INTRODUCTION

The majority of Augmented Reality (AR) systems operate with prior knowledge of the user's environment - i.e, some form of map. This could be a map of a city, CAD model of a component requiring maintenance, or even a sparse map of fiducials known to be present in the scene. The application then allows the user to interact with this environment based on prior information on salient parts of this model (e.g. "This location is of interest" or "remove this nut from this component"). If the map or model provided is comprehensive, registration can be performed directly from it, and this is the common approach to camera-based AR tracking.

Unfortunately, a comprehensive map is often not available, often a small map of only an object of interest is available - for example, a single physical object in a room or a single fiducial marker. Tracking is then limited to the times when this known feature can be measured by some sensor, and this limits range and quality of registration. This has led to the development of a class of techniques known (in the AR context) as *extensible tracking* [21, 14, 4, 28, 2] in which the system attempts to add previously unknown scene elements to its initial map, and these then provide registration even when the original map is out of sensing range. In [4], the initial map is minimal, consisting only of a template which provides metric scale; later versions of this monocular SLAM algorithm can now operate without this initialisation template.

The logical extension of extensible tracking is to track in scenes without any prior map, and this is the focus of this paper. Specifically, we aim to track a calibrated hand-held camera in a previously unknown scene without any known objects or initialisation target, while building a map of this environment. Once a rudimentary map has been built, it is used to insert virtual objects into the scene, and these should be accurately registered to real objects in the environment.

Since we do not use a prior map, the system has no deep understanding of the user's environment and this precludes many task-based AR applications. One approach for providing the user with meaningful augmentations is to employ a *remote expert* [4, 16] who can annotate the generated map. In this paper we take a different approach: we treat the generated map as a sandbox in which virtual simulations can be created. In particular, we estimate a dominant plane (a virtual ground plane) from the mapped points - an example of this is shown in Figure 1 - and allow this to be populated with virtual characters. In essence, we would like to transform any flat (and reasonably textured) surface into a playing field for VR simulations (at this stage, we have developed a simple but fast-paced action game). The hand-held camera then becomes both a viewing device and a user interface component.

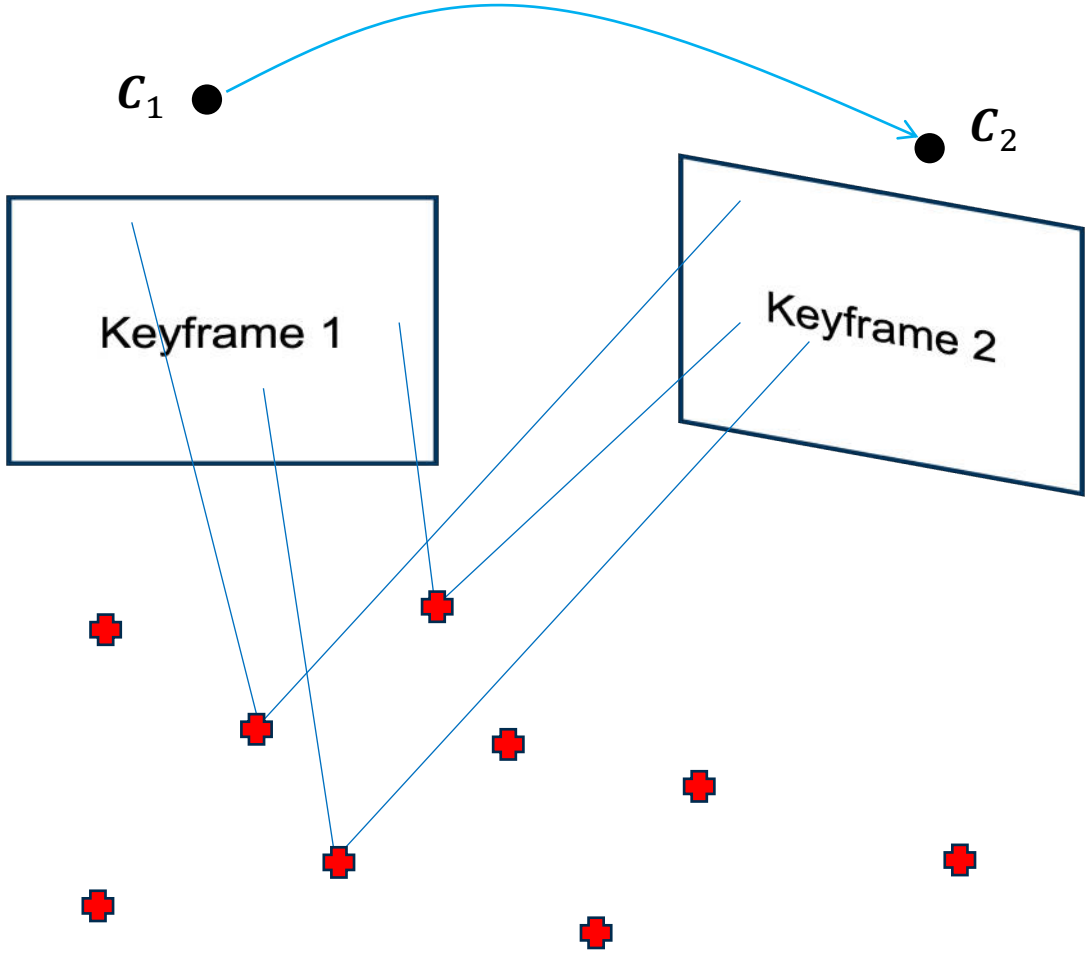
To further provide the user with the freedom to interact with the simulation, we require fast, accurate and robust camera tracking, all while refining the map and expanding it if new regions are explored. This is a challenging problem, and to simplify the task somewhat we have imposed some constraints on the scene to be tracked: it should be mostly *static*, i.e. not deformable, and it should be *small*. By small we mean that the user will spend most of his or her time in the same place: for example, at a desk, in one corner of a room, or in front of a single building. We consider this to be compatible with a large number of workspace-related AR applications, where the user is anyway often tethered to a computer. Exploratory tasks such as running around a city are not supported.

The next section outlines the proposed method and contrasts this to previous methods. Subsequent sections describe in detail the method used, present results and evaluate the method's performance.

*e-mail: gk@robots.ox.ac.uk
†e-mail:dwm@robots.ox.ac.uk


Grow
Grow!
Reset
Pause
Teachers
F
G
C
V
B
N
M

PTAM Map Initialisation



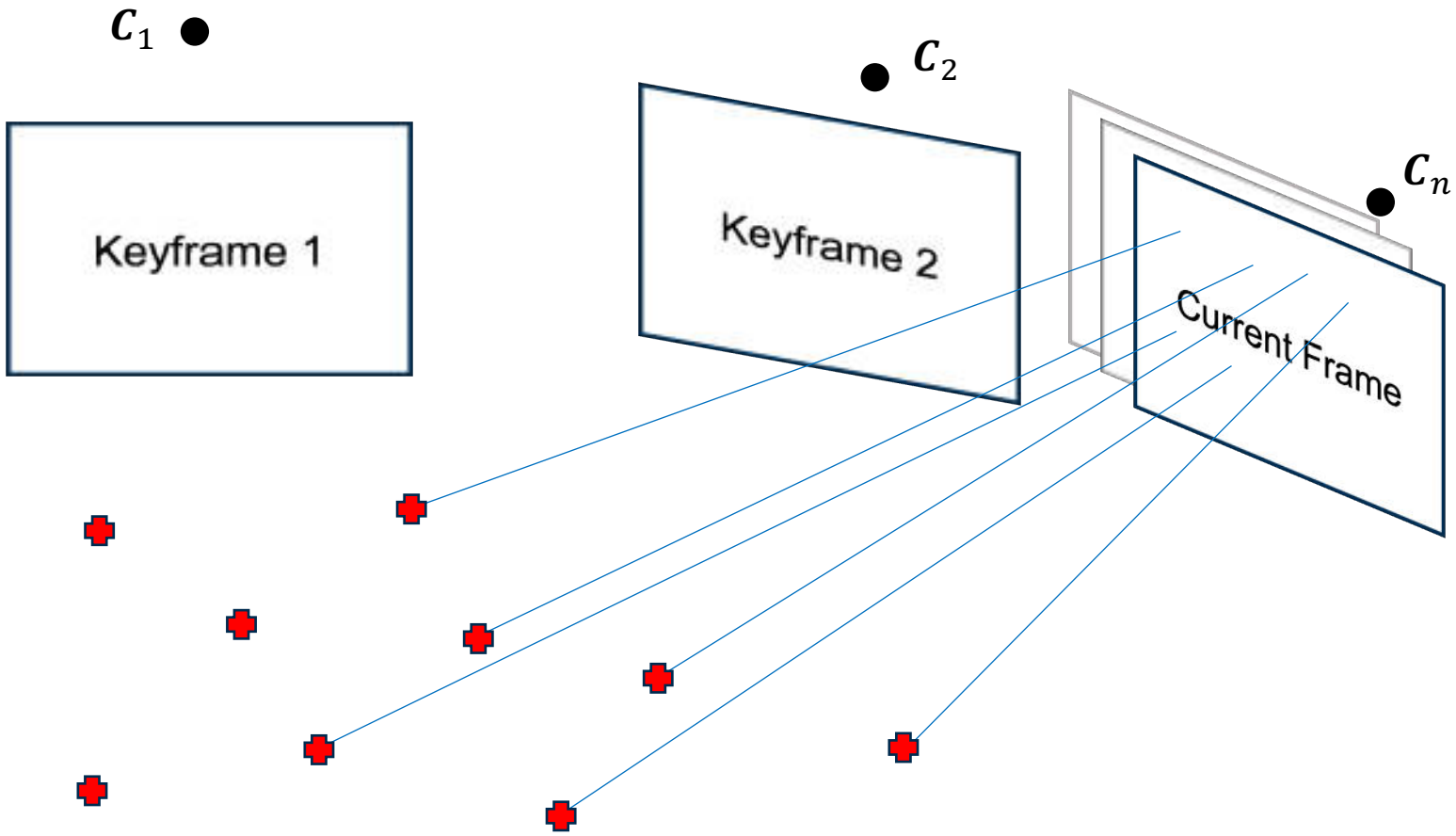
- Find 1000 interest points
- Move the camera, track points

- Five-point Essential Matrix estimation with RANSAC
- Triangulate 3D points
- Refinement with Bundle Adjustment
- Scale factor: assume 10cm movement

- Find dominant plane (RANSAC), align map so that dominant plane aligns with XY plane (z=0)

CRICOS No.00213J

PTAM Tracking



Tracking:

Update current pose based on 3D-2D correspondences.

CRICOS No.00213J

PTAM Tracking

5.2 Camera pose and projection

To project map points into the image plane, they are first transformed from the world coordinate frame to the camera-centred coordinate frame $\mathcal{C}$. This is done by left-multiplication with a 4×4 matrix denoted $E_{\mathcal{CW}}$, which represents camera pose.

$$\boldsymbol{p}_{j\mathcal{C}} = E_{\mathcal{CW}}\boldsymbol{p}_{j\mathcal{W}} \tag{1}$$

The subscript $\mathcal{CW}$ may be read as "frame $\mathcal{C}$ from frame $\mathcal{W}$". The matrix $E_{\mathcal{CW}}$ contains a rotation and a translation component and is a member of the Lie group SE(3), the set of 3D rigid-body transformations.

To project points in the camera frame into image, a calibrated camera projection model CamProj() is used:

$$\begin{pmatrix} u_i \\ v_i \end{pmatrix} = \mathrm{CamProj}(E_{\mathcal{CW}}\boldsymbol{p}_{i\mathcal{W}}) \tag{2}$$

We employ a pin-hole camera projection function which supports lenses exhibiting barrel radial distortion. The radial distortion model which transforms $r \rightarrow r'$ is the FOV-model of [6]. The camera parameters for focal length (f_u, f_v), principal point (u_0, v_0) and distortion (ω) are assumed to be known:

$$\mathrm{CamProj}\begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} u_0 \\ v_0 \end{pmatrix} + \begin{bmatrix} f_u & 0 \\ 0 & f_v \end{bmatrix} \frac{\mathrm{r}'}{\mathrm{r}} \begin{pmatrix} \frac{x}{z} \\ \frac{y}{z} \end{pmatrix} \tag{3}$$

$$r = \sqrt{\frac{x^2 + y^2}{z^2}} \tag{4}$$

$$r' = \frac{1}{\omega}\arctan(2r\tan\frac{\omega}{2}) \tag{5}$$

PTAM Tracking

5.4 Pose update

Given a set S of successful patch observations, a camera pose update can be computed. Each observation yields a found patch position $(\hat{u}\ \hat{v})^T$ (referred to level zero pixel units) and is assumed to have measurement noise of $\sigma^2 = 2^{2l}$ times the 2×2 identity matrix (again in level zero pixel units). The pose update is computed iteratively by minimising a robust objective function of the reprojection error:

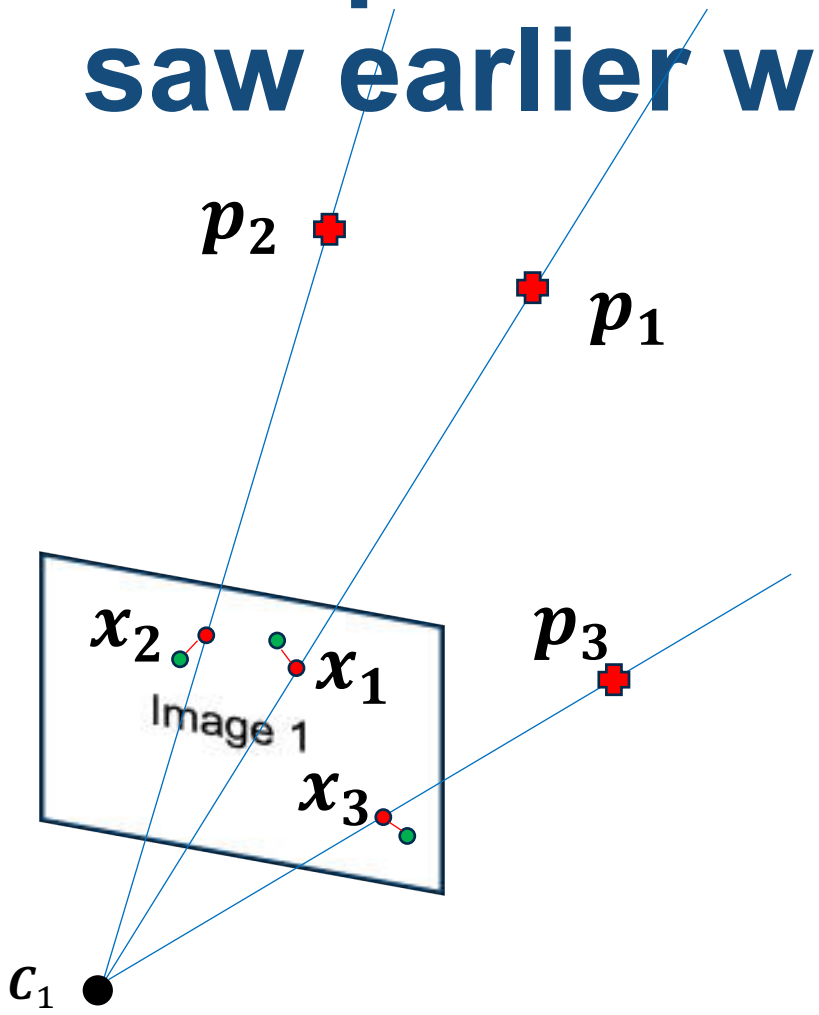
$$\boldsymbol{\mu}' = \underset{\boldsymbol{\mu}}{\operatorname{argmin}} \sum_{j \in S} \operatorname{Obj}\left(\frac{|\boldsymbol{e}_j|}{\sigma_j}, \sigma_T\right) \tag{8}$$

where $\boldsymbol{e}_j$ is the reprojection error vector:

$$\boldsymbol{e}_j = \begin{pmatrix} \hat{u}_j \\ \hat{v}_j \end{pmatrix} - \operatorname{CamProj}\left(\exp\left(\boldsymbol{\mu}\right) E_{\mathcal{CW}} \boldsymbol{p}_j\right). \tag{9}$$

$\operatorname{Obj}(\cdot, \sigma_T)$ is the Tukey biweight objective function [13] and σ_T a robust (median-based) estimate of the distribution's standard deviation derived from all the residuals. We use ten iterations of reweighted least squares to allow the M-estimator to converge from any one set of measurements.

Compare the “generic” loss function we saw earlier with PTAM’s approach.



$$\text{argmin}_{R,t} \sum_i \left\| x_\tau^i - K(R \cdot p^i{}_{\tau-1} + t) \right\|^2$$

vs

The pose update is computed iteratively by minimising a robust objective function of the reprojection error:

$$\mu' = \underset{\mu}{\text{argmin}} \sum_{j \in S} \text{Obj}\left(\frac{|e_j|}{\sigma_j}, \sigma_T\right) \tag{8}$$

where e_j is the reprojection error vector:

$$e_j = \begin{pmatrix} \hat{u}_j \\ \hat{v}_j \end{pmatrix} - \text{CamProj}\left(\exp(\mu)\, E_{CW} p_j\right). \tag{9}$$

$\text{Obj}(\cdot, \sigma_T)$ is the Tukey biweight objective function [13] and σ_T a robust (median-based) estimate of the distribution’s standard deviation derived from all the residuals. We use ten iterations of reweighted least squares to allow the M-estimator to converge from any one set of measurements.

CRICOS No.00213J

Compare the “generic” loss function we saw earlier with PTAM’s approach.

$$\text{argmin}_{\boldsymbol{R},\boldsymbol{t}} \sum_i \left\| \boldsymbol{x}^i_{\boldsymbol{\tau}} - \boldsymbol{K}(\boldsymbol{R} \cdot \boldsymbol{p}^i{}_{\tau-1} + \boldsymbol{t}) \right\|^2$$

vs

Component	PTAM Tracking
Parameters	$\boldsymbol{R}, \boldsymbol{t}$ but expressed as a vector $\boldsymbol{\mu}$
Prediction	Projected 2D point, using lens distortion
Residual / Error	Reprojection Error
Objective	Tukey biweight function

The pose update is computed iteratively by minimising a robust objective function of the reprojection error:

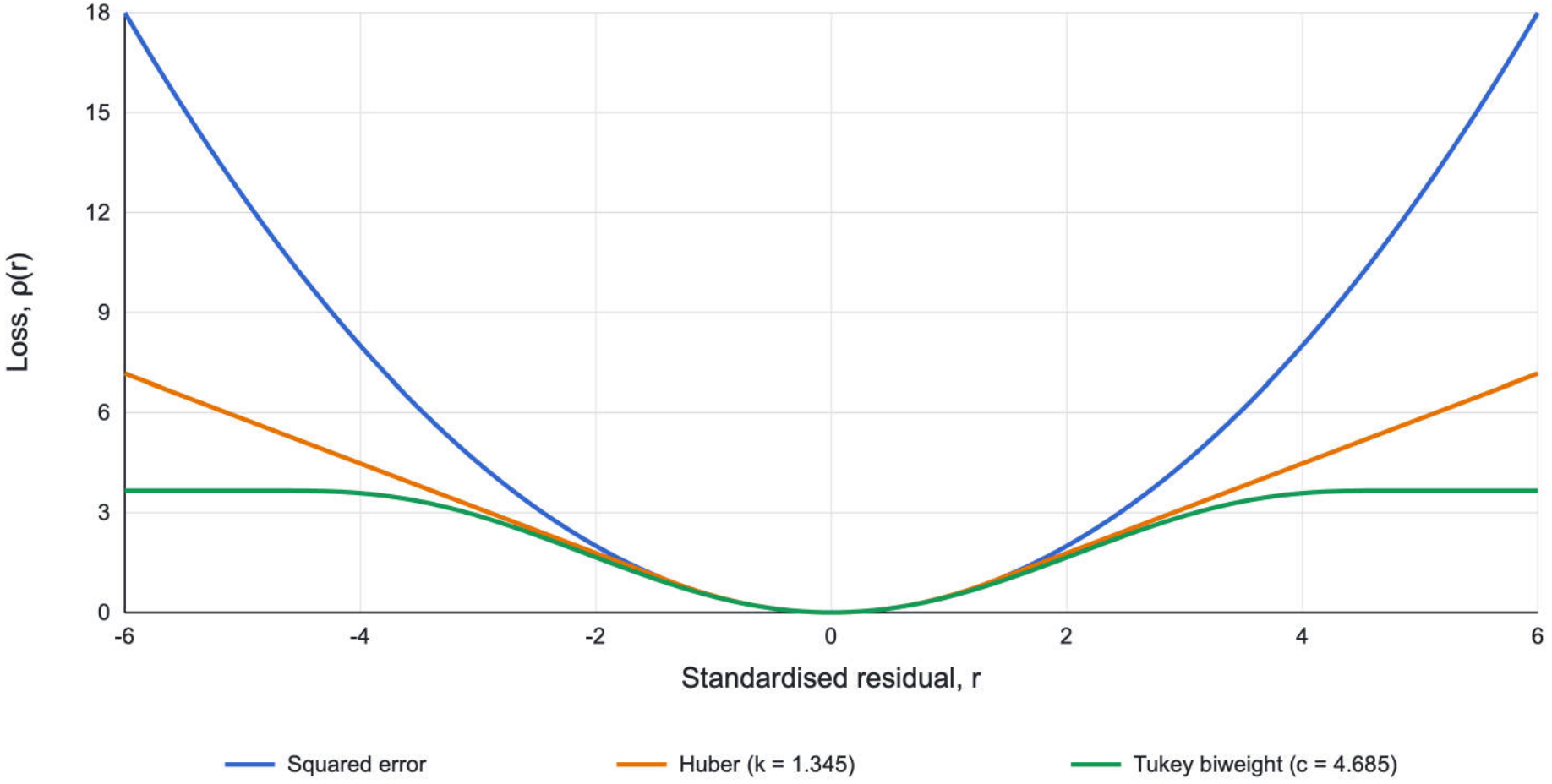
$$\boldsymbol{\mu}' = \underset{\boldsymbol{\mu}}{\text{argmin}} \sum_{j \in S} \text{Obj}\left(\frac{|\boldsymbol{e}_j|}{\sigma_j}, \sigma_T \right) \quad (8)$$

where $\boldsymbol{e}_j$ is the reprojection error vector:

$$\boldsymbol{e}_j = \begin{pmatrix} \hat{u}_j \\ \hat{v}_j \end{pmatrix} - \text{CamProj}\left(\exp(\boldsymbol{\mu})\, E_{\mathcal{CW}} \boldsymbol{p}_j\right). \quad (9)$$

$\text{Obj}(\cdot, \sigma_T)$ is the Tukey biweight objective function [13] and σ_T a robust (median-based) estimate of the distribution’s standard deviation derived from all the residuals. We use ten iterations of reweighted least squares to allow the M-estimator to converge from any one set of measurements.

Robust Objective Functions (M-Estimators)



Huber: switches from quadratic to linear behaviour for r>k

Tukey: starts quadratic but gradually flattens and then stays constant

M-Estimators

Category	Examples	Interpretation
Unbounded influence	Quadratic	Outliers can dominate the estimate
Bounded influence	L1, Huber, pseudo-Huber, Fair	Limits outlier influence but never completely ignores observations
Redescending influence	Tukey, Cauchy, Hampel, Welsch, Andrews	Influence decreases for increasingly extreme residuals
Finite rejection point	Tukey, Hampel, Andrews	Residuals beyond a threshold receive exactly zero influence

"Influence": how strongly an observation with a given residual can pull the fitted model toward itself.

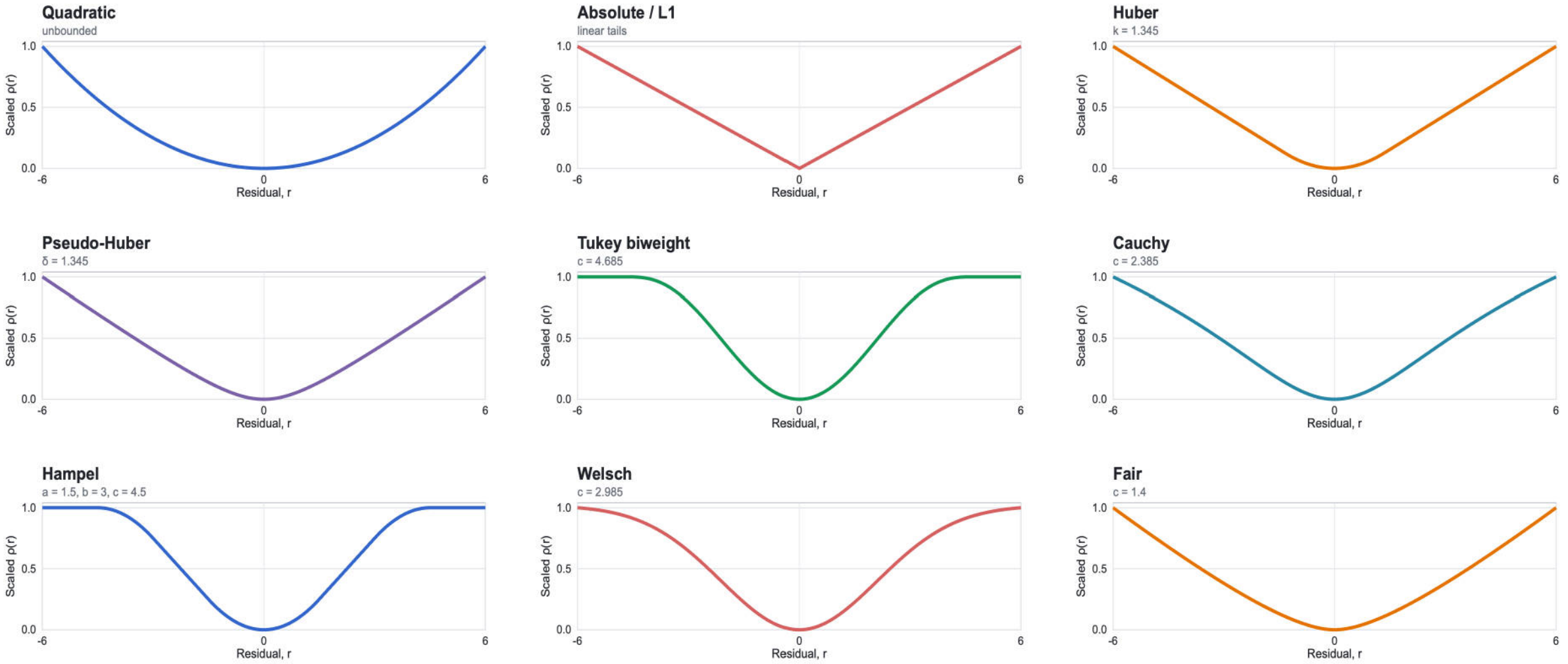
CRICOS No.00213J

M-estimator	Behaviour	Summary
Quadratic / squared error	Loss increases with the square of the residual. The influence of an observation increases without limit.	Ordinary least squares. Very efficient for normally distributed errors, but highly sensitive to outliers.
Absolute error / L1 / LAD	Loss increases linearly with the absolute residual. Large residuals have a fixed level of influence.	More robust than squared error. Results in estimating the median instead of the mean.
Huber	Uses squared error for small residuals and linear loss beyond a chosen threshold.	A widely used compromise between robustness, efficiency and computational stability. The common threshold is 1.345.
Pseudo-Huber	Provides a smooth transition from squared behaviour near zero to approximately linear behaviour for large residuals.	A differentiable approximation to Huber loss, useful in gradient-based optimisation.
Tukey biweight	Loss gradually flattens and becomes constant beyond a chosen cutoff. Influence falls to zero at the cutoff.	Completely ignores sufficiently extreme residuals. The common cutoff is 4.685.
Cauchy / Lorentzian	Loss grows slowly for large residuals. Their influence gradually decreases toward zero.	Useful for heavy-tailed data and smoothly reduces the effect of extreme observations.
Hampel	Uses three thresholds. Influence initially increases, becomes constant, and then decreases to zero.	Offers flexible outlier treatment, but requires three tuning choices.
Welsch / Leclerc	Loss smoothly approaches a maximum. The influence of extreme residuals rapidly approaches zero.	Provides strong, smooth suppression of outliers but can be sensitive to the starting estimate.
Fair	Behaves like squared error near zero and approximately like absolute error for large residuals.	A smooth alternative to Huber with a gradual transition between the two regions.
Andrews' sine	Uses a smooth sine-shaped loss within an acceptance range. Influence returns to zero at the cutoff.	Smoothly rejects sufficiently extreme residuals, but is less commonly used than Tukey biweight.

CRICOS No.00213J

M-estimator loss functions

Residual range −6 to 6; each loss independently scaled to a maximum of 1



PTAM Tracking

A fundamental requirement of the tracking (and also the mapping) system is the ability to differentiate Eq. 2 with respect to changes in camera pose $E_{\mathcal{CW}}$. Changes to camera pose are represented by left-multiplication with a 4×4 camera motion M:

$$E'_{\mathcal{CW}} = ME_{\mathcal{CW}} = \exp(\boldsymbol{\mu})E_{\mathcal{CW}} \tag{6}$$

where the camera motion is also a member of SE(3) and can be minimally parametrised with a six-vector $\boldsymbol{\mu}$ using the exponential map. Typically the first three elements of $\boldsymbol{\mu}$ represent a translation and the latter three elements represent a rotation axis and magnitude. This representation of camera state and motion allows for trivial differentiation of Eq. 6, and from this, partial differentials of Eq. 2 of the form $\frac{\partial u}{\partial \mu_i}, \frac{\partial v}{\partial \mu_i}$ are readily obtained in closed form. Details of the Lie group SE(3) and its representation may be found in [29].

5.4 Pose update

Given a set S of successful patch observations, a camera pose update can be computed. Each observation yields a found patch position $(\hat{u}\ \hat{v})^T$ (referred to level zero pixel units) and is assumed to have measurement noise of $\sigma^2 = 2^{2l}$ times the 2×2 identity matrix (again in level zero pixel units). The pose update is computed iteratively by minimising a robust objective function of the reprojection error:

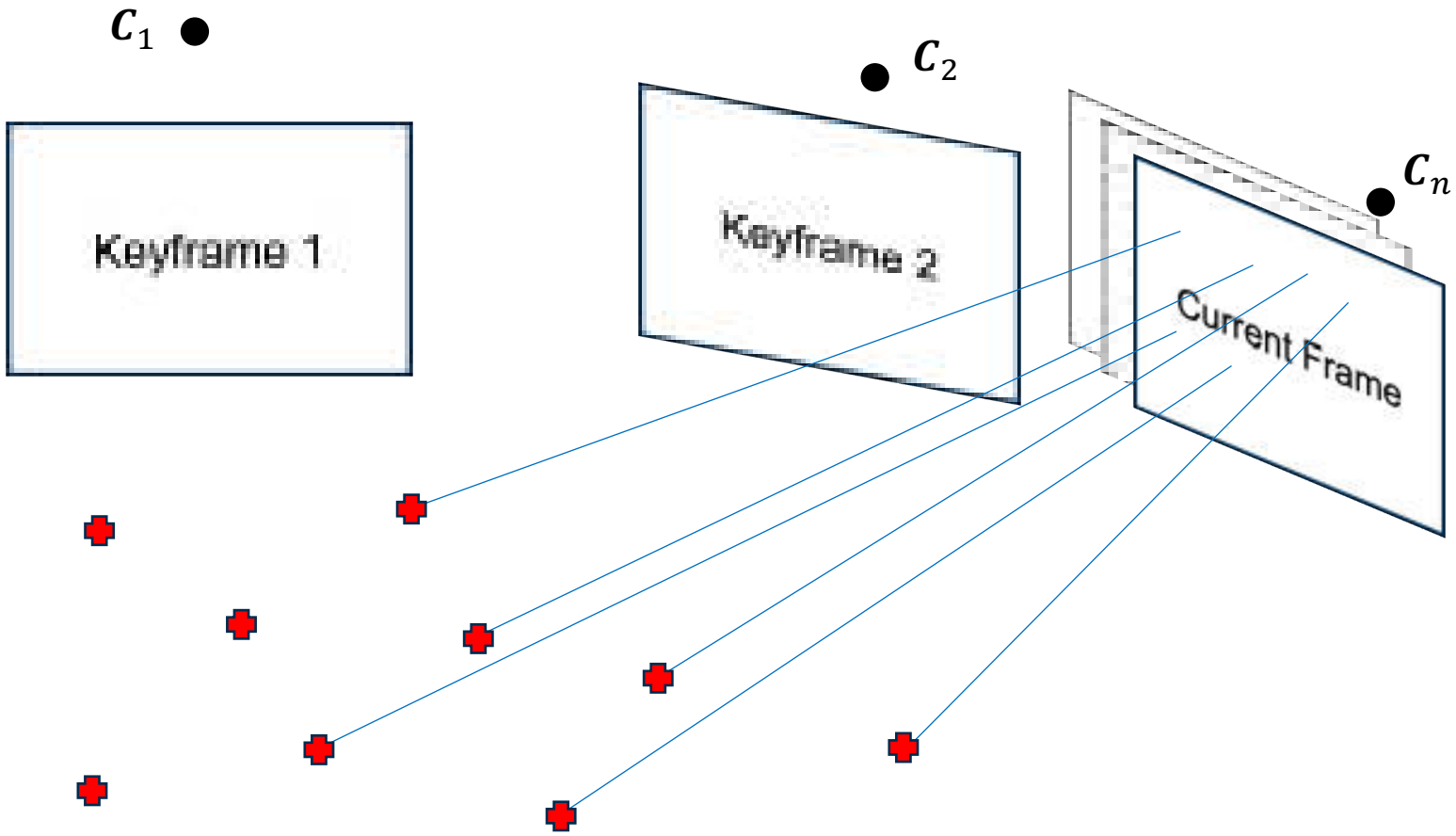
$$\boldsymbol{\mu}' = \underset{\boldsymbol{\mu}}{\operatorname{argmin}} \sum_{j \in S} \operatorname{Obj}\left(\frac{|\boldsymbol{e}_j|}{\sigma_j}, \sigma_T\right) \tag{8}$$

where $\boldsymbol{e}_j$ is the reprojection error vector:

$$\boldsymbol{e}_j = \begin{pmatrix} \hat{u}_j \\ \hat{v}_j \end{pmatrix} - \operatorname{CamProj}\left(\exp\left(\boldsymbol{\mu}\right) E_{\mathcal{CW}} \boldsymbol{p}_j\right). \tag{9}$$

$\operatorname{Obj}(\cdot, \sigma_T)$ is the Tukey biweight objective function [13] and σ_T a robust (median-based) estimate of the distribution's standard deviation derived from all the residuals. We use ten iterations of reweighted least squares to allow the M-estimator to converge from any one set of measurements.

PTAM Tracking

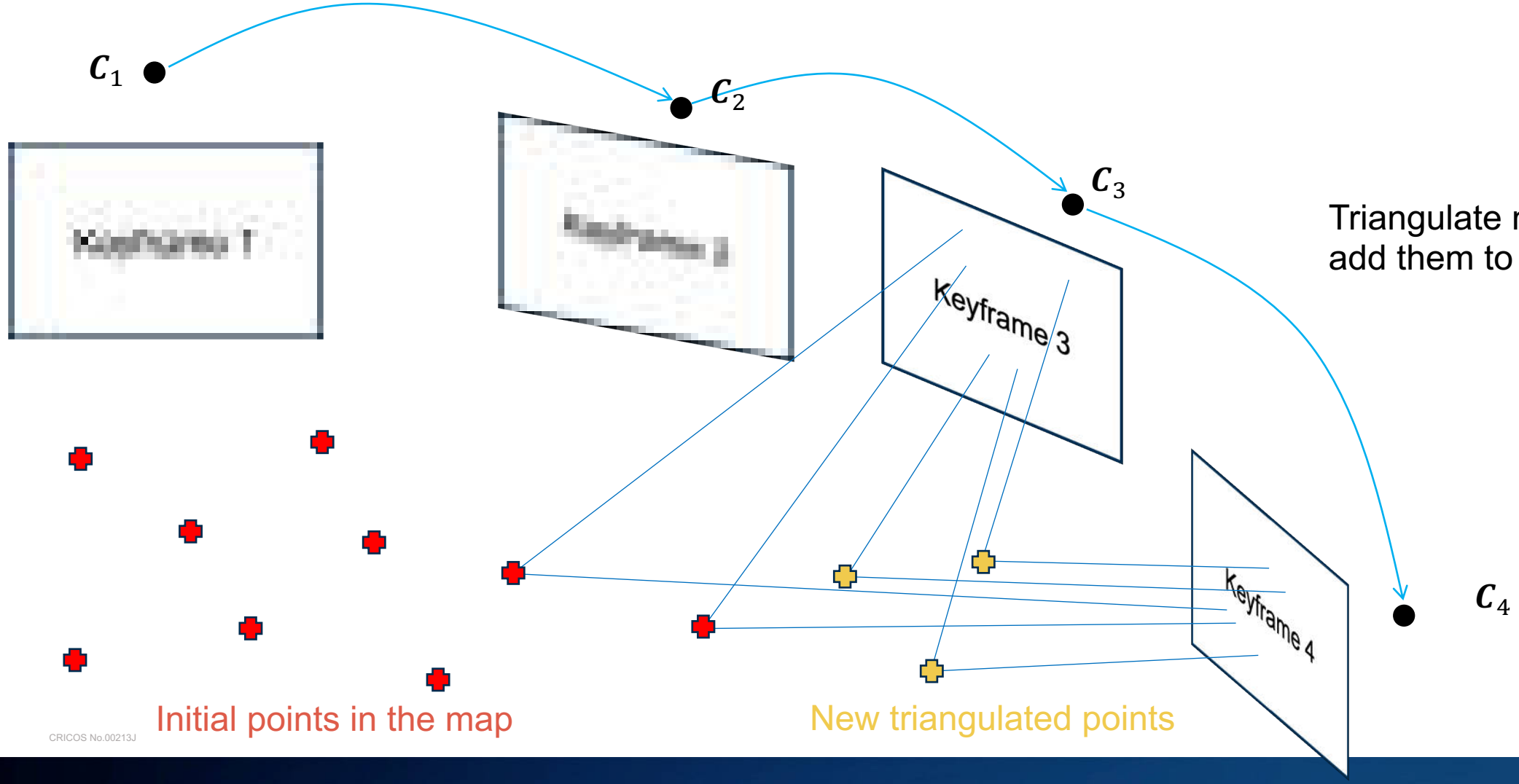


Tracking:

Update current pose based on 3D-2D correspondences.

CRICOS No.00213J

PTAM Map Update



Triangulate new points and add them to the map.

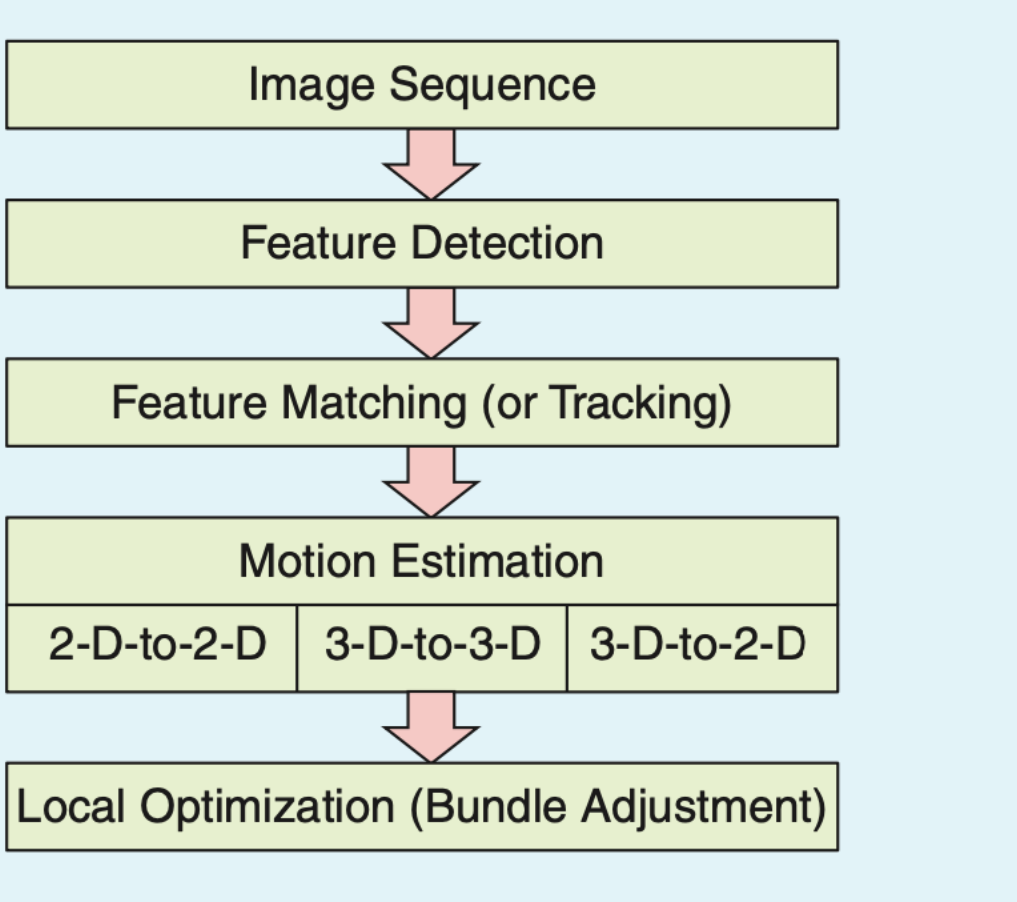
CRICOS No.00213J


Grow
Grow!
Reset
Pause
Teachers
F
G
C
V
B
N
M



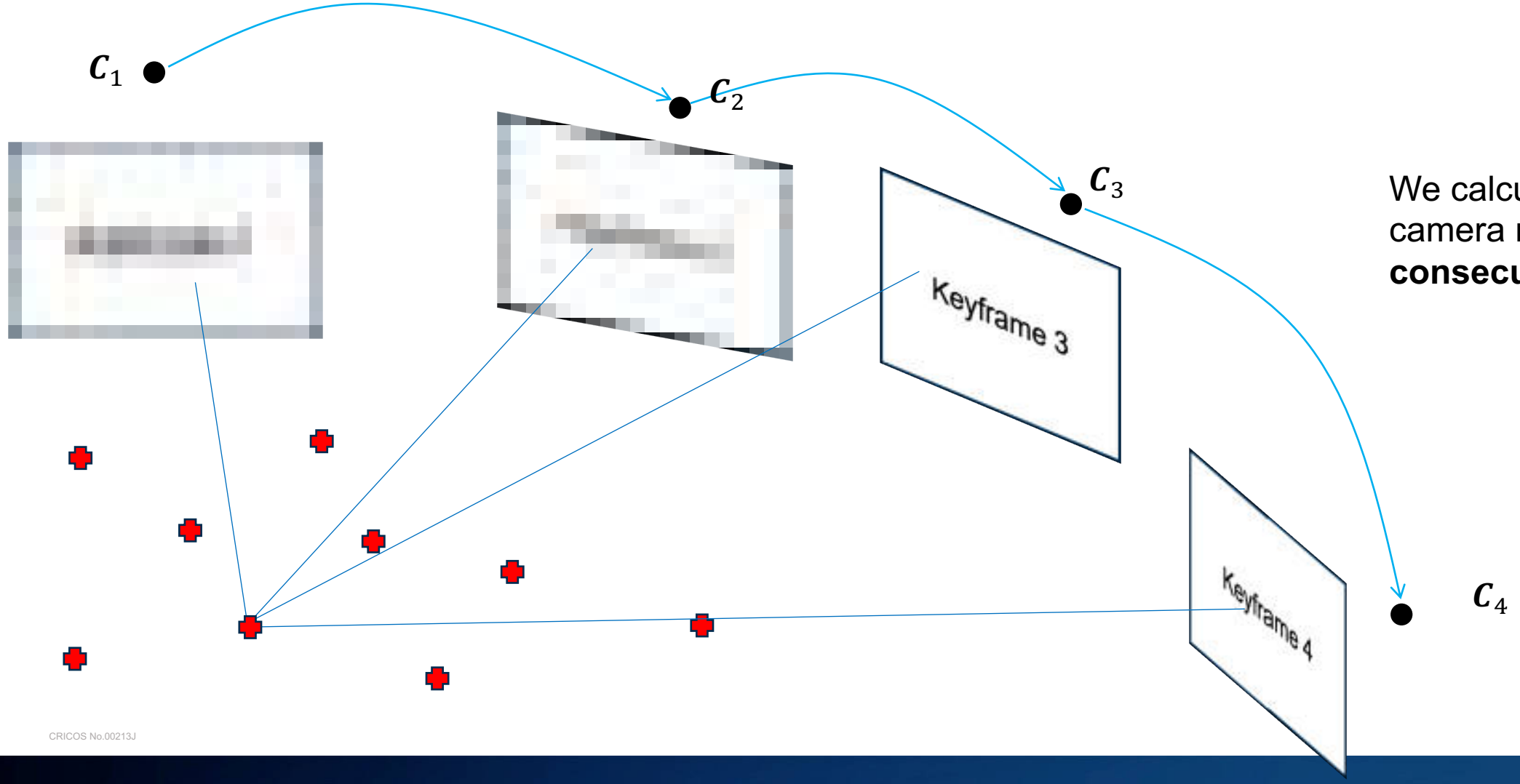
Local Optimisation
(Pose Graphs and Bundle Adjustment)

Recap – What we learned so far



CRICOS No.00213J

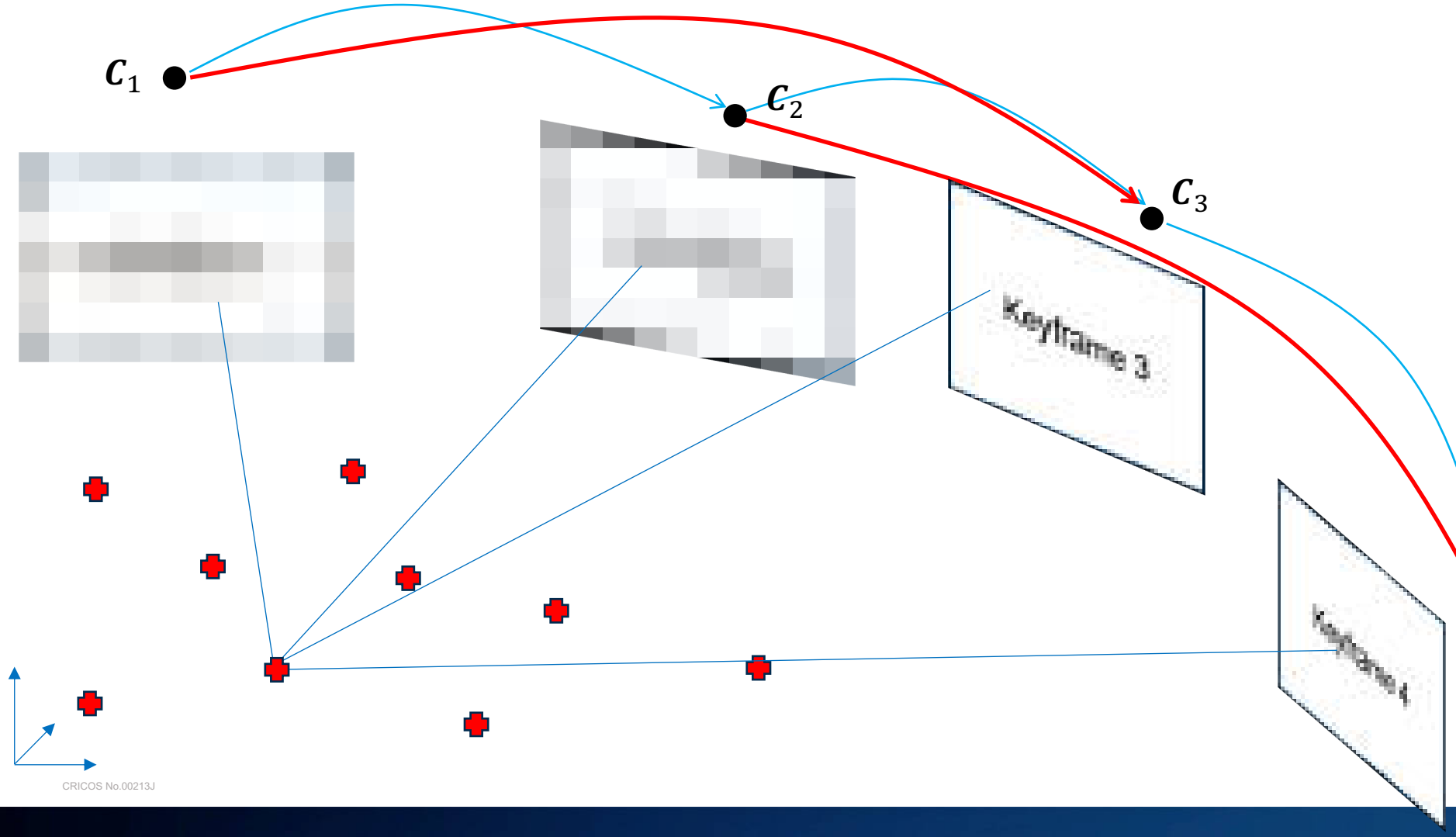
Pose Graph Optimisation



We calculated relative camera motion between **consecutive** frames.

CRICOS No.00213J

Pose Graph Optimisation

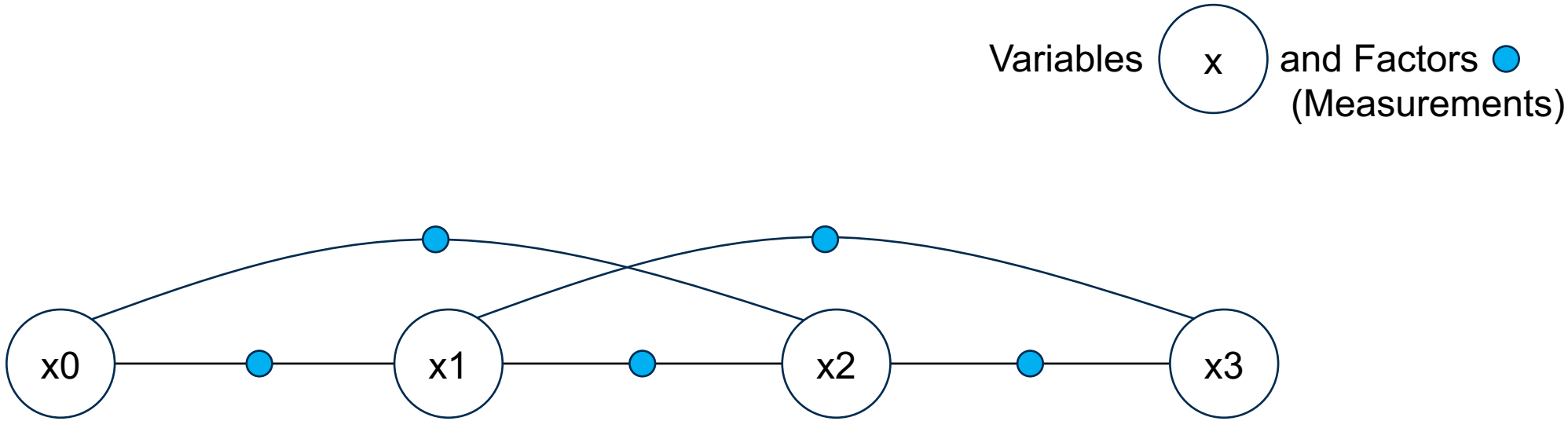


We calculated relative camera motion between **consecutive** frames.

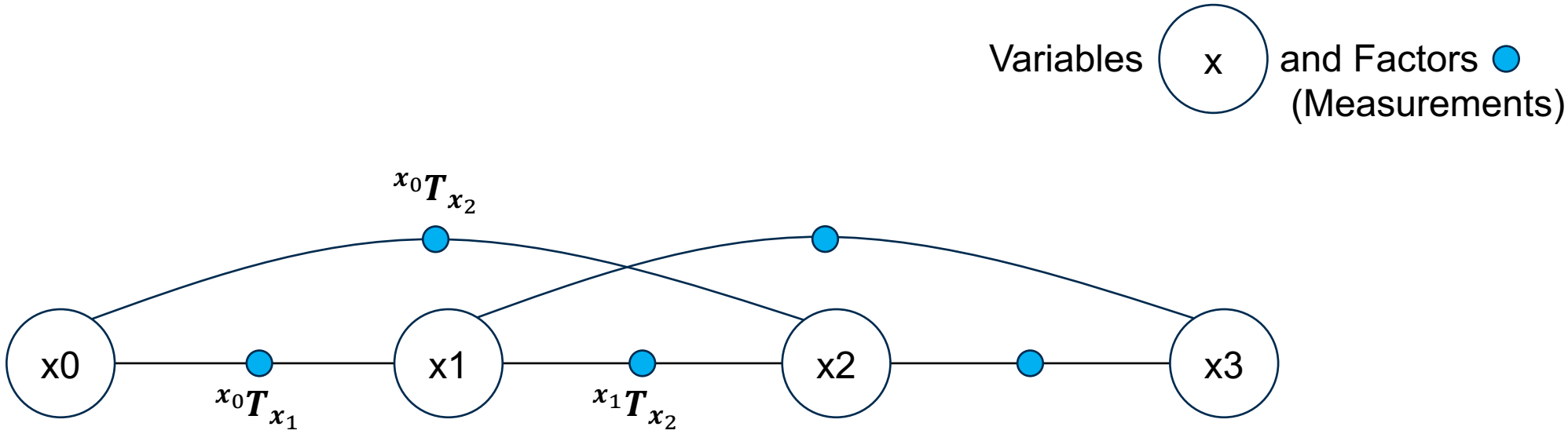
But we can also do that for **non-consecutive** frames!

CRICOS No.00213J

Pose Graph / Factor Graph

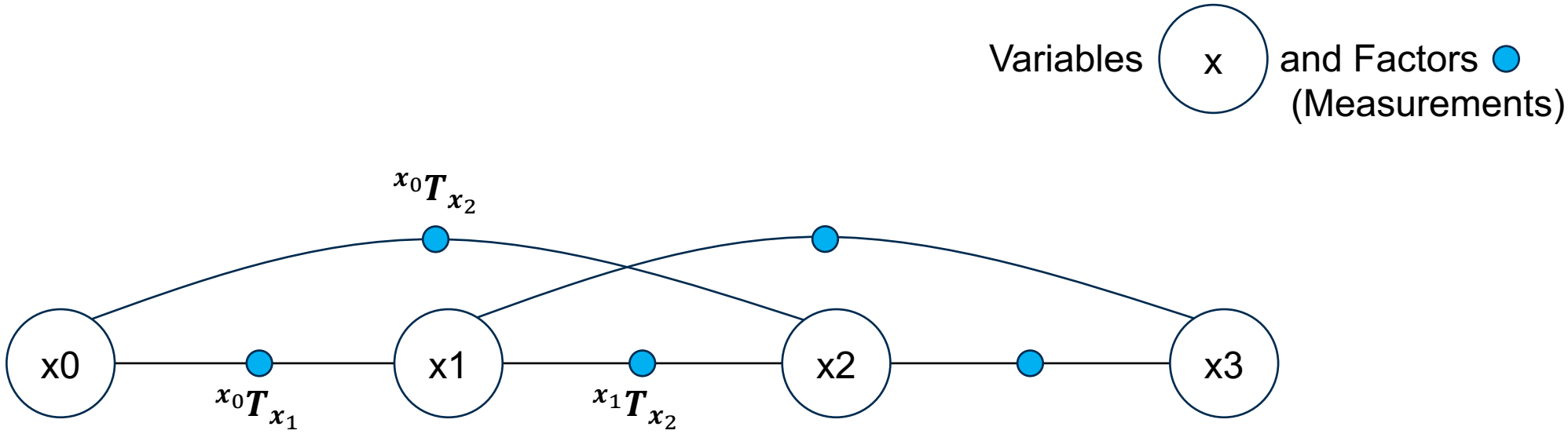


Pose Graph / Factor Graph



CRICOS No.00213J

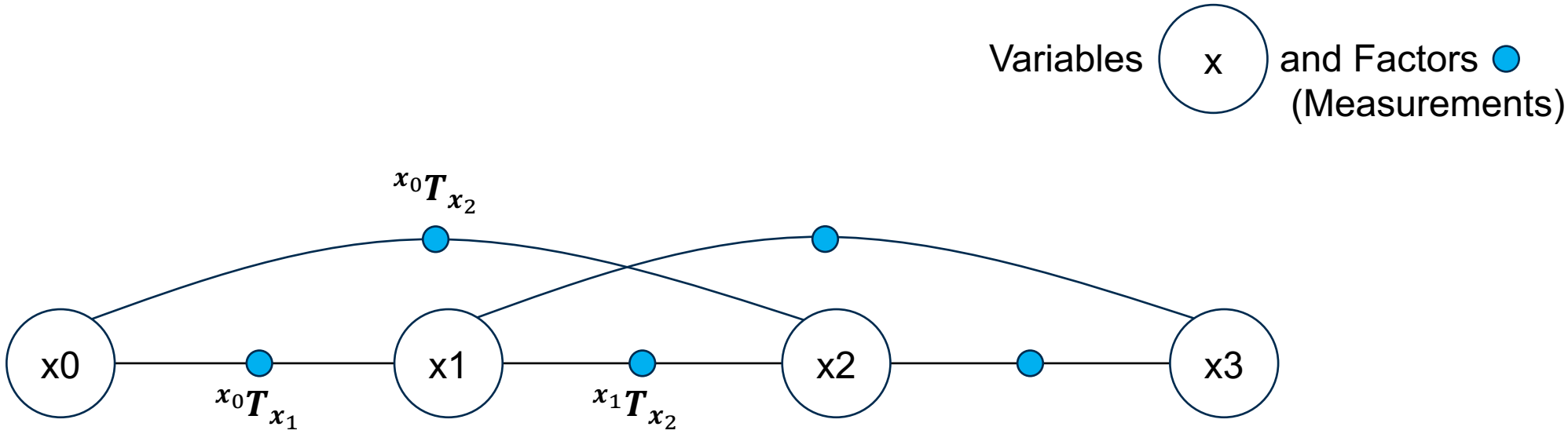
Pose Graph / Factor Graph



What if the result of ${}^{x_0}\boldsymbol{T}_{x_1} \cdot {}^{x_1}\boldsymbol{T}_{x_2}$ is not the same as ${}^{x_0}\boldsymbol{T}_{x_2}$?
What if we have different confidence in the "measurements" ${}^{x_i}\boldsymbol{T}_{x_j}$?

CRICOS No.00213J

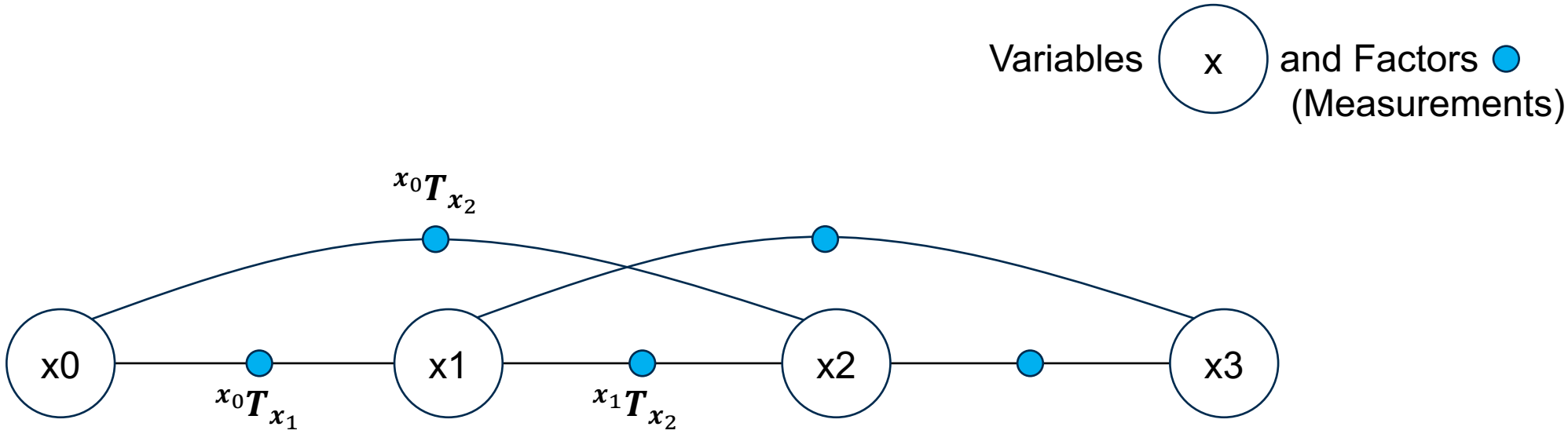
Pose Graph / Factor Graph



Solve by formulating it as an **optimisation problem** over the set of poses $\boldsymbol{x_i}$.

$$\boldsymbol{X}^* = \underset{\boldsymbol{X}}{\operatorname{argmin}} f(\boldsymbol{X}, \boldsymbol{T})$$

Pose Graph / Factor Graph

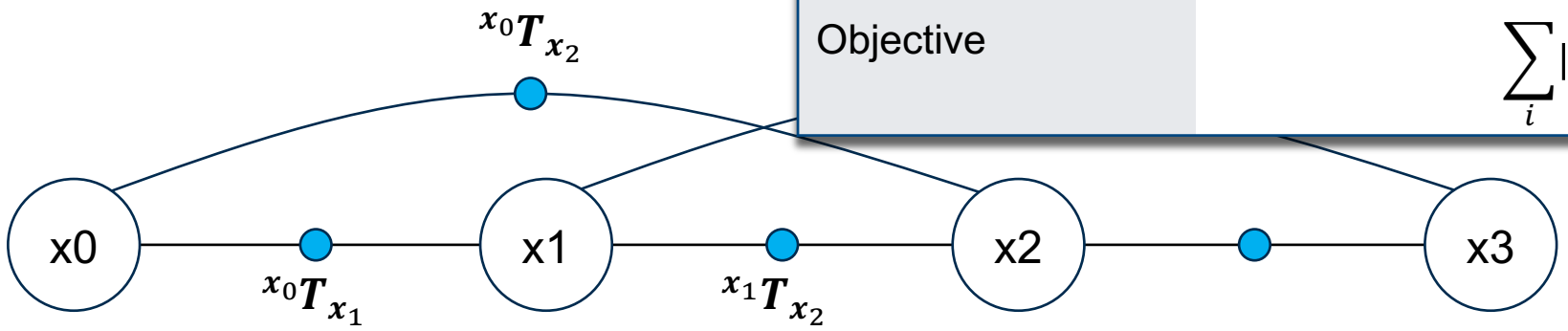


Solve by formulating it as an **optimisation problem** over the set of poses $\boldsymbol{x_i}$.

$$\boldsymbol{X}^* = \underset{\boldsymbol{X}}{\operatorname{argmin}} \sum_{i,j} \left\| \boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j \right\|^2$$

Pose Graph / Factor

Component	Pose Graph
Parameters	$\boldsymbol{X} = \{\boldsymbol{R_i}, \boldsymbol{t_i}\}$
Prediction	Transformed Poses $^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j$
Residual / Error	Differences between Poses $\boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j$
Objective	sum of squared errors $\sum_i \| \cdots \|^2$

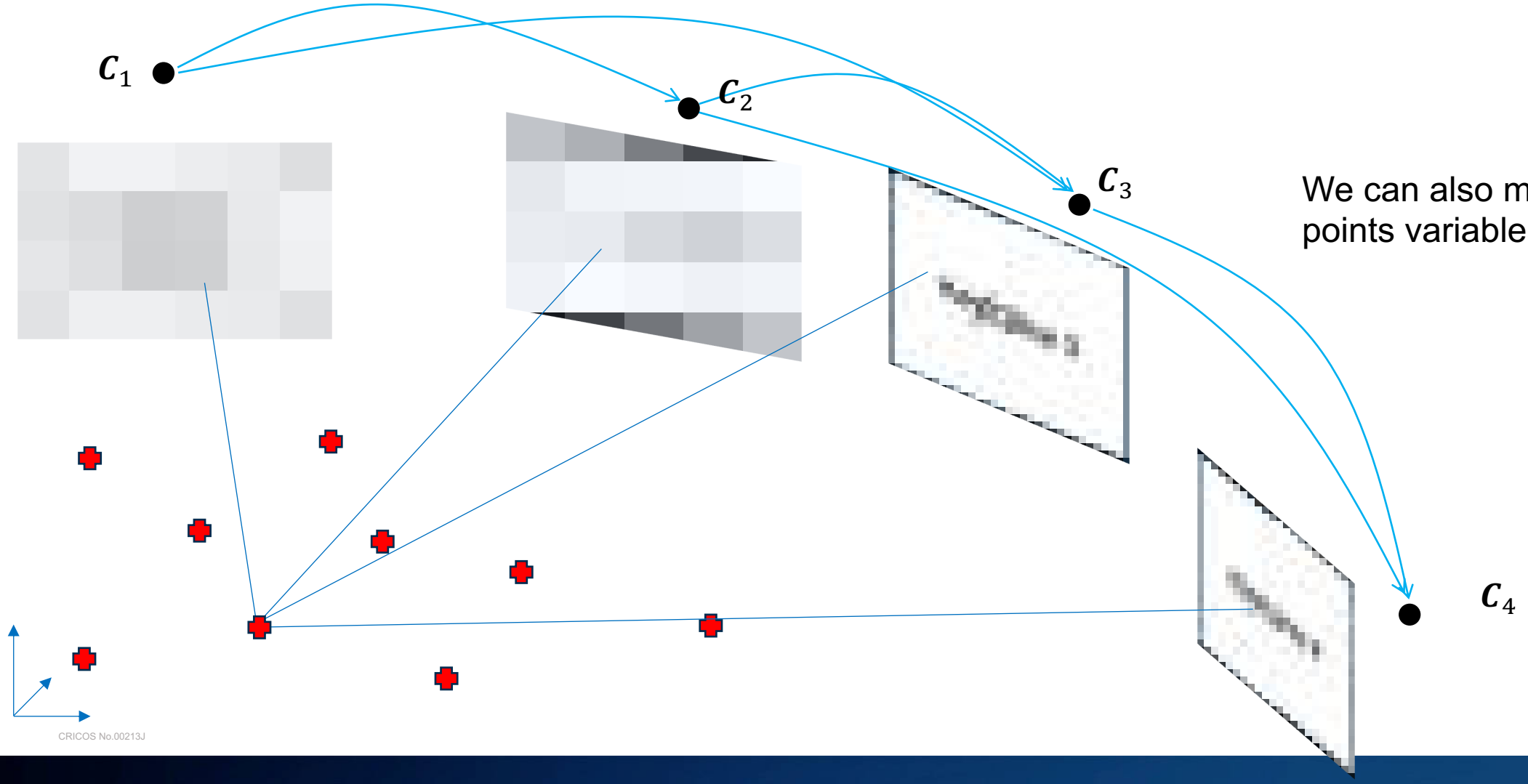


Solve by formulating it as an **optimisation problem** over the set of poses $\boldsymbol{x}_i$.

$$\boldsymbol{X}^* = \underset{\boldsymbol{X}}{\operatorname{argmin}} \sum_{i,j} \left\| \boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j \right\|^2$$

This minus sign seems innocuous but is more involved than it seems.
We are taking the difference between two SE3s!

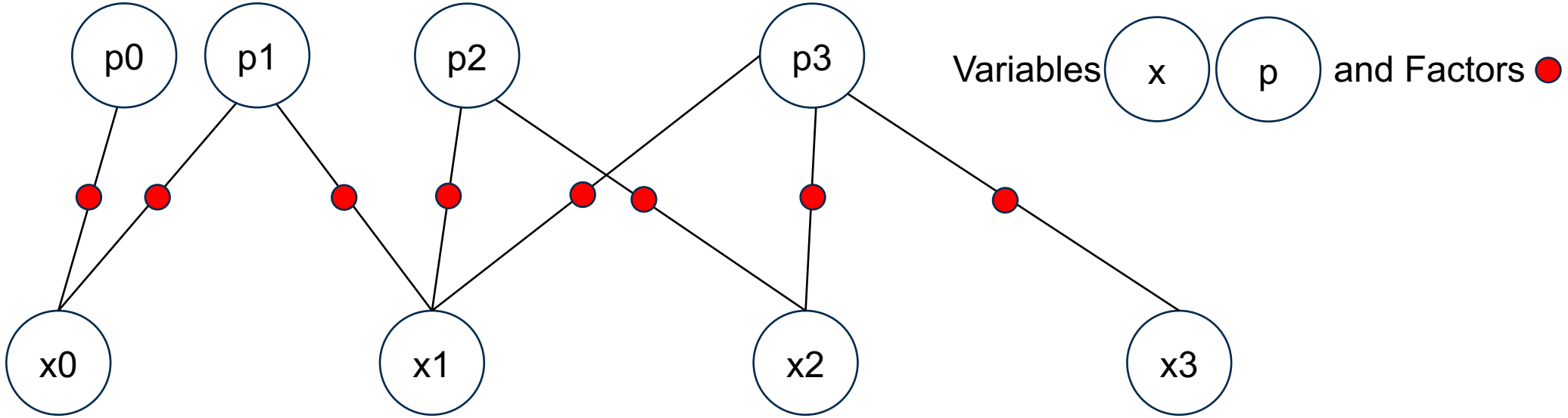
Bundle Adjustment



We can also make the 3D points variables of the problem.

CRICOS No.00213J

Bundle Adjustment



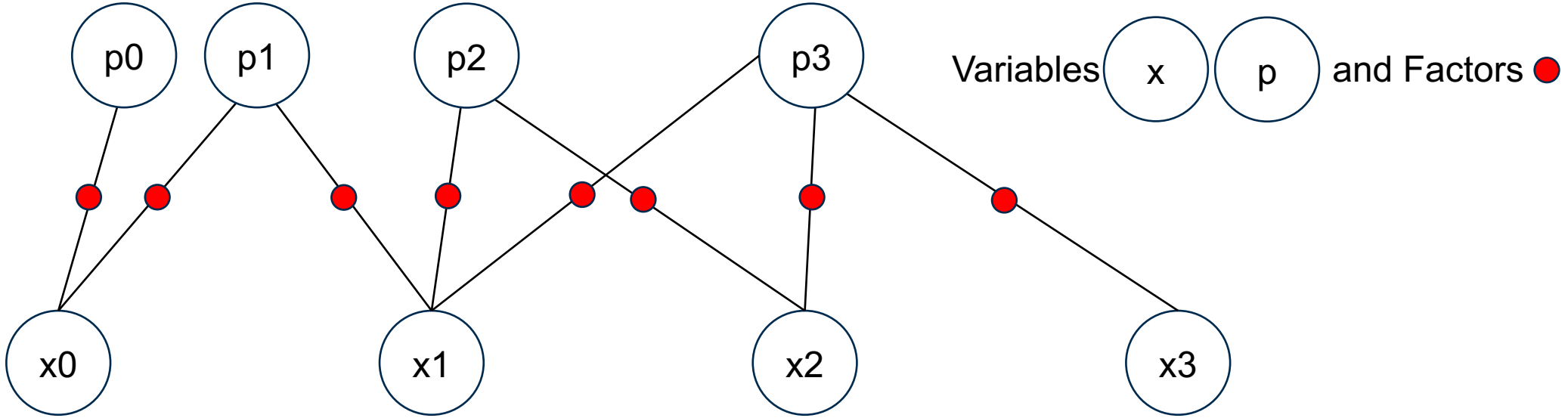
Optimise an error function (reprojection error) jointly over camera poses **x** and 3D point coordinates **p**.

*"What are the optimal camera poses **x** and 3D point coordinates **p**, given the observations **y**?"*

$$\boldsymbol{P}^*, \boldsymbol{X}^* = \underset{\boldsymbol{P},\boldsymbol{X}}{\operatorname{argmin}} f(\boldsymbol{X}, \boldsymbol{P}, \boldsymbol{Y})$$

CRICOS No.00213J

Bundle Adjustment



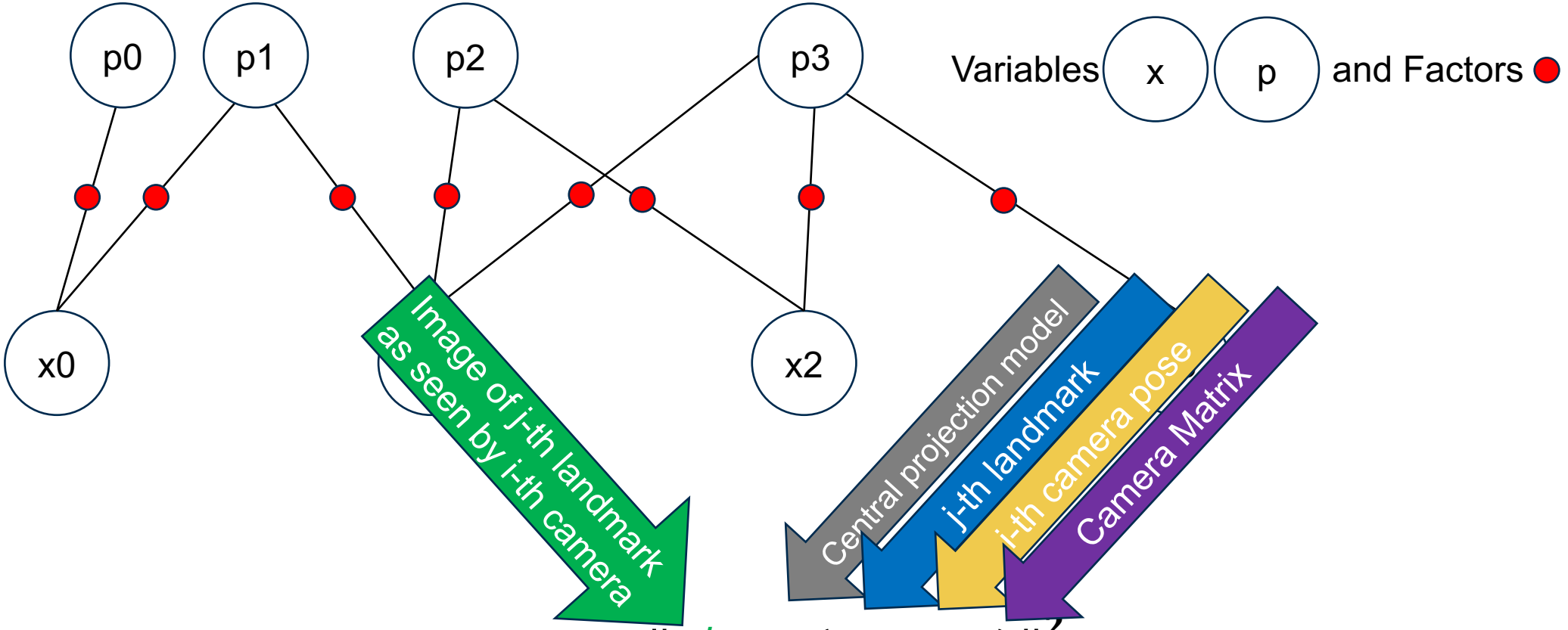
Optimise an error function (reprojection error) jointly over camera poses **x** and 3D point coordinates **p**.

*"What are the optimal camera poses **x** and 3D point coordinates **p**, given the observations **y**?"*

$$\boldsymbol{P}^*, \boldsymbol{X}^* = \underset{\boldsymbol{P},\boldsymbol{X}}{\operatorname{argmin}} \sum_{i,j} \left\| \boldsymbol{y}_j^i - \pi\left(\boldsymbol{p}_j, \boldsymbol{x}_i, \boldsymbol{K}\right) \right\|^2$$

CRICOS No.00213J

Bundle Adjustment

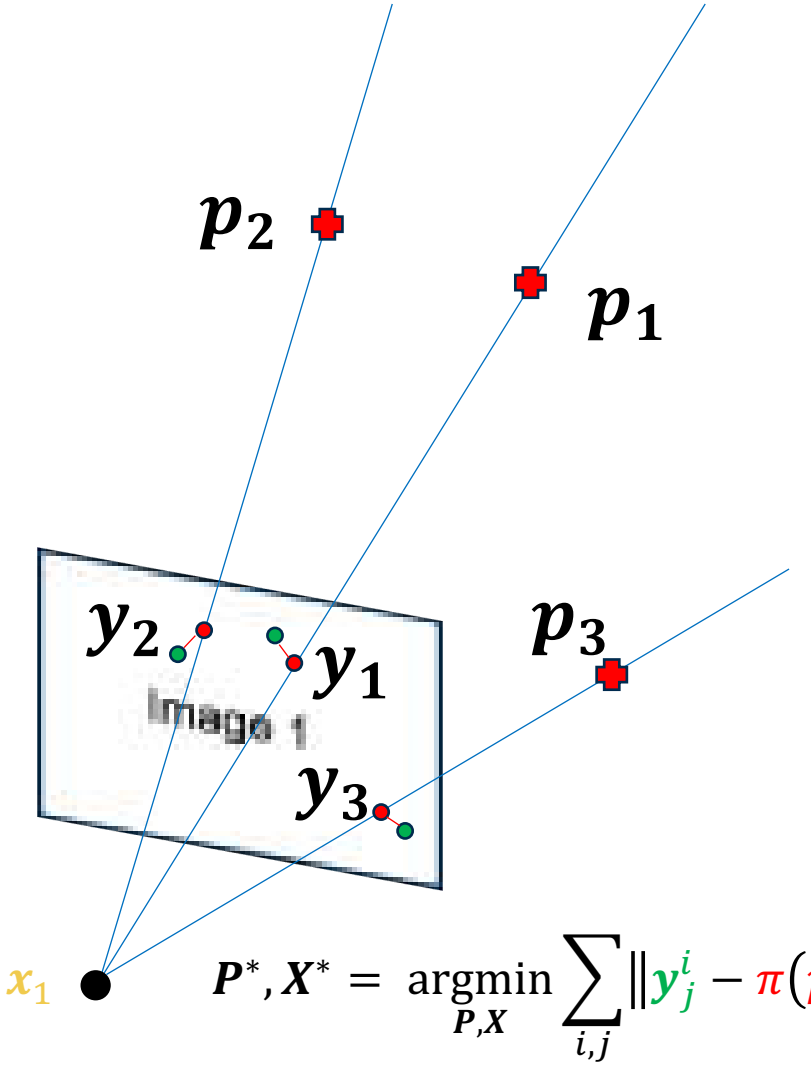


$$\boldsymbol{P}^*, \boldsymbol{X}^* = \underset{\boldsymbol{P},\boldsymbol{X}}{\operatorname{argmin}} \sum_{i,j} \left\| \boldsymbol{y}_j^i - \pi\left(\boldsymbol{p}_j, \boldsymbol{x}_i, \boldsymbol{K}\right) \right\|^2$$

CRICOS No.00213J

Bundle Adjustment



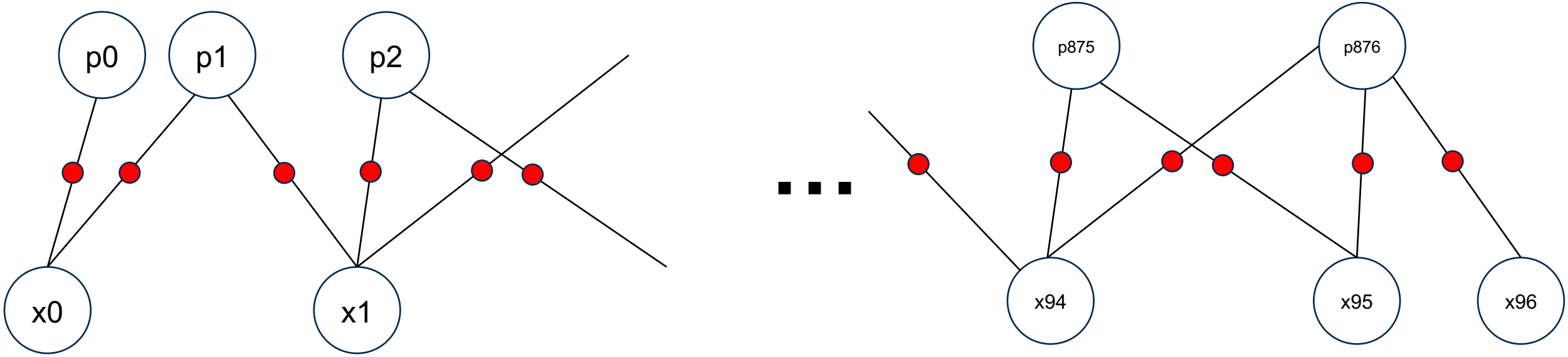


$$P^*, X^* = \underset{P,X}{\operatorname{argmin}} \sum_{i,j} \left\| y_j^i - \pi(p_j, x_i, K) \right\|^2$$

*"What are the optimal camera poses **x** and 3D point coordinates **p**, given the observations **y**?"*

CRICOS No.00213J

Sliding Window Bundle Adjustment



Factor graphs can grow very big over time,
→ increasing computational costs

Solution: only optimise over the most recent n frames (“sliding window”)

CRICOS No.00213J



How to solve these Optimisation Problems

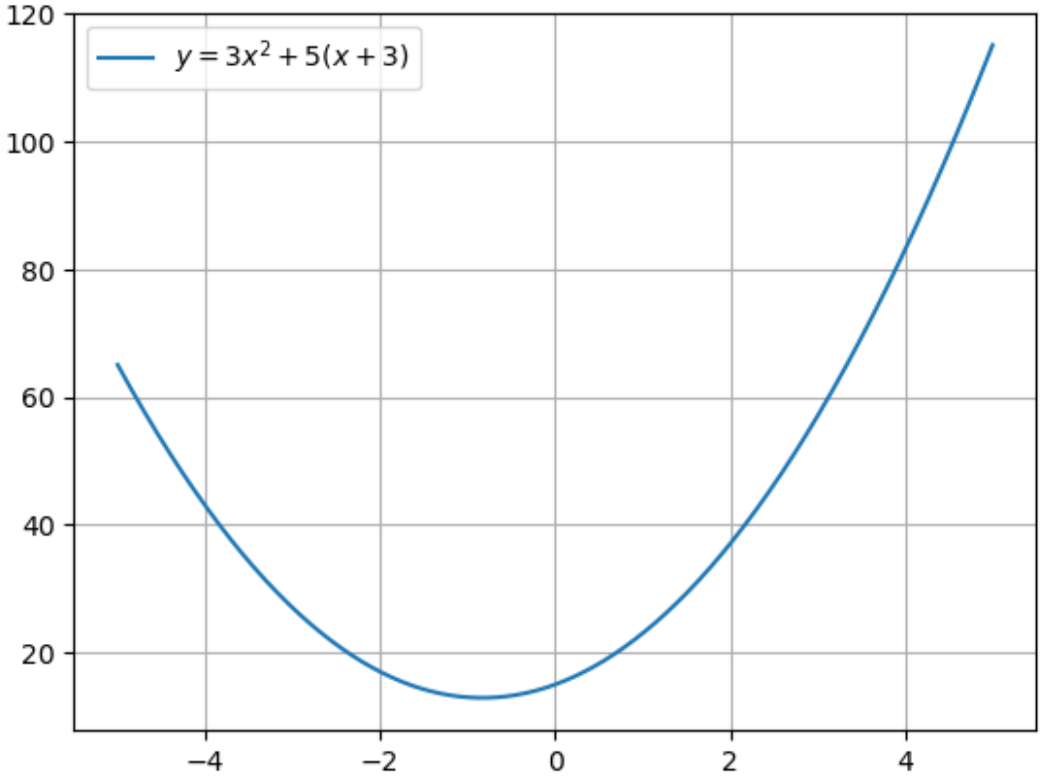
A glimpse into the black box …

How to Find the Minimum of a Function?

- If we know the function,

$$f(x) = 3x^2 + 5(x + 3)$$

- … ?



CRICOS No.00213J

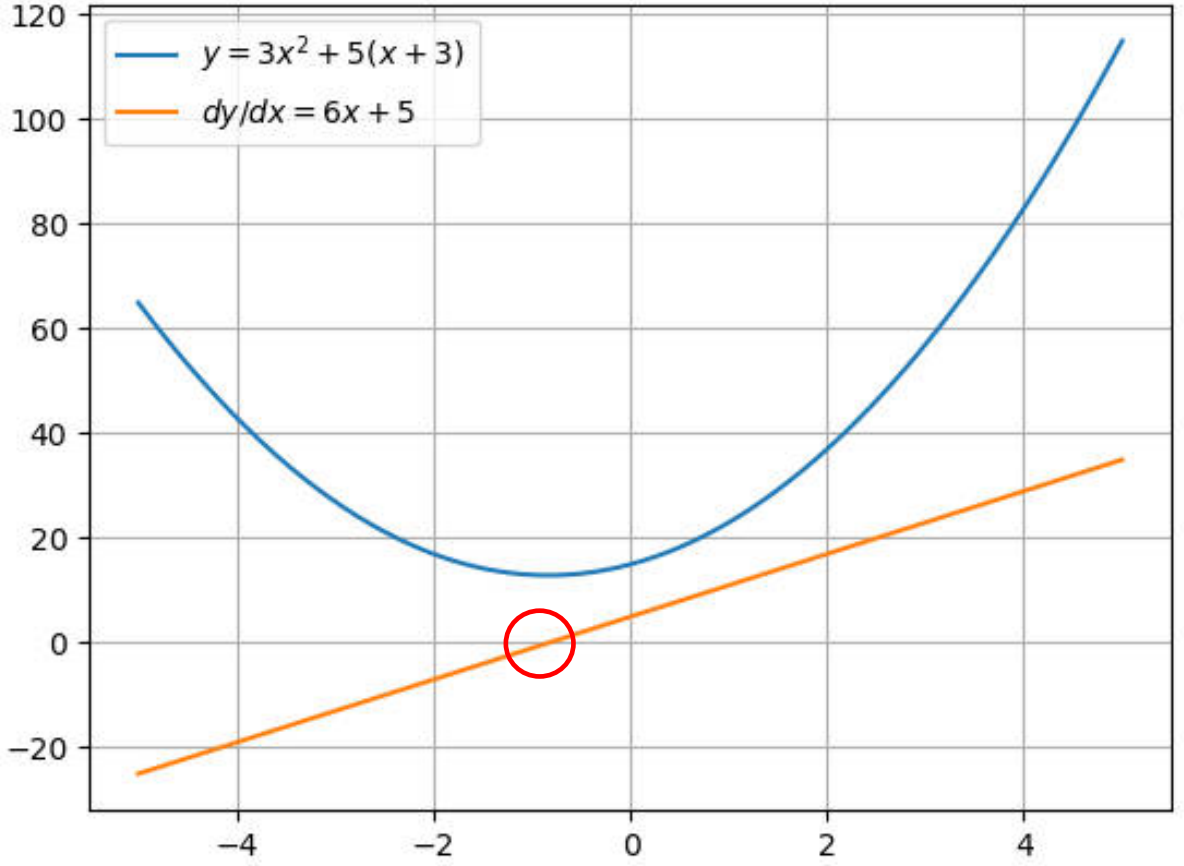
How to Find the Minimum of a Function?

- If we know the function,
$$f(x) = 3x^2 + 5(x+3)$$

- Determine the derivative
$$\frac{f(x)}{dx} = 6x + 5$$

- Find its root, i.e. solve $\frac{f(x)}{dx} = 0$ $\quad x = -\frac{5}{6}$

- (of course this could also be the max, so check 2nd derivative etc.)



CRICOS No.00213J

How to Find the Minimum of a Complicated (Squared) Function?

- Setting the derivative to zero produces a coupled nonlinear system with no direct solution.
  - Iterative methods repeatedly construct and solve a simpler local approximation instead.

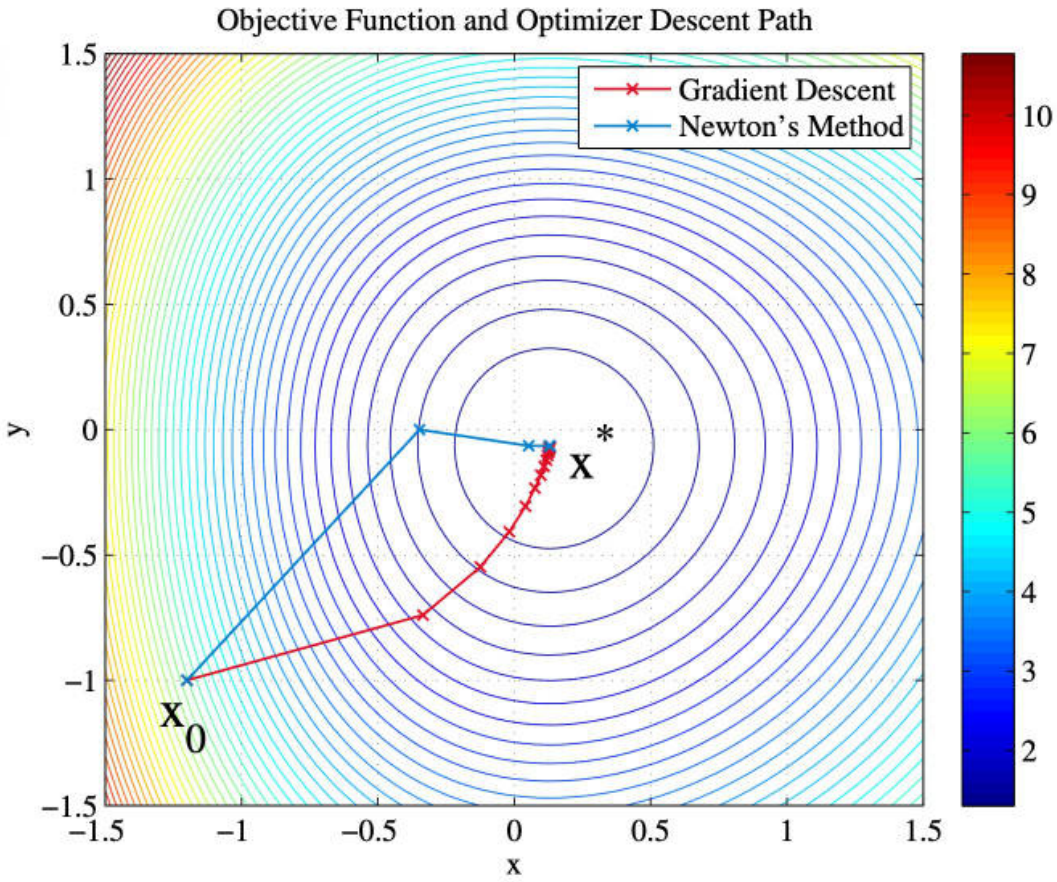
- Typical optimization problems can be huge, with thousands of terms!
  - Function has **thousands of sum terms** we have to minimise over.
  - They will typically contain **nonlinear** factors (e.g. rotations).
  - They will be **vector-valued** functions, not scalars
  - All this makes it **more complicated**!

- ➔ We can't just take the derivative and solve for 0

CRICOS No.00213J

How to Find the Minimum of a Complicated (Squared) Function?

- Idea: **Iterative** process – Gradient Descent
  - Start from an initial guess $\boldsymbol{x}$ for the solution
  - Then use derivative information to step closer to the minimum: $\boldsymbol{x} = \boldsymbol{x} + \Delta\boldsymbol{x}$.
  - Re-evaluate the derivative at the new position, move closer, continue until converged.

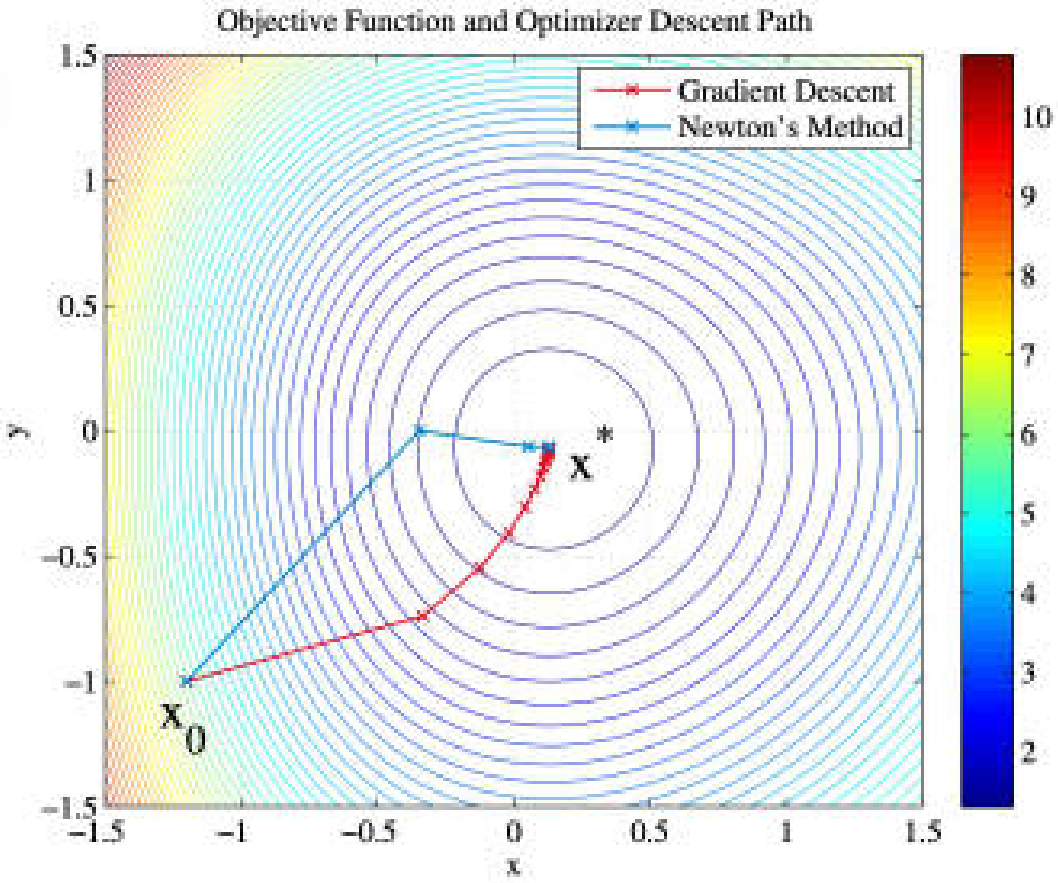


How to Find the Minimum of a Complicated (Squared) Function?

- Many algorithms exist, they try to be smart and reduce the number of steps before they find the optimum.

Method	Step $\Delta\mathbf{x}$
Gradient Descent	$\Delta\mathbf{x} = -\alpha\mathbf{J}^{\mathsf{T}}\mathbf{f}(\mathbf{x})$
Newton's Method	$(\mathbf{H}^{\mathsf{T}}\mathbf{f} + \mathbf{J}^{\mathsf{T}}\mathbf{J})\Delta\mathbf{x} = -\mathbf{J}^{\mathsf{T}}\mathbf{f}(\mathbf{x})$
Gauss-Newton	$\mathbf{J}^{\mathsf{T}}\mathbf{J}\Delta\mathbf{x} = -\mathbf{J}^{\mathsf{T}}\mathbf{f}(\mathbf{x})$
Levenberg-Marquardt	$(\mathbf{J}^{\mathsf{T}}\mathbf{J} + \lambda\mathbf{I})\Delta\mathbf{x} = -\mathbf{J}^{\mathsf{T}}\mathbf{f}(\mathbf{x})$

- $\boldsymbol{J}$ here is the Jacobian, i.e. the derivative of the error function $\boldsymbol{f}(\boldsymbol{x})$

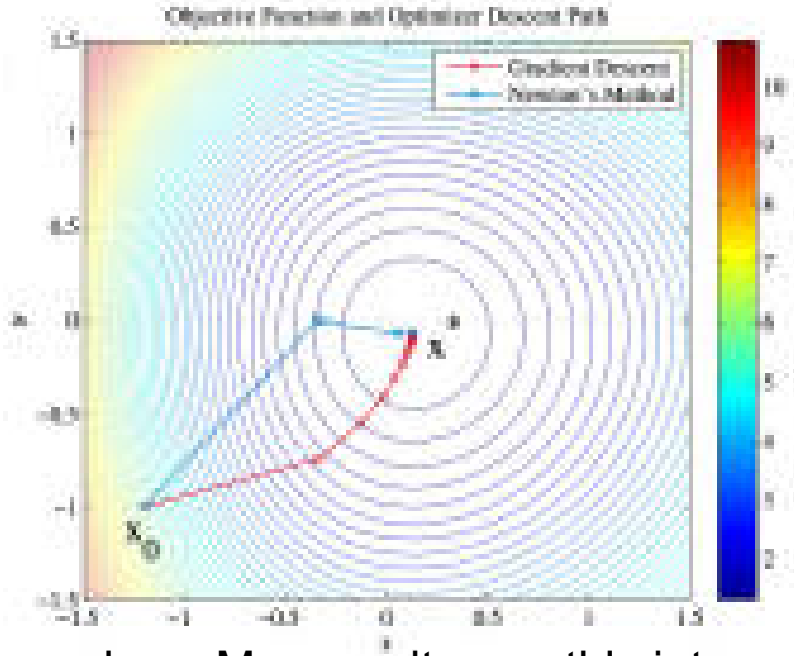


CRICOS No.00213J

How to Find the Minimum of a Complicated (Squared) Function?

- Many algorithms exist, they try to be smart and reduce the number of steps before they find the optimum.

Method	Step $\Delta\mathbf{x}$
Gradient Descent	$\Delta\mathbf{x} = -\alpha\mathbf{J}^{\mathsf{T}}\mathbf{f}(\mathbf{x})$
Newton's Method	$(\mathbf{H}^{\mathsf{T}}\mathbf{f} + \mathbf{J}^{\mathsf{T}}\mathbf{J})\Delta\mathbf{x} = -\mathbf{J}^{\mathsf{T}}\mathbf{f}(\mathbf{x})$
Gauss-Newton	$\mathbf{J}^{\mathsf{T}}\mathbf{J}\Delta\mathbf{x} = -\mathbf{J}^{\mathsf{T}}\mathbf{f}(\mathbf{x})$
Levenberg-Marquardt	$(\mathbf{J}^{\mathsf{T}}\mathbf{J} + \lambda\mathbf{I})\Delta\mathbf{x} = -\mathbf{J}^{\mathsf{T}}\mathbf{f}(\mathbf{x})$



Levenberg-Marquardt smoothly interpolates between the fast local convergence of Gauss–Newton and the more cautious behaviour of gradient descent.

- $\boldsymbol{J}$ here is the Jacobian, i.e. the derivative of the error function $\boldsymbol{f}(\boldsymbol{x})$

CRICOS No.00213J

How to Find the Minimum of a Complicated (Squared) Function?

- Many algorithms exist, they try to be smart and reduce the number of steps before they find the optimum.

Method	Step $\Delta\mathbf{x}$
Gradient Descent	$\Delta\mathbf{x} = -\alpha\mathbf{J}^\mathsf{T}\mathbf{f}(\mathbf{x})$
Newton's Method	$(\mathbf{H}^\mathsf{T}\mathbf{f} + \mathbf{J}^\mathsf{T}\mathbf{J})\Delta\mathbf{x} = -\mathbf{J}^\mathsf{T}\mathbf{f}(\mathbf{x})$
Gauss-Newton	$\mathbf{J}^\mathsf{T}\mathbf{J}\Delta\mathbf{x} = -\mathbf{J}^\mathsf{T}\mathbf{f}(\mathbf{x})$
Levenberg-Marquardt	$(\mathbf{J}^\mathsf{T}\mathbf{J} + \lambda\mathbf{I})\Delta\mathbf{x} = -\mathbf{J}^\mathsf{T}\mathbf{f}(\mathbf{x})$

- $\boldsymbol{J}$ here is the Jacobian, i.e. the derivative of the error function $\boldsymbol{f}(\boldsymbol{x})$

Determining the update $\Delta\boldsymbol{x}$ requires solving these equations in every iteration.

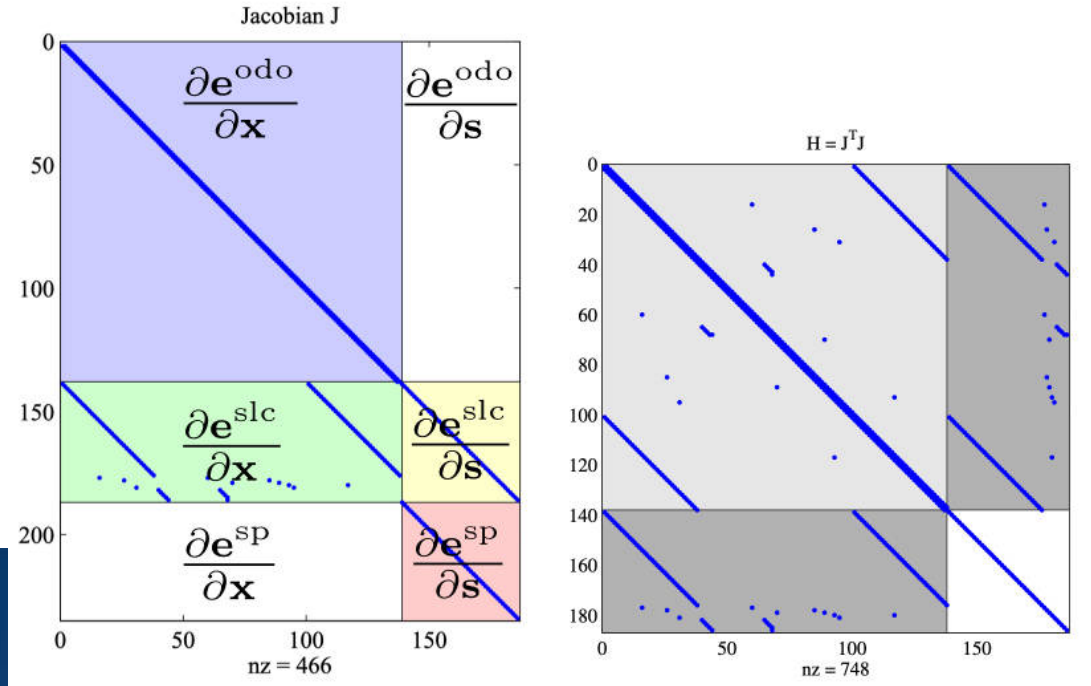
That sounds hard (thousands of variables), however …

CRICOS No.00213J

How to Find the Minimum of a Complicated (Squared) Function?

- Determining the update Δx requires solving these equations in every iteration.
- That sounds hard (thousands of variables), however …
- Jacobian is usually very **sparse**, i.e. most elements are zero.
- This reflects the dependency structure of the problem (and the factor graph connections):
  - A variable typically depends only on a very small number of measurements
  - Any measurement affects only a few variables

Method	Step $\Delta\mathbf{x}$
Gradient Descent	$\Delta\mathbf{x} = -\alpha\mathbf{J}^\mathsf{T}\mathbf{f}(\mathbf{x})$
Newton's Method	$(\mathbf{H}^\mathsf{T}\mathbf{f} + \mathbf{J}^\mathsf{T}\mathbf{J})\Delta\mathbf{x} = -\mathbf{J}^\mathsf{T}\mathbf{f}(\mathbf{x})$
Gauss-Newton	$\mathbf{J}^\mathsf{T}\mathbf{J}\Delta\mathbf{x} = -\mathbf{J}^\mathsf{T}\mathbf{f}(\mathbf{x})$
Levenberg-Marquardt	$(\mathbf{J}^\mathsf{T}\mathbf{J} + \lambda\mathbf{I})\Delta\mathbf{x} = -\mathbf{J}^\mathsf{T}\mathbf{f}(\mathbf{x})$



What is actually happening with these equations?

They are all of a similar form, called “normal equation”.

$$A^T Ax = A^T b$$

Multiple ways of solving for x are known, each with pros and cons.

Efficient due to exploiting sparsity of A.

Method	Step $\Delta\mathbf{x}$
Gradient Descent	$\Delta\mathbf{x} = -\alpha\mathbf{J}^\mathsf{T}\mathbf{f}(\mathbf{x})$
Newton’s Method	$(\mathbf{H}^\mathsf{T}\mathbf{f} + \mathbf{J}^\mathsf{T}\mathbf{J})\Delta\mathbf{x} = -\mathbf{J}^\mathsf{T}\mathbf{f}(\mathbf{x})$
Gauss-Newton	$\mathbf{J}^\mathsf{T}\mathbf{J}\Delta\mathbf{x} = -\mathbf{J}^\mathsf{T}\mathbf{f}(\mathbf{x})$
Levenberg-Marquardt	$(\mathbf{J}^\mathsf{T}\mathbf{J} + \lambda\mathbf{I})\Delta\mathbf{x} = -\mathbf{J}^\mathsf{T}\mathbf{f}(\mathbf{x})$

1. If $\mathbf{A}^\mathsf{T}\mathbf{A}$ is invertible, the solution is found directly by $\mathbf{x}^* = (\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}\mathbf{b}$.

2. A better alternative uses the *QR decomposition* of $\mathbf{A}$ into an upper triangular matrix $\mathbf{R}$ and an orthogonal matrix $\mathbf{Q}$, so that $\mathbf{A} = \mathbf{QR}$. The solution $\mathbf{x}^*$ is then given by $\mathbf{x}^* = \mathbf{R}^{-1}\mathbf{Q}^\mathsf{T}b$. This method prevents the calculation of $\mathbf{A}^\mathsf{T}\mathbf{A}$ and its inversion, which can become numerically unstable.

3. Instead of the QR decomposition, the *Cholesky decomposition* of $\mathbf{A}^\mathsf{T}\mathbf{A}$ can be used. This method decomposes $\mathbf{A}^\mathsf{T}\mathbf{A}$ into $\mathbf{LL}^\mathsf{T}$, then solves $\mathbf{Ly} = \mathbf{A}^\mathsf{T}\mathbf{b}$ for $\mathbf{y}$ and finds $\mathbf{x}^*$ by solving $\mathbf{L}^\mathsf{T}\mathbf{x} = y$. Notice that $\mathbf{L}$ is a lower triangular matrix with positive diagonal elements. Compared to the QR decomposition, Cholesky is faster, but numerically more unstable because $\mathbf{A}^\mathsf{T}\mathbf{A}$ has to be calculated.

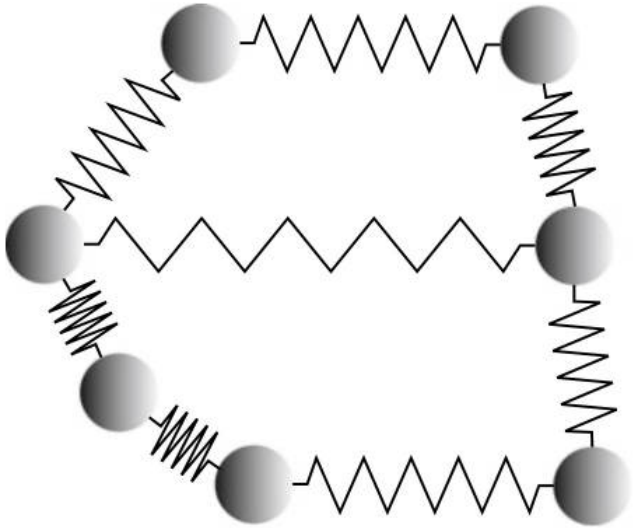
4. The solution to the linear least squares problem with $\mathbf{f}(\mathbf{x}) = \mathbf{Ax} - \mathbf{b}$ can as well be determined using the *singular value decomposition*, SVD. Here, $\mathbf{A}$ would be decomposed into $\mathbf{A} = \mathbf{UDV}^\mathsf{T}$. After calculating a new vector $\mathbf{b}' = \mathbf{U}^\mathsf{T}\mathbf{b}$ and defining a vector $\mathbf{y}$ with $y_i = b'_i/d_i$ where d_i is the i-th diagonal entry of $\mathbf{D}$, the solution $\mathbf{x}^*$ is found by $\mathbf{x}^* = \mathbf{Vy}$. Further details can be found in [Hartley and Zisserman, 2004, ch. A5].

QUT

Intuition –
A Factor Graph as a Spring-Mass Model

Spring-mass model of the optimization for pose graph SLAM.

The springs represent the constraint between the variables, which can be imagined as small masses. The solution to the optimization problem corresponds to the minimum energy configuration of the spring-mass system.



Spring-Mass model of a pose graph.

CRICOS No.00213J

More than one minimum!

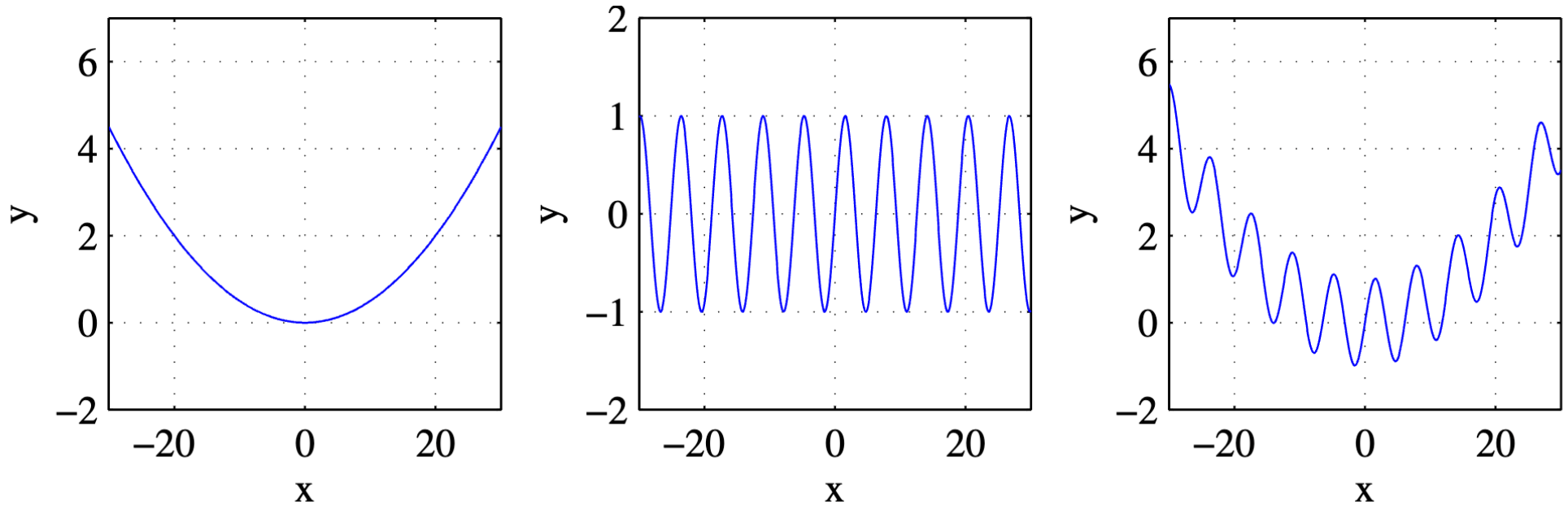
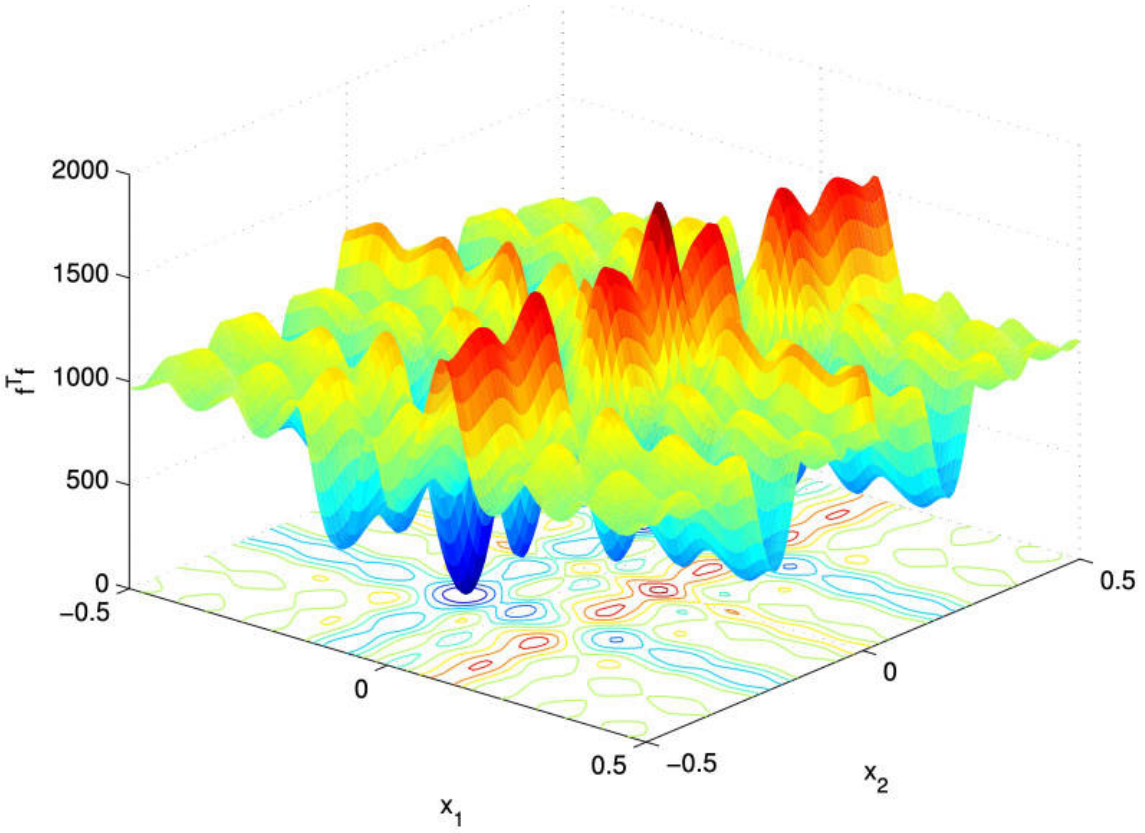
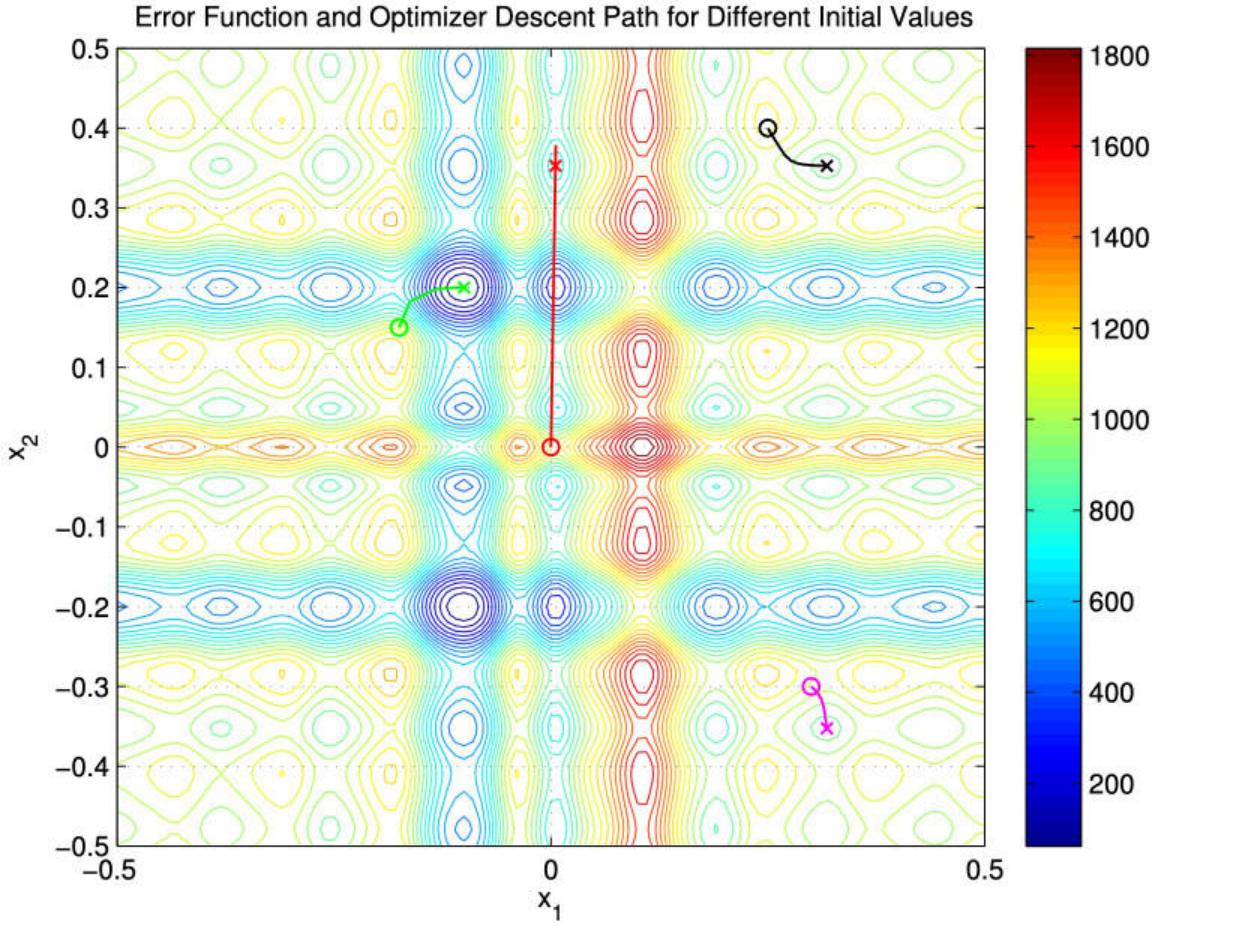


Figure 3.1: *Functions can have one global minimum (left), multiple local minima (middle), and a global minimum with multiple local minima (right).*

CRICOS No.00213J

Solution depends on initial guess!





CRICOS No.00213J

Important to keep in mind!

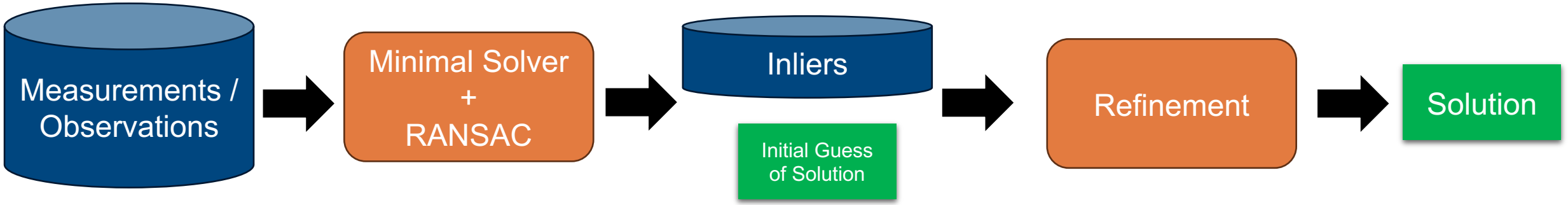
- The **initial guess** must be *"close"* to the true solution.
  - Otherwise a local minima might be found.
- This is vulnerable to **outliers**!
- **Sparse** problems are more efficient than non-sparse problems.

- There are a number of software packages to use for solving factor graphs:
  - gtsam (C++ & Python)
  - Ceres Solver (C++)
  - Symforce (Python)

- Writing the **Jacobians by hand** is most error-prone part.
  - Symforce avoids this through **auto-differentiation**

CRICOS No.00213J

A very typical pipeline in Computer Vision



- Typically a lot of data but contains outliners
- *e.g. matched features*

Identifies outliers

- Reduced set of inliers, guaranteed* to be free of outliers
- Solution based only on minimal set
- *e.g. 8 point pairs for F, 3 point pairs for P3P*

- Typically iterative (nonlinear) Least Squares
- Requires initial guess
- Very vulnerable to outliers

CRICOS No.00213J

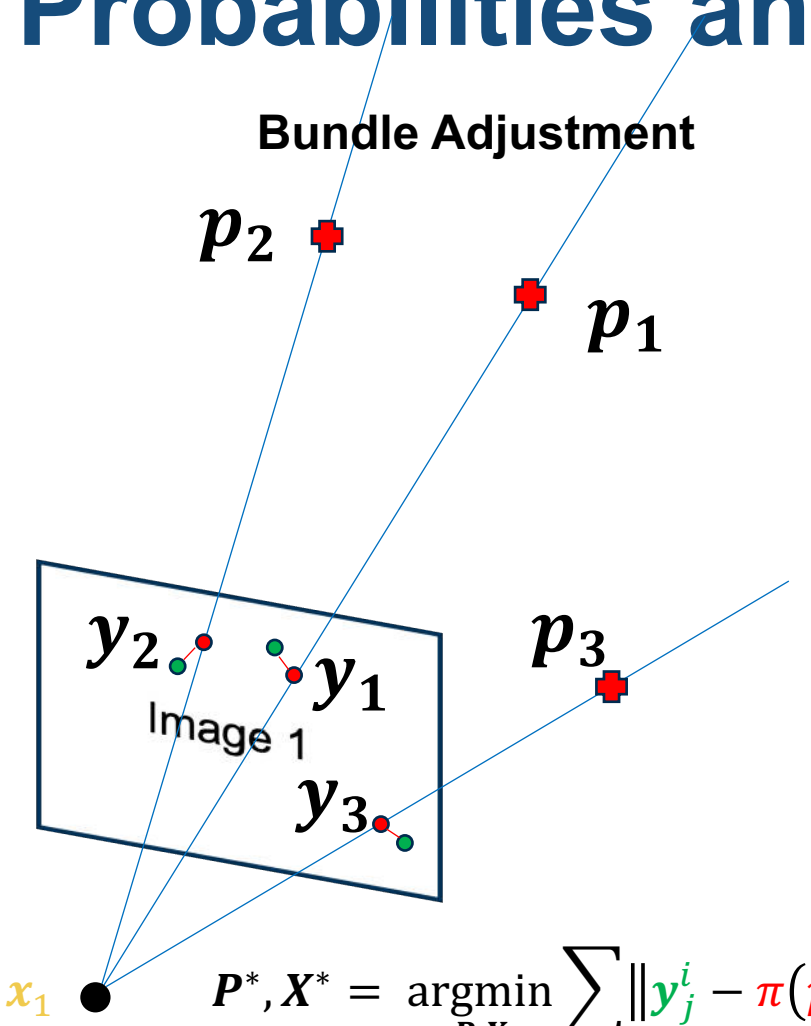


Factor Graphs and how to “solve” them

A glimpse into the black box …

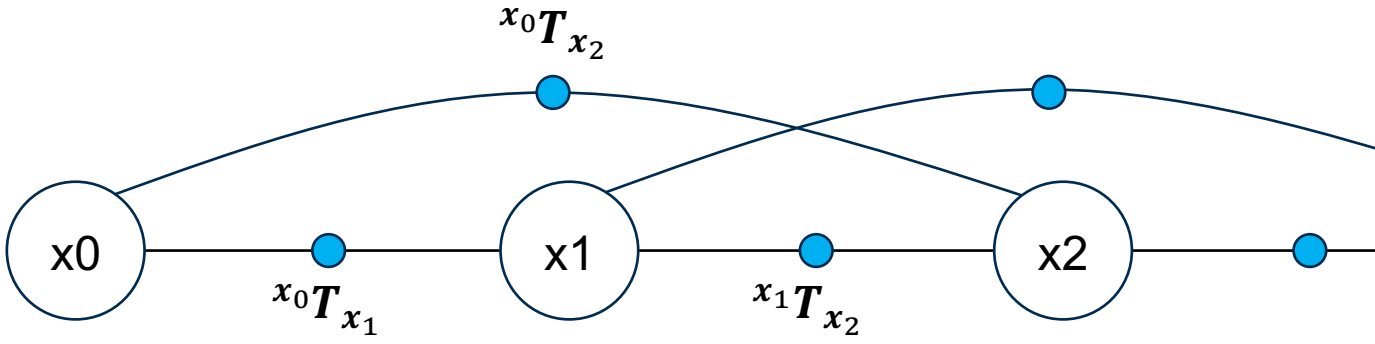
On the Connection between Factor Graphs, Probabilities and Nonlinear Least Squares

Bundle Adjustment



$$P^*, X^* = \underset{P,X}{\operatorname{argmin}} \sum_{i,j} \left\| y_j^i - \pi(p_j, x_i, K) \right\|^2$$

Pose Graph Optimisation



$$X^* = \underset{X}{\operatorname{argmin}} \sum_{i,j} \left\| x_i - {}^i T_j \cdot x_j \right\|^2$$

CRICOS No.00213J

Probabilistic Estimation Problems in Robotics

- We often need to estimate *unknown quantities* based on *sensor measurements* or *prior knowledge*.
  - “variables”, “hidden variables”, “latent variables”
  - “observations”, “measurements”, ”priors”

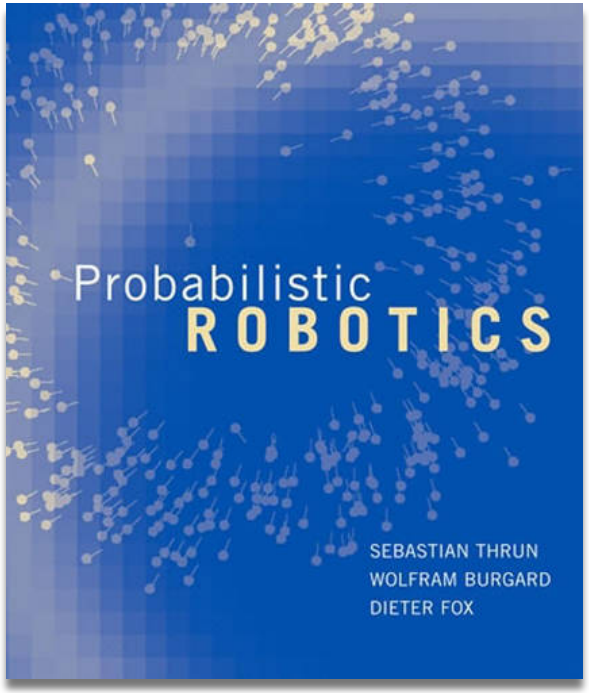
- What are some examples?
  - Position of an outdoor robot based on GPS measurements.
  - Velocity and turn rate based on visual odometry.
  - Landmark positions and pose of the robot over time, based on visual SLAM.

CRICOS No.00213J

Probabilistic Estimation Problems in Robotics

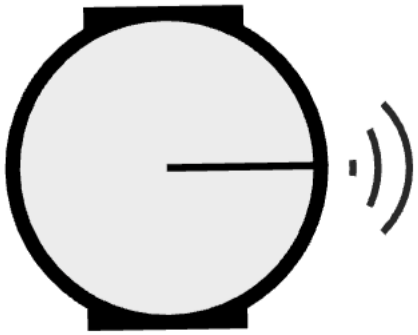
- We often have not just one, but multiple sensors we want to combine.
  - ”sensor fusion”

- Multiple sensors:
  - … might contradict each other
  - … have different error and noise characteristics
    → Probabilistic Estimation



CRICOS No.00213J

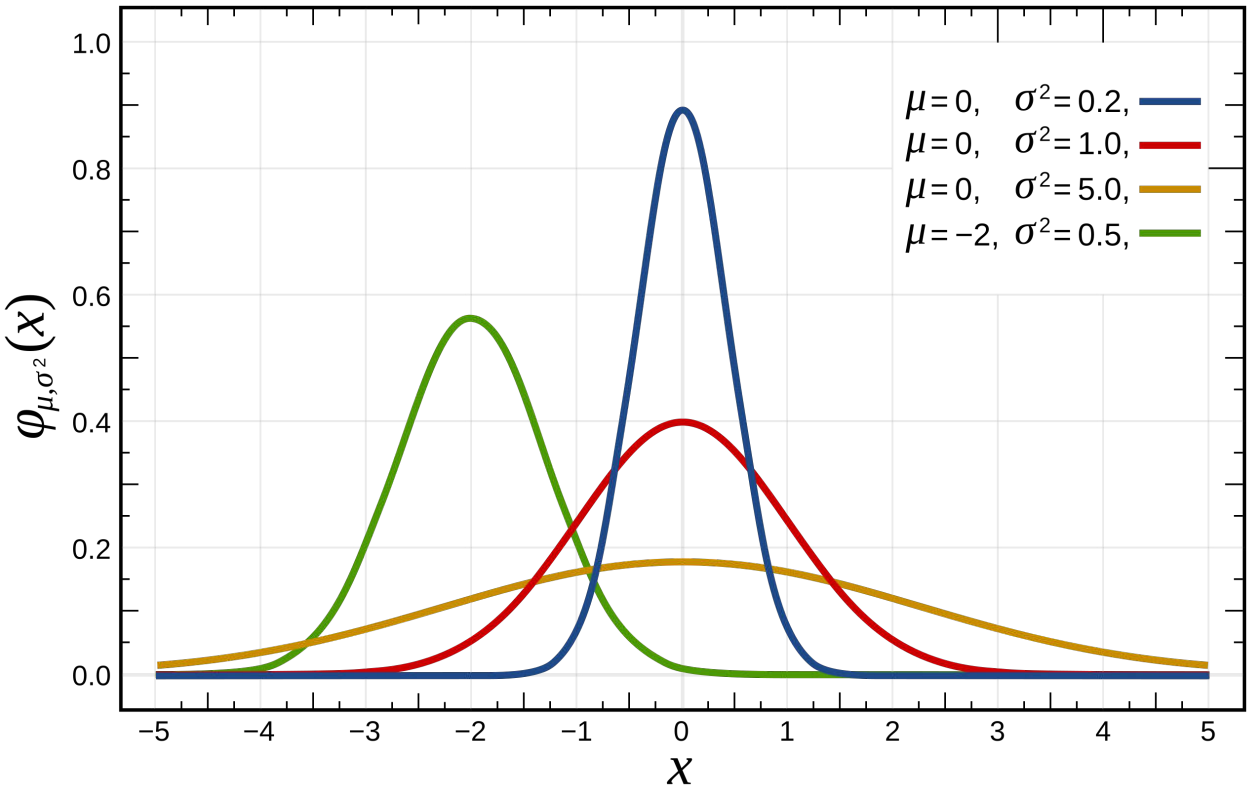
Probabilistic (?!) Estimation Problems





Sonar sensor on the robot measures 5 meters distance. How far away from the wall is the robot?

Gaussian / Normal Distribution



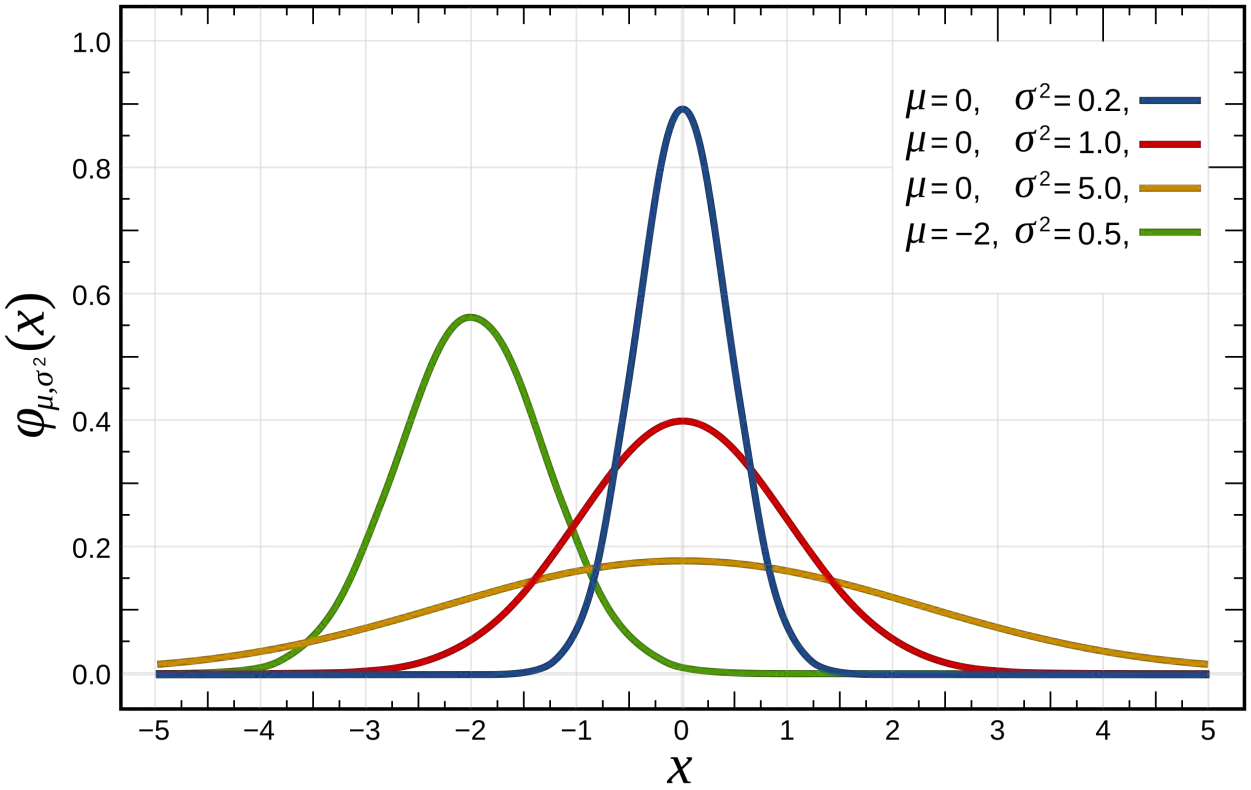
CRICOS No.00213J

Gaussian / Normal Distribution

- Two parameters:
  - Mean μ and variance σ^2
- Has a convenient parametric form:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

- Density function: expresses our belief in the true value of x.
- *Nothing is ever truly Gaussian, but often close enough.*



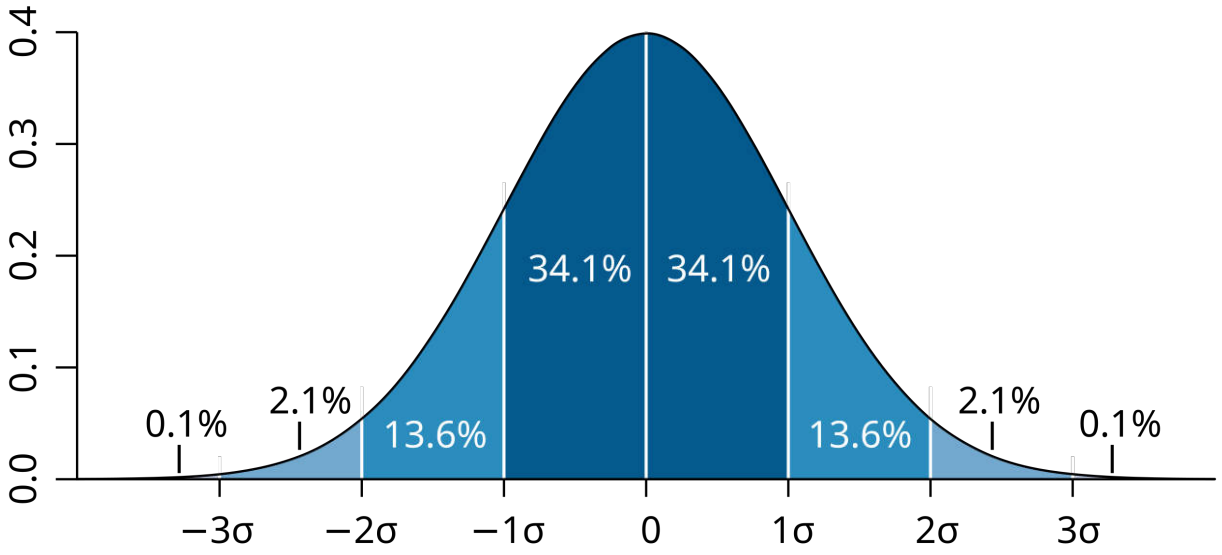
CRICOS No.00213J

Gaussian / Normal Distribution

- Two parameters:
  - Mean μ and variance σ^2
- Has a convenient parametric form:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

- Density function: expresses our belief in the true value of x.
- *Nothing is ever truly Gaussian, but often close enough.*



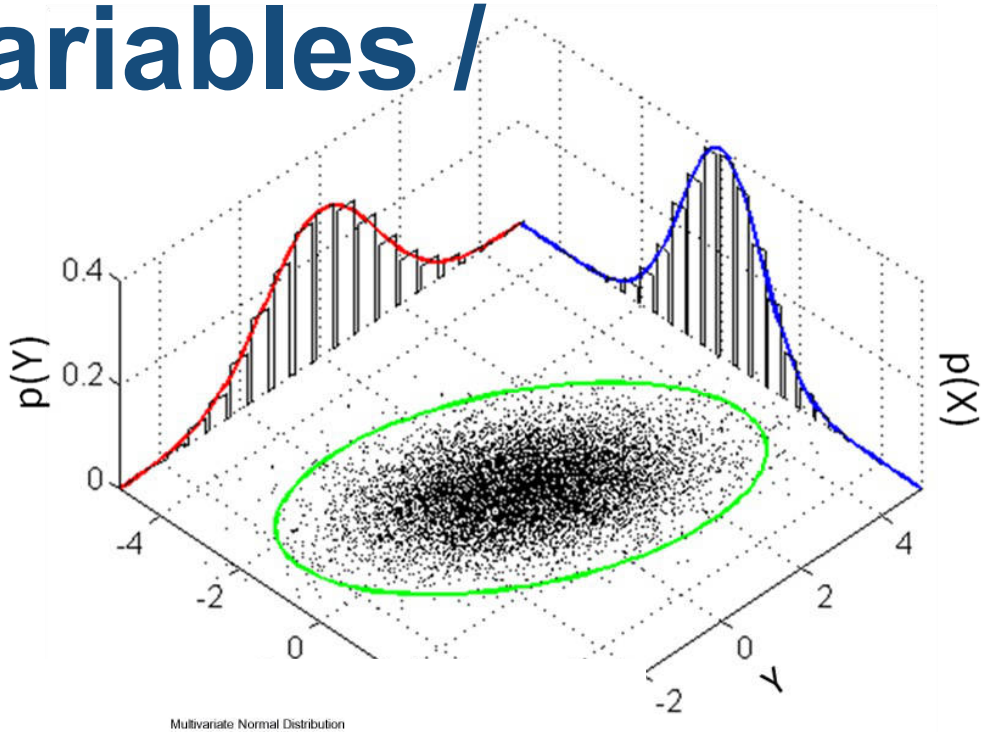
CRICOS No.00213J

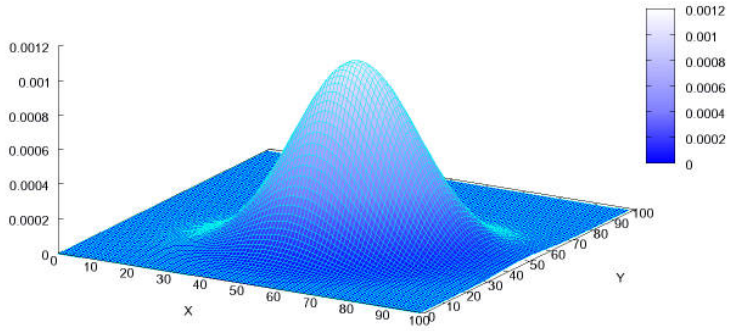
What about Multivariate Variables / Measurements?

- Gaussians also exist in 2D (ellipses) or 3D (ellipsoids), or any other dimension!
- Still has a nice parametric form with mean vector $\boldsymbol{\mu}$ and covariance matrix $\boldsymbol{\Sigma}$

$$f_{\mathbf{X}}(x_1, \ldots, x_k) = \frac{\exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^{\mathrm{T}}\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right)}{\sqrt{(2\pi)^k|\boldsymbol{\Sigma}|}}$$

- Compare to the 1D case : $f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$

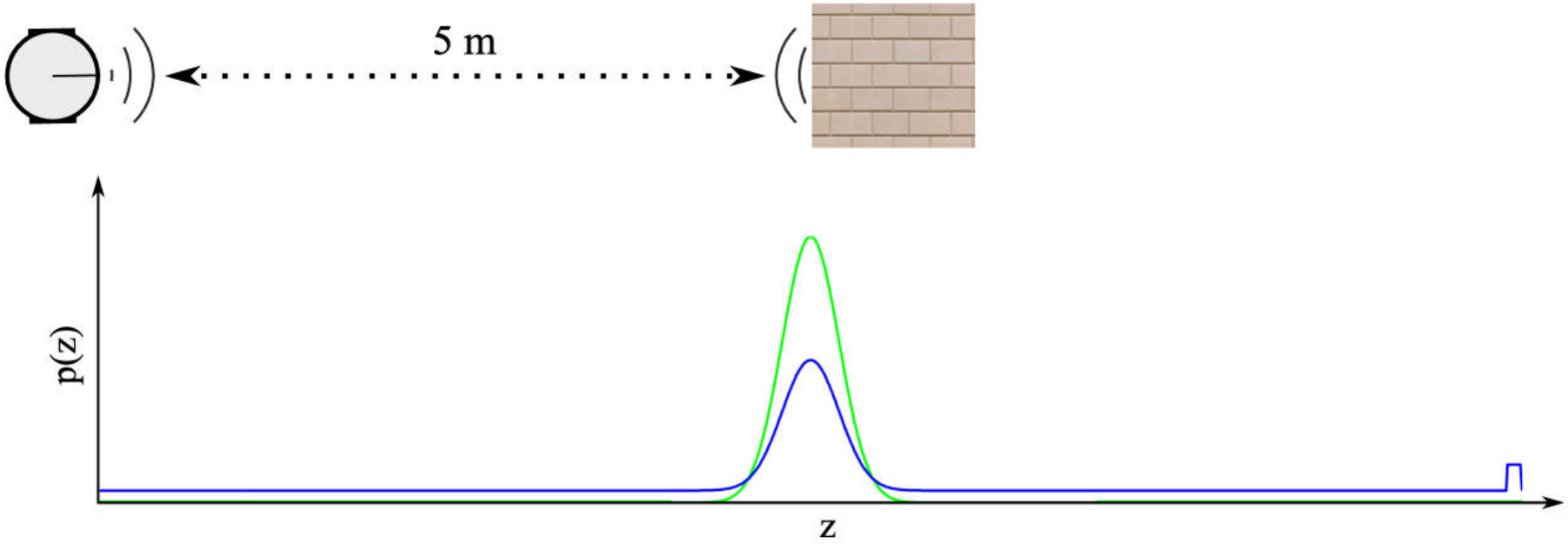




CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=1260349

Probabilistic (?!) Estimation Problems

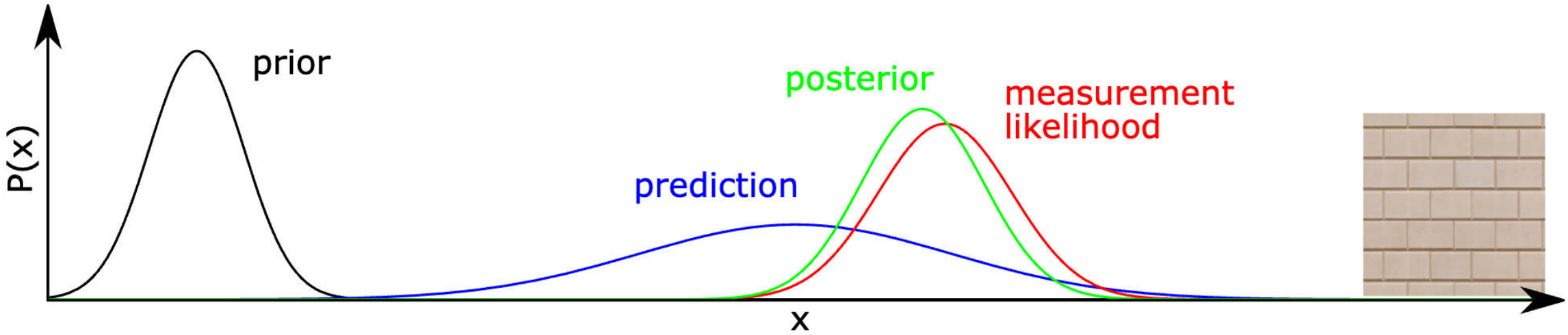
- Comparison of an ideal **Gaussian** (green) and more **realistic** (blue) model for a sonar sensor (adapted after [Thrun et al., 2005, ch. 6.3]).



CRICOS No.00213J

Combine Multiple Measurements?

Sensor Fusion lets us combine multiple measurements



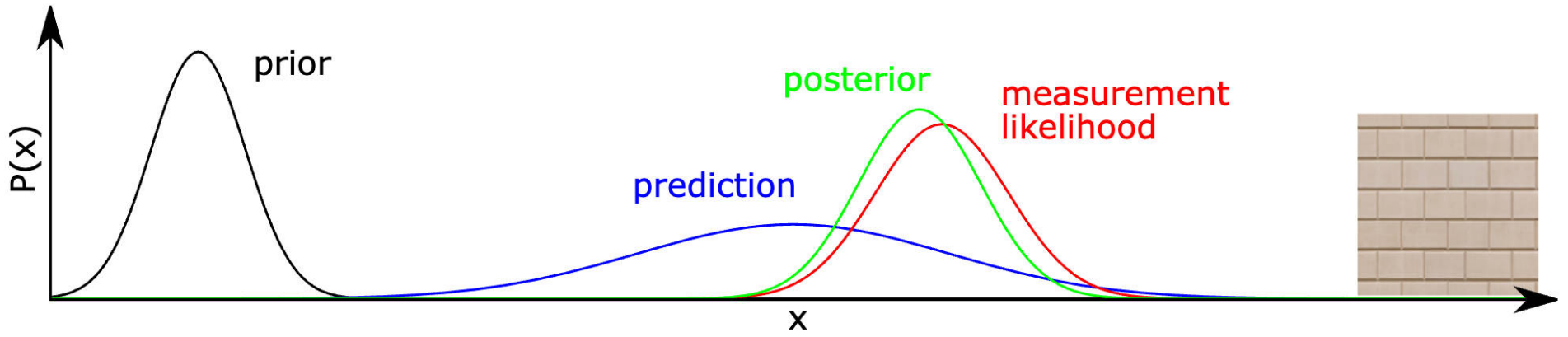
Simple localization example using the **Kalman filter**. Notice how the probability distributions used to represent the different quantities and estimates in the filter are all Gaussians.

- Starting from the prior, the robot moves towards the right.
- Its new position is predicted according to the state transition model and the control input and represented by the blue Gaussian.
- Notice that due to the noise in the control inputs, the prediction is rather uncertain about the robot's position.
- After measuring the distance to the wall on the far right side, the Kalman filter can correct the predicted position by incorporating the measurement likelihood (red).
- Notice how the green posterior is much closer to the measurement likelihood than to the prediction, because the measurement uncertainty is smaller than the uncertainty of the prediction.

Probabilistic Questions

We are actually asking questions about different probability distributions:

- p(x) is the prior belief about the robot's position
- p(x | u) is the prediction after the robot moved, given its control inputs u
- p(x | u, z) is the posterior after incorporating the sonar measurement z
- p(z | x) is the measurement likelihood → this is our sensor model



CRICOS No.00213J

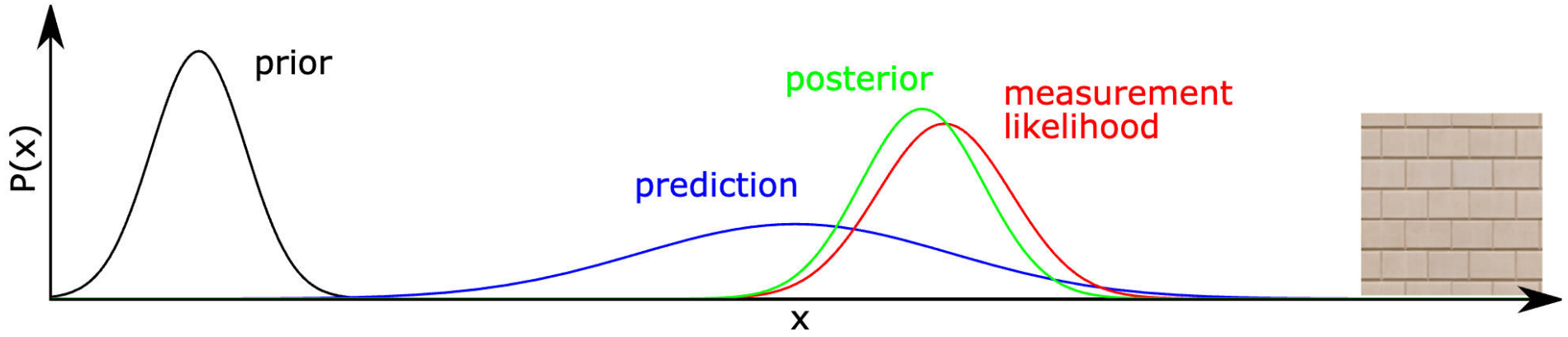
Probabilistic Questions

"Where was the robot, based on all I know about the control inputs and sonar measurements?"

$$p(x_0, x_1, \ldots, x_t | u_0, u_1, \ldots, u_{t-1}, z_0, z_1, \ldots, z_t)$$

"Where is the robot NOW, based on all I know about the PAST control inputs and sonar measurements?"

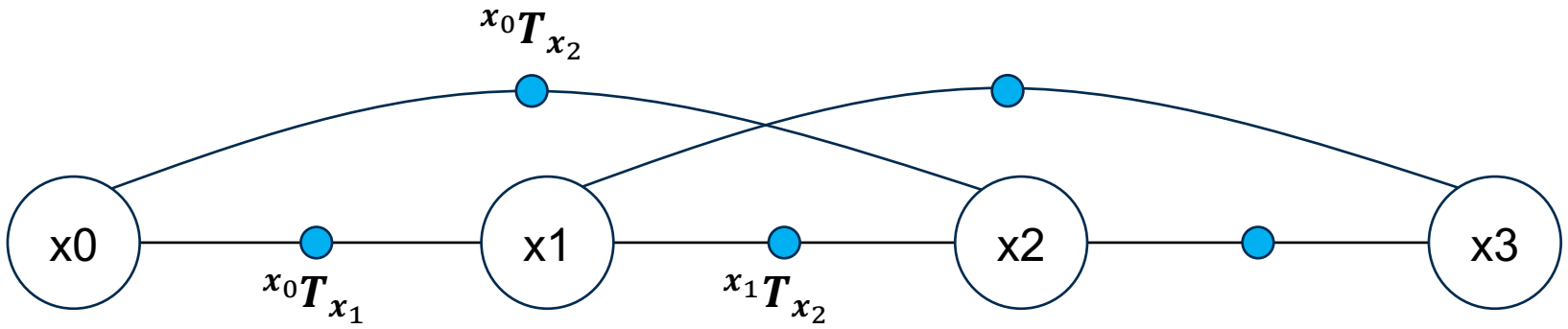
$$p(x_t | u_0, u_1, \ldots, u_{t-1}, z_0, z_1, \ldots, z_t)$$



CRICOS No.00213J

Graphical Representation

- Factor Graphs are a convenient way to model complicated estimation problems.
  - The variable nodes represent the unknown random variables in the estimation problem.
  - The factor nodes represent probabilistic constraints on those variables, derived from measurements or prior knowledge.
- We have already encountered some examples:



Conventions:

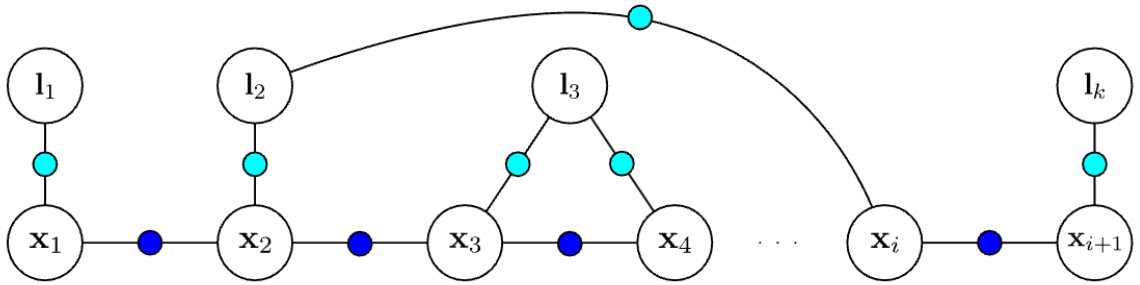
Variables: x

Factors (Measurements): ● or sometimes ■

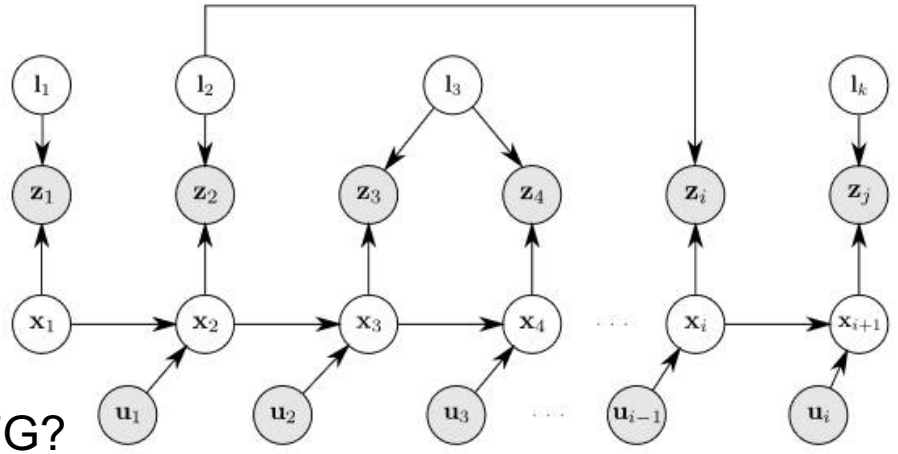
CRICOS No.00213J

Properties of Factor Graphs

- Factor graphs are
  - “bipartite” graphs
    - there are two sets of nodes (variables and measurements)
    - every edge connects a variable node with a measurement node
  - “undirected”
    - edges don’t have a direction



Which one is a FG?



Let’s Build a Factor Graph!

- Unkown: robot position at different time steps t=0 … 3
- Measurements: GPS

CRICOS No.00213J

Let's Build a Factor Graph!

- Unkown: robot position at different time steps t=0 … 3
- Measurements: GPS



CRICOS No.00213J

Let's Build a Factor Graph!

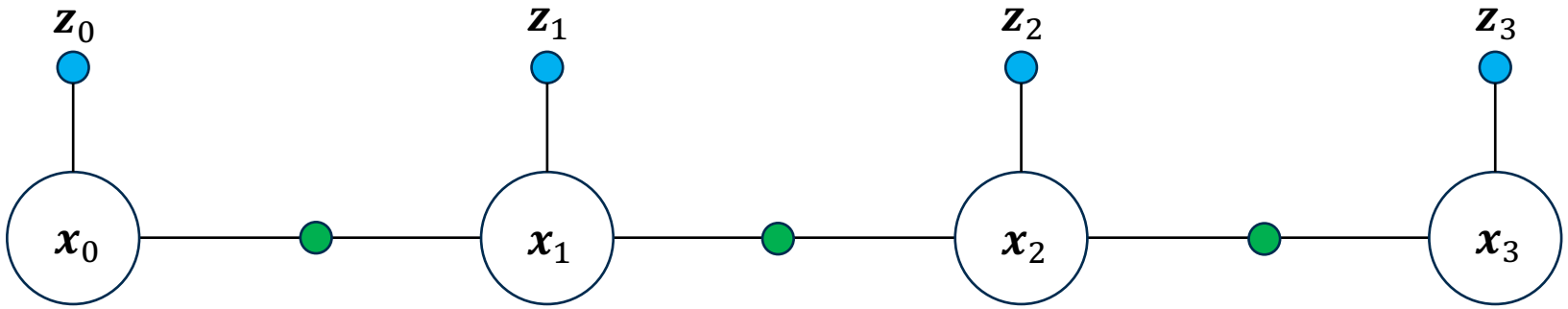
- Unkown: robot position at different time steps t=0 … 3
- Measurements: GPS + visual odometry



CRICOS No.00213J

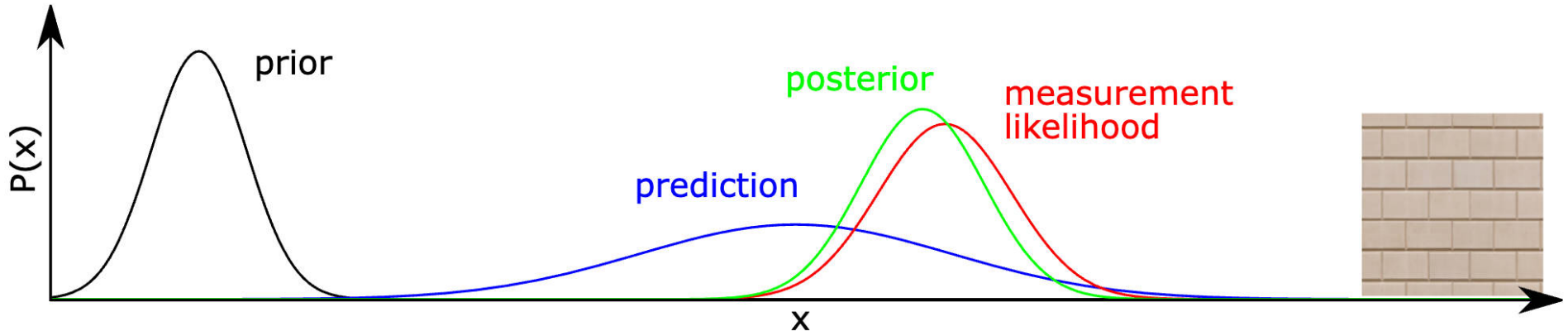
Let's Build a Factor Graph!

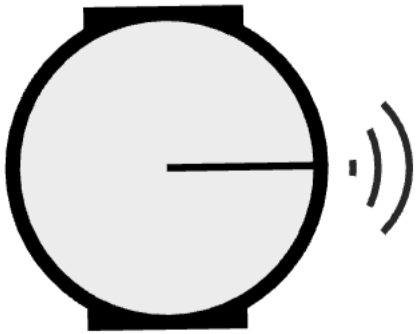
- Unkown: robot position at different time steps t=0 … 3
- Measurements: GPS + visual odometry



CRICOS No.00213J

Let's Build a Factor Graph!



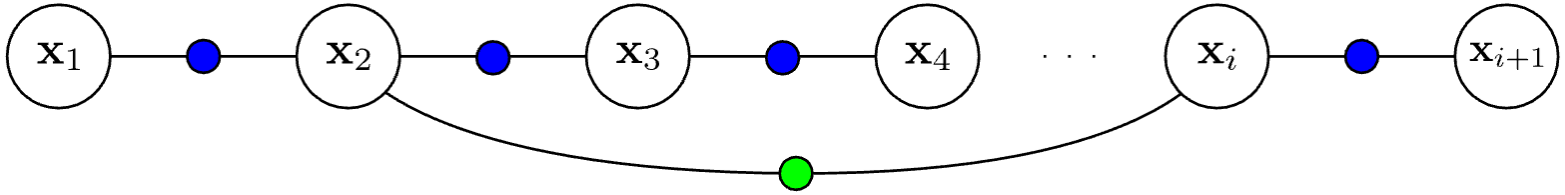


Turn this situation into a factor graph!

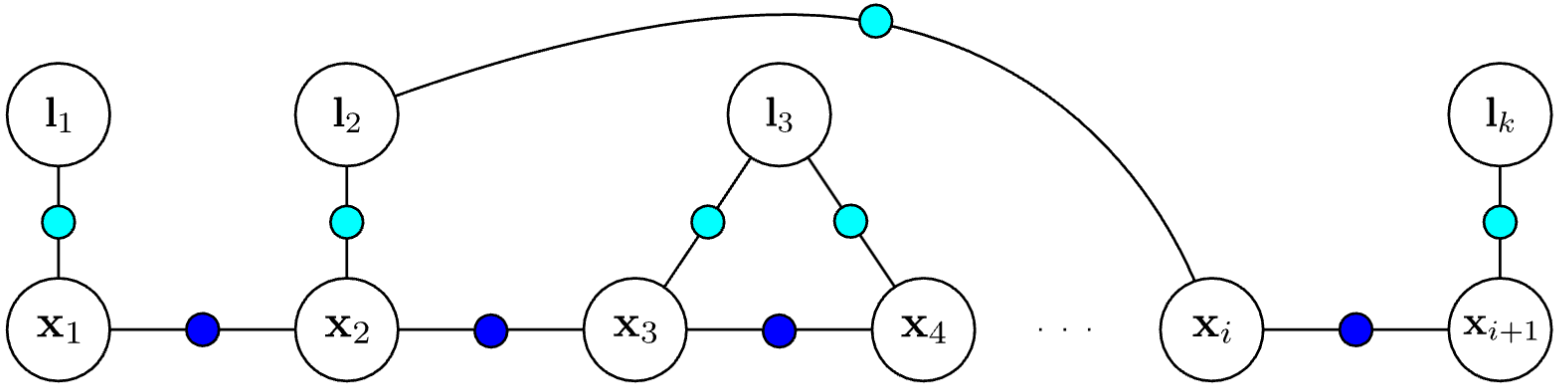


CRICOS No.00213J

More Examples



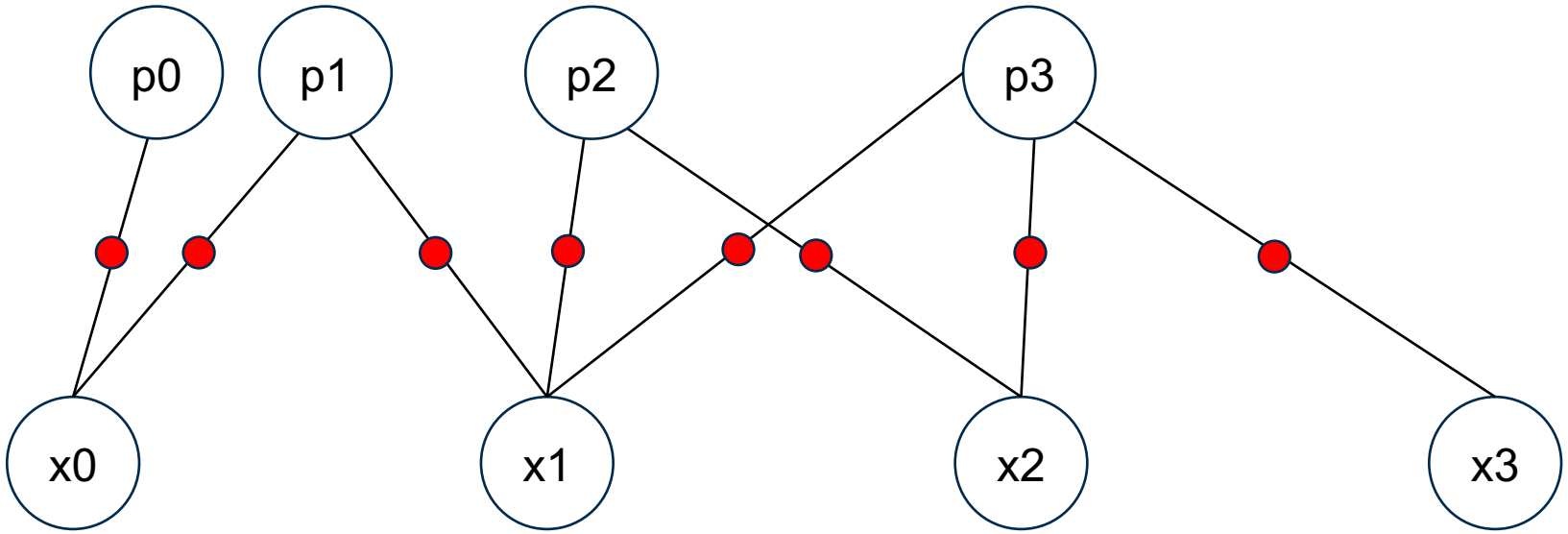
Pose Graph SLAM with a loop closure.



SLAM with landmarks.

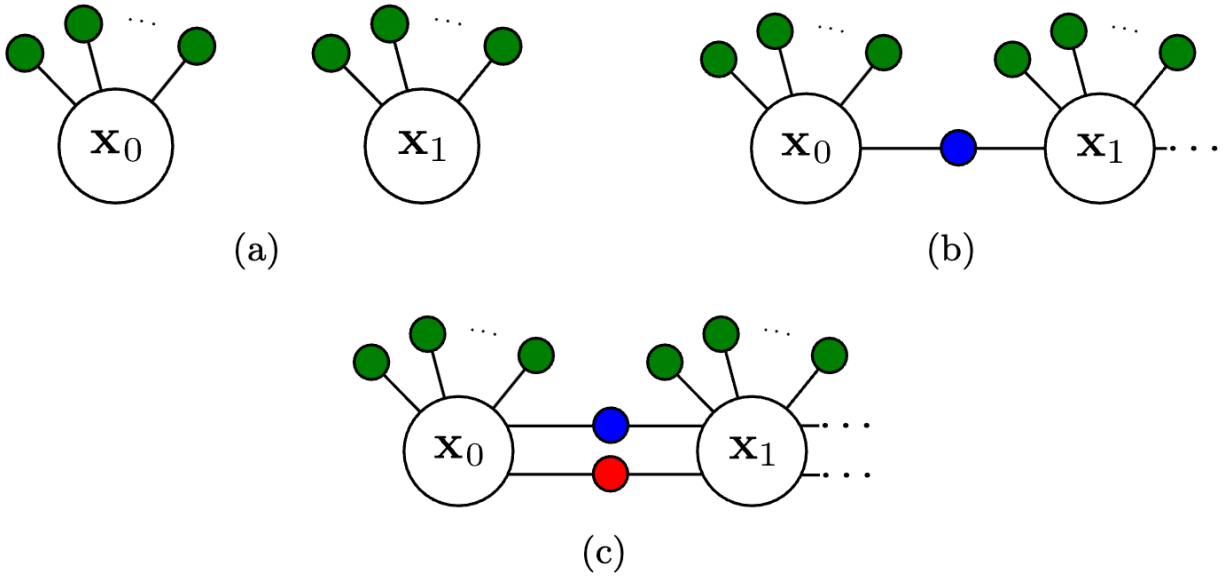
CRICOS No.00213J

More Examples



Bundle Adjustment

More Examples



Two vehicle state nodes with their associated pseudorange factors (green). (a) in the most general graphical model, there are no connections between the vehicle state nodes. In (b), a state transition factor joins two successive vehicle nodes. In (c), an additional motion model is incorporated.

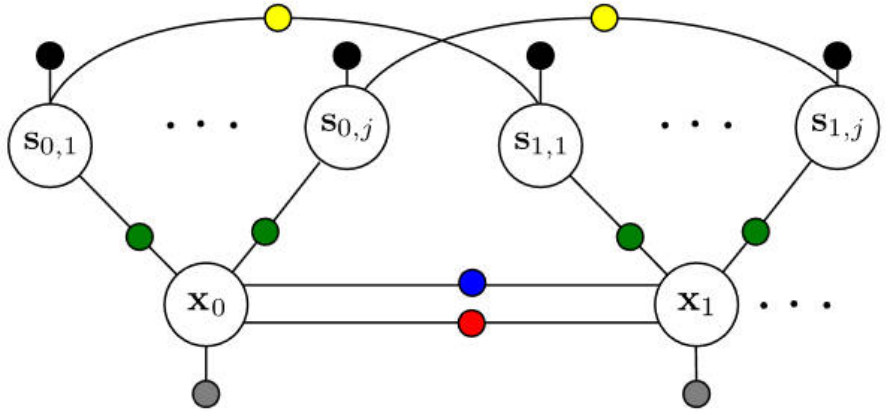


Illustration of the most complex factor graph used in the evaluation for multipath mitigation: There are two types of variables (vehicle states xt and switch variables stj . The switch variables are associated with pseudorange factors (green) and connected by switch transition factors (yellow). The switch priors are depicted in black. The vehicle state variables are connected by state transition (blue) and motion model (red) factors and furthermore governed by prior factors (grey).

CRICOS No.00213J

Switchable Constraints for Robust Simultaneous Localization and Mapping and Satellite-Based Localization. PhD Thesis, Suenderhauf, 2012. Springer Verlag 2023.

Probabilistic Questions

"Where was the robot, based on all I know about the control inputs and sonar measurements?"

$$p(x_0, x_1, \dots, x_t | u_0, u_1, \dots, u_{t-1}, z_0, z_1, \dots, z_t)$$

"Where is the robot NOW, based on all I know about the PAST control inputs and sonar measurements?"

$$p(x_t | u_0, u_1, \dots, u_{t-1}, z_0, z_1, \dots, z_t)$$

"Where is the robot NOW, based on my previous estimate and the most recent control inputs and sonar measurements?"

$$p(x_t | x_{t-1}, u_{t-1}, z_t)$$

CRICOS No.00213J

Factorisation into Smaller Subsets

- Such big joint probability distributions can often be *factorised* into smaller sets.
- Independence between the involved probabilistic variables matters!

$$p(x_0, x_1, \dots, x_t | u_0, u_1, \dots, u_{t-1}, z_0, z_1, \dots, z_t) = \prod_i p(x_i | x_{i-1}, u_{i-1}) \cdot \prod_k p(x_k | z_k)$$

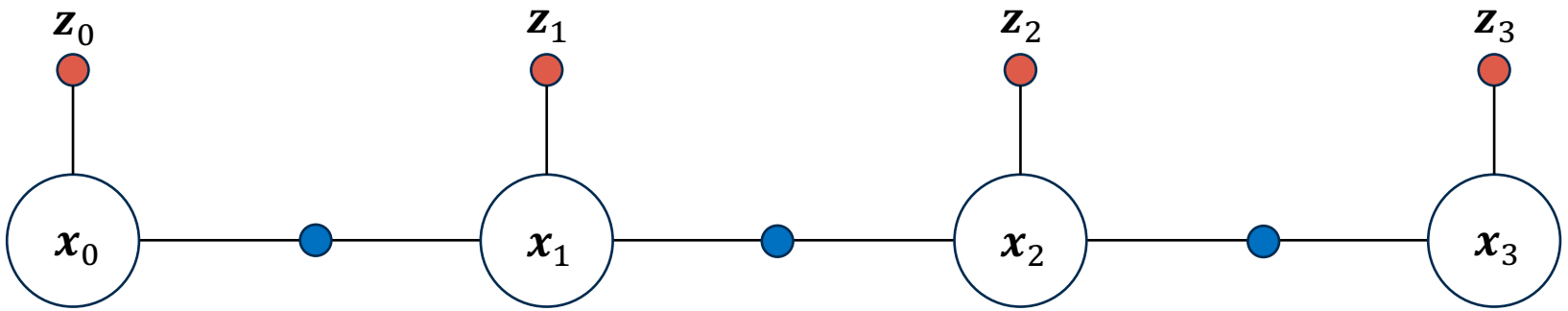
CRICOS No.00213J

Factorisation into Smaller Subsets

- Such big joint probability distributions can often be *factorised* into smaller sets.
- Independence between the involved probabilistic variables matters!

$$p(x_0, x_1, \dots, x_t | u_0, u_1, \dots, u_{t-1}, z_0, z_1, \dots, z_t) = \prod_i p(x_i | x_{i-1}, u_{i-1}) \cdot \prod_k p(x_k | z_k)$$

- These are the "**factors**" in the **Factor Graph**!

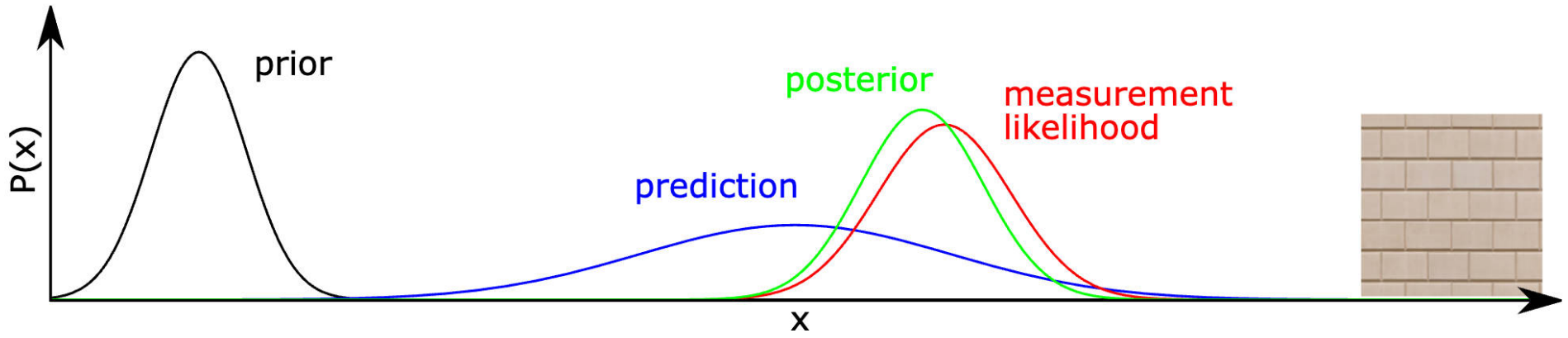


CRICOS No.00213J

Distribution vs MAP-Estimate

"Where is the robot NOW, based on all I know about the PAST control inputs and sonar measurements?"

$$p(x_t | u_0, u_1, \ldots, u_{t-1}, z_0, z_1, \ldots, z_t)$$



CRICOS No.00213J

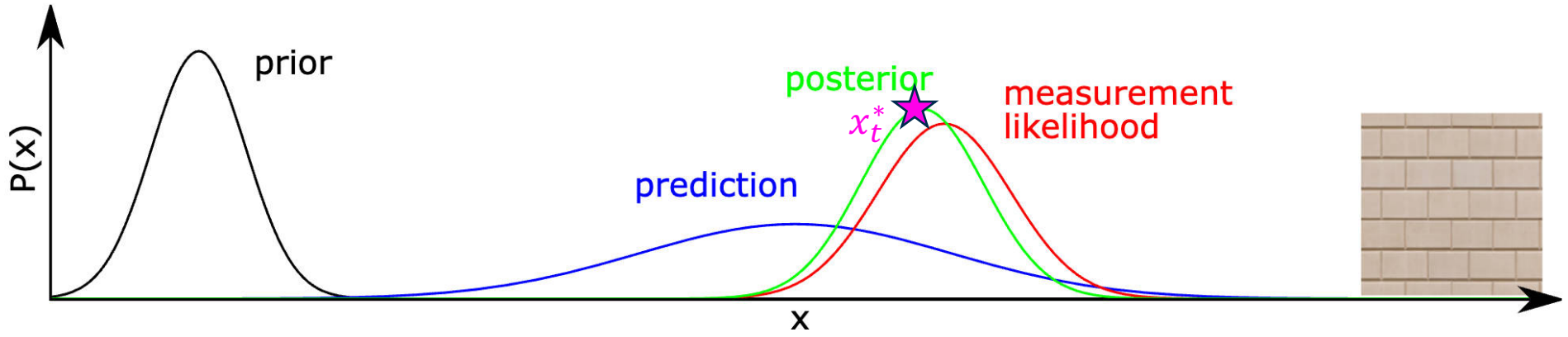
Distribution vs MAP-Estimate

"Where is the robot NOW, based on all I know about the PAST control inputs and sonar measurements?"

$$p(x_t | u_0, u_1, \dots, u_{t-1}, z_0, z_1, \dots, z_t)$$

→ I can ask for the **complete distribution** over x_t or the **most likely value** (written as x_t^*).

$$x_t^* = \underset{x}{\operatorname{argmax}}\, p(x_t | u_0, u_1, \dots, u_{t-1}, z_0, z_1, \dots, z_t)$$



CRICOS No.00213J

Distribution vs MAP-Estimate

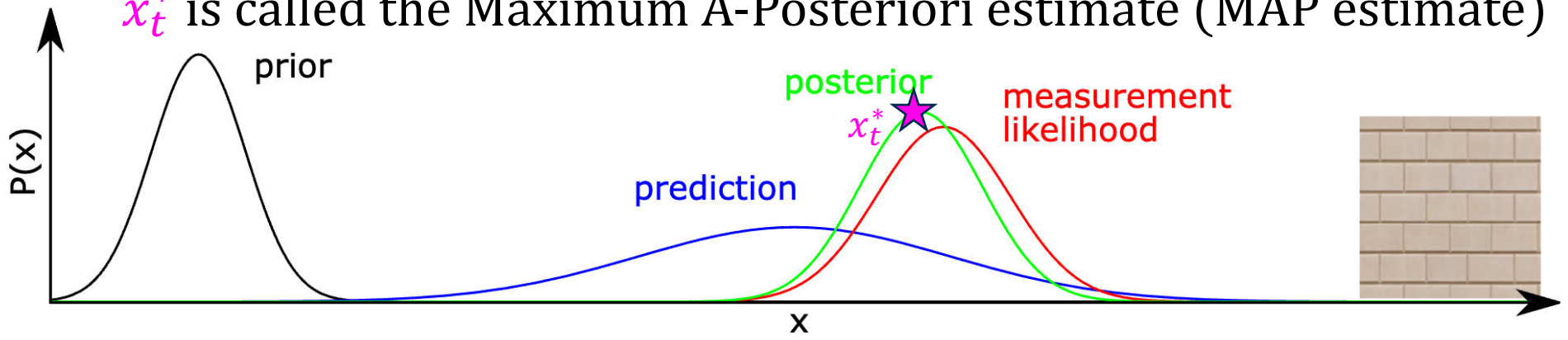
"Where is the robot NOW, based on all I know about the PAST control inputs and sonar measurements?"

$$p(x_t | u_0, u_1, \dots, u_{t-1}, z_0, z_1, \dots, z_t)$$

→ I can ask for the **complete distribution** over x_t or the **most likely value** (written as x_t^*).

$$x_t^* = \underset{x}{\operatorname{argmax}}\, p(x_t | u_0, u_1, \dots, u_{t-1}, z_0, z_1, \dots, z_t)$$

x_t^* is called the Maximum A-Posteriori estimate (MAP estimate)

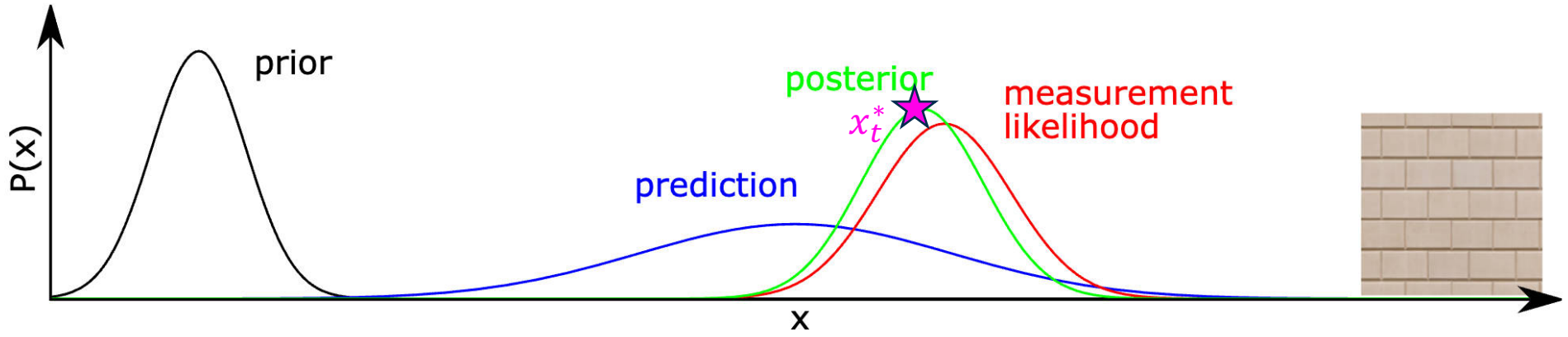


CRICOS No.00213J

Distribution vs MAP-Estimate

Pros and Cons:

- Full distribution gives us the “full picture”, e.g. mean and variance
  - We can continue the estimation since we still have a full probability distribution.
- MAP-Estimate is easier to calculate, but we lose potentially important information about the uncertainty of the estimate.
  - but: we can recover it later (at higher cost)



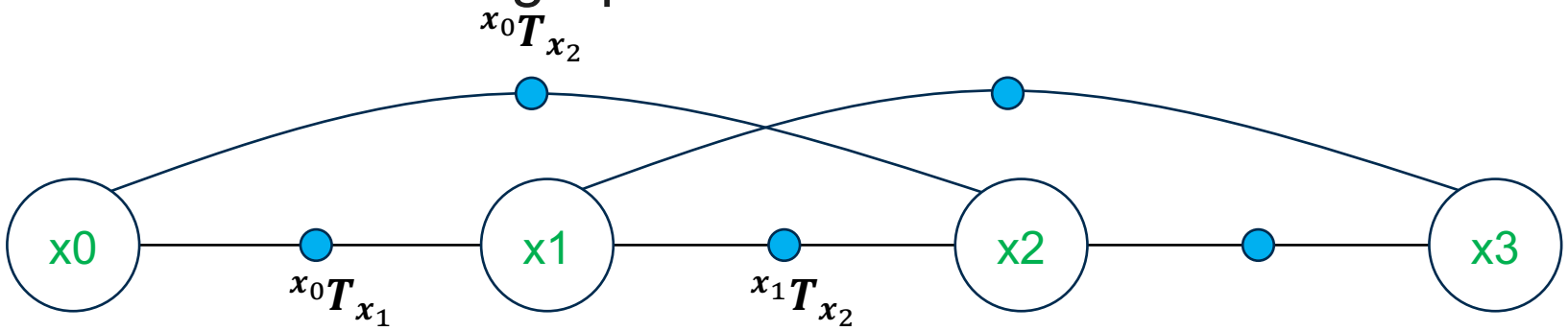
CRICOS No.00213J

What is the Connection between Factor Graphs and Optimisation Problems?

We now understand the pose-graph problem can be expressed as:

$$p(x_0, x_1, \dots, x_t | {}^{x_0}\boldsymbol{T}_{x_1}, {}^{x_1}\boldsymbol{T}_{x_2}\ {}^{x_0}\boldsymbol{T}_{x_2}, \dots) = \prod_{i,j} p(x_i | x_j, {}^{x_i}\boldsymbol{T}_{x_j})$$

Which corresponds to this factor graph:



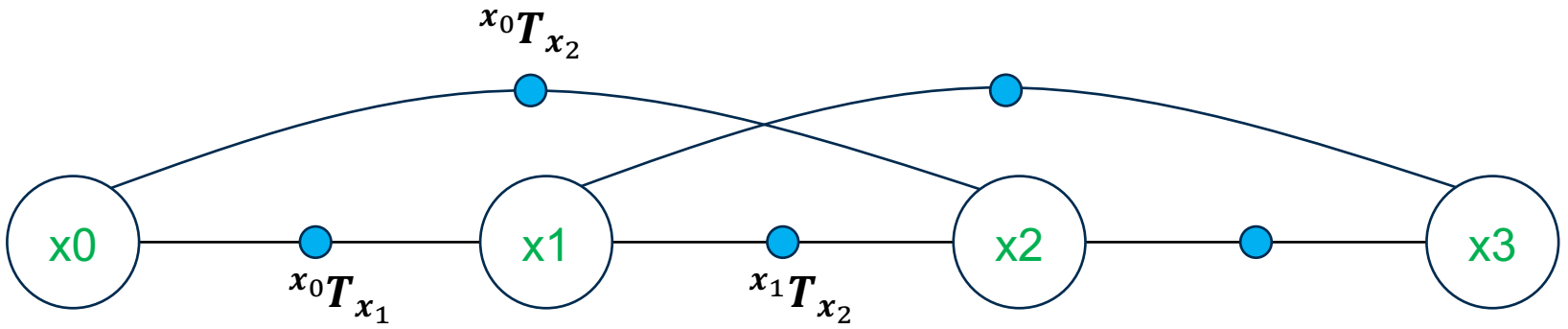
CRICOS No.00213J

What is the Connection between Factor Graphs and Optimisation Problems?

We now understand the pose-graph problem can be expressed as:

$$p(x_0, x_1, \dots, x_t | {}^{x_0}\boldsymbol{T}_{x_1}, {}^{x_1}\boldsymbol{T}_{x_2}\ {}^{x_0}\boldsymbol{T}_{x_2}, \dots) = \prod_{i,j} p(x_i | x_j, {}^{x_i}\boldsymbol{T}_{x_j})$$

Which corresponds to this factor graph:



We also said that we can solve this via nonlinear least squares optimization:

$$\boldsymbol{X}^* = \underset{\boldsymbol{X}}{\operatorname{argmin}} \sum_{i,j} \left\| \boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j \right\|^2$$

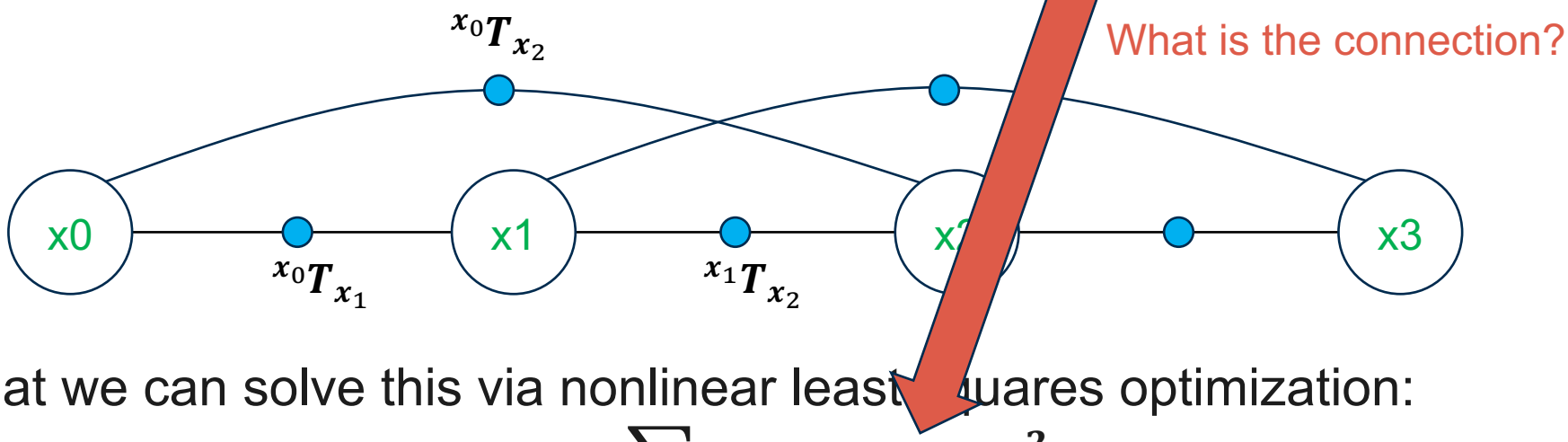
CRICOS No.00213J

What is the Connection between Factor Graphs and Optimisation Problems?

We now understand the pose-graph problem (week 4, 5) can be expressed as:

$$p(x_0, x_1, \dots, x_t | {}^{x_0}\boldsymbol{T}_{x_1}, {}^{x_1}\boldsymbol{T}_{x_2}\ {}^{x_0}\boldsymbol{T}_{x_2}, \dots) = \prod_{i,j} p(x_i | x_j, {}^{x_i}\boldsymbol{T}_{x_j})$$

Which corresponds to this factor graph:



We also said that we can solve this via nonlinear least squares optimization:

$$\boldsymbol{X}^* = \underset{\boldsymbol{X}}{\operatorname{argmin}} \sum_{i,j} \left\| \boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j \right\|^2$$

MAP Estimate!!

CRICOS No.00213J

Time to put it all together!

CRICOS No.00213J

Time to put it all together!

- Let's start with the full distribution and its factorization:

$$p(\boldsymbol{x}_0, \boldsymbol{x}_1, \ldots, x_t | {}^{\boldsymbol{x}_0}\boldsymbol{T}_{\boldsymbol{x}_1}, {}^{\boldsymbol{x}_1}\boldsymbol{T}_{\boldsymbol{x}_2}\ {}^{\boldsymbol{x}_0}\boldsymbol{T}_{\boldsymbol{x}_2}, \ldots) = \prod_{i,j} p(\boldsymbol{x}_i | \boldsymbol{x}_j, {}^{\boldsymbol{x}_i}\boldsymbol{T}_{\boldsymbol{x}_j})$$

CRICOS No.00213J

Time to put it all together!

- Let's start with the full distribution and its factorization:

$$p(\boldsymbol{x}_0, \boldsymbol{x}_1, \dots, x_t | {}^{\boldsymbol{x}_0}\boldsymbol{T}_{\boldsymbol{x}_1}, {}^{\boldsymbol{x}_1}\boldsymbol{T}_{\boldsymbol{x}_2}\ {}^{\boldsymbol{x}_0}\boldsymbol{T}_{\boldsymbol{x}_2}, \dots) = \prod_{i,j} p(\boldsymbol{x}_i | \boldsymbol{x}_j, {}^{\boldsymbol{x}_i}\boldsymbol{T}_{\boldsymbol{x}_j})$$

- Let's say we are only interested in the Maximum A-Posteriori Estimates

$$\mathbf{X}^* = \underset{x}{\operatorname{argmax}}\, p(\boldsymbol{x}_0, \boldsymbol{x}_1, \dots, x_t | {}^{\boldsymbol{x}_0}\boldsymbol{T}_{\boldsymbol{x}_1}, {}^{\boldsymbol{x}_1}\boldsymbol{T}_{\boldsymbol{x}_2}\ {}^{\boldsymbol{x}_0}\boldsymbol{T}_{\boldsymbol{x}_2}, \dots\ )$$

$$\mathbf{X}^* = \underset{x}{\operatorname{argmax}} \prod_{i,j} p(\boldsymbol{x}_i | \boldsymbol{x}_j, {}^{\boldsymbol{x}_i}\boldsymbol{T}_{\boldsymbol{x}_j})$$

CRICOS No.00213J

Time to put it all together!

- Let's start with the full distribution and its factorization:

$$p(\boldsymbol{x}_0, \boldsymbol{x}_1, \ldots, x_t | {}^{x_0}\boldsymbol{T}_{x_1}, {}^{x_1}\boldsymbol{T}_{x_2}\ {}^{x_0}\boldsymbol{T}_{x_2}, \ldots) = \prod_{i,j} p(\boldsymbol{x}_i | \boldsymbol{x}_j, {}^{x_i}\boldsymbol{T}_{x_j})$$

- Let's say we are only interested in the Maximum A-Posteriori Estimates

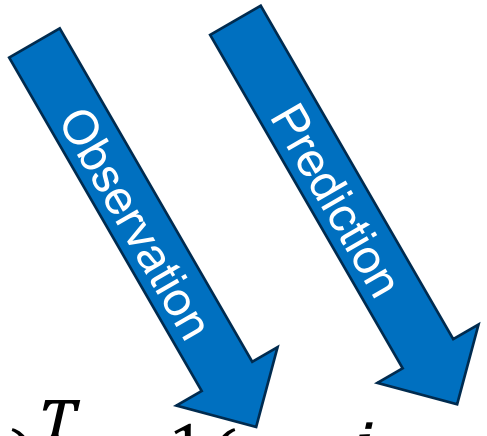
$$\mathbf{X}^* = \underset{x}{\operatorname{argmax}}\, p(\boldsymbol{x}_0, \boldsymbol{x}_1, \ldots, x_t | {}^{x_0}\boldsymbol{T}_{x_1}, {}^{x_1}\boldsymbol{T}_{x_2}\ {}^{x_0}\boldsymbol{T}_{x_2}, \ldots)$$

$$\mathbf{X}^* = \underset{x}{\operatorname{argmax}} \prod_{i,j} p(\boldsymbol{x}_i | \boldsymbol{x}_j, {}^{x_i}\boldsymbol{T}_{x_j})$$

Now we make an important assumption: $p\left(\boldsymbol{x}_i | \boldsymbol{x}_j, {}^{x_i}\boldsymbol{T}_{x_j}\right)$ is a Gaussian!

Time to put it all together!

If $p(\boldsymbol{x}_i|\boldsymbol{x}_j, {}^{x_i}\boldsymbol{T}_{x_j})$ is Gaussian, it is of the form:



$$p(\boldsymbol{x}_i|\boldsymbol{x}_j, {}^{x_i}\boldsymbol{T}_{x_j}) = \frac{1}{\sqrt{2\pi|\boldsymbol{\Sigma}_{ij}|}} e^{-\frac{1}{2}\left(\boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j\right)^T \boldsymbol{\Sigma}_{ij}^{-1}\left(\boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j\right)}$$

Compare to the standard form of the multivariate Gaussian!

$$\frac{\exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right)}{\sqrt{(2\pi)^k|\boldsymbol{\Sigma}|}}$$

CRICOS No.00213J

Time to put it all together!

If $p(\boldsymbol{x}_i|\boldsymbol{x}_j, {}^{x_i}\boldsymbol{T}_{x_j})$ is Gaussian, it is of the form:

$$p(\boldsymbol{x}_i|\boldsymbol{x}_j, {}^{x_i}\boldsymbol{T}_{x_j}) = \frac{1}{\sqrt{2\pi|\boldsymbol{\Sigma_{ij}}|}} e^{-\frac{1}{2}\left(\boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j\right)^T \boldsymbol{\Sigma}_{ij}^{-1} \left(\boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j\right)}$$

This term appears twice, we multiply it by itself and then "divide" by $\boldsymbol{\Sigma_{ij}}$!

CRICOS No.00213J

Time to put it all together!

Remember, we are after the MAP-Estimate, not the full distribution.

$$\mathbf{X}^* = \underset{\boldsymbol{x}}{\operatorname{argmax}} \prod_{i,j} p(\boldsymbol{x}_i | \boldsymbol{x}_j, {}^{\boldsymbol{x}_i}\boldsymbol{T}_{\boldsymbol{x}_j}) = \underset{\boldsymbol{x}}{\operatorname{argmax}} \prod_{i,j} \frac{1}{\sqrt{2\pi|\boldsymbol{\Sigma}_{\boldsymbol{ij}}|}} e^{-\frac{1}{2}\left(\boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j\right)^T \boldsymbol{\Sigma}_{ij}^{-1} \left(\boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j\right)}$$

CRICOS No.00213J

Time to put it all together!

Remember, we are after the MAP-Estimate

$$\mathbf{X}^* = \underset{\boldsymbol{x}}{\operatorname{argmax}} \prod_{i,j} p(\boldsymbol{x}_i | \boldsymbol{x}_j, {}^{\boldsymbol{x}_i}\boldsymbol{T}_{\boldsymbol{x}_j}) = \underset{\boldsymbol{x}}{\operatorname{argmax}} \prod_{i,j} \frac{1}{\sqrt{2\pi|\boldsymbol{\Sigma}_{\boldsymbol{ij}}|}} e^{-\frac{1}{2}\left(\boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j\right)^T \boldsymbol{\Sigma}_{ij}^{-1} \left(\boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j\right)}$$

Now: Notice how the term $\frac{1}{\sqrt{2\pi|\boldsymbol{\Sigma}_{\boldsymbol{ij}}|}}$ does not depend on $\boldsymbol{x}$ so we can remove it.

CRICOS No.00213J

Time to put it all together!

Remember, we are after the MAP-Estimate

$$\mathbf{X}^* = \underset{\boldsymbol{x}}{\operatorname{argmax}} \prod_{i,j} p(\boldsymbol{x}_i | \boldsymbol{x}_j, {}^{\boldsymbol{x}_i}\boldsymbol{T}_{\boldsymbol{x}_j}) = \underset{\boldsymbol{x}}{\operatorname{argmax}} \prod_{i,j} \frac{1}{\sqrt{2\pi|\boldsymbol{\Sigma}_{\boldsymbol{ij}}|}} e^{-\frac{1}{2}\left(\boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j\right)^T \boldsymbol{\Sigma}_{ij}^{-1} \left(\boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j\right)}$$

Now: Notice how the term $\frac{1}{\sqrt{2\pi|\boldsymbol{\Sigma}_{\boldsymbol{ij}}|}}$ does not depend on $\boldsymbol{x}$ so we can remove it.

$$\mathbf{X}^* = \underset{\boldsymbol{x}}{\operatorname{argmax}} \prod_{i,j} p(\boldsymbol{x}_i | \boldsymbol{x}_j, {}^{\boldsymbol{x}_i}\boldsymbol{T}_{\boldsymbol{x}_j}) = \underset{\boldsymbol{x}}{\operatorname{argmax}} \prod_{i,j} e^{-\frac{1}{2}\left(\boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j\right)^T \boldsymbol{\Sigma}_{ij}^{-1} \left(\boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j\right)}$$

CRICOS No.00213J

Time to put it all together!

$$\mathbf{X}^* = \underset{\boldsymbol{x}}{\operatorname{argmax}} \prod_{i,j} e^{-\frac{1}{2}\left(\boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j\right)^T \boldsymbol{\Sigma}_{ij}^{-1} \left(\boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j\right)}$$

Remove the cumbersome exponent by taking the log.

This does not change the maximiser of x because log is a monotonous function and argmax f(x) = argmax log f(x).

This also turns the products into sums!

Because $\log\left(e^a \cdot e^b\right) = a + b$.

$$\mathbf{X}^* = \underset{\boldsymbol{x}}{\operatorname{argmax}} \sum_{i,j} \left(-\frac{1}{2} \left(\boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j\right)^T \boldsymbol{\Sigma}_{ij}^{-1} \left(\boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j\right) \right)$$

CRICOS No.00213J

Time to put it all together!

$$\mathbf{X}^* = \underset{\boldsymbol{x}}{\operatorname{argmax}} \sum_{i,j} \left(-\frac{1}{2} \left(\boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j \right)^T \boldsymbol{\Sigma}_{ij}^{-1} \left(\boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j \right) \right)$$

This term appears twice, we multiply it by itself and then "divide" by $\boldsymbol{\Sigma_{ij}}$!

- We can write this shorter as $\left\| \boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j \right\|^2_{\boldsymbol{\Sigma}_{i,j}}$
  - This is called the "Mahalanobis Distance".
  - It is like the Euclidean Distance, but scaled by the uncertainty in $\boldsymbol{\Sigma}$.

$$\mathbf{X}^* = \underset{\boldsymbol{x}}{\operatorname{argmax}} \sum_{i,j} -\frac{1}{2} \left\| \boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j \right\|^2_{\boldsymbol{\Sigma}_{i,j}}$$

CRICOS No.00213J

Time to put it all together!

$$\mathbf{X}^* = \underset{\boldsymbol{x}}{\operatorname{argmax}} \sum_{i,j} -\frac{1}{2} \left\| \boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j \right\|^2_{\boldsymbol{\Sigma}_{i,j}}$$

Now the last step: we like minimisation problems better than maximisation, so let's just turn it around:

$$\mathbf{X}^* = \underset{\boldsymbol{x}}{\operatorname{argmin}} -\sum_{i,j} -\frac{1}{2} \left\| \boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j \right\|^2_{\boldsymbol{\Sigma}_{i,j}}$$

$$\mathbf{X}^* = \underset{\boldsymbol{x}}{\operatorname{argmin}} \sum_{i,j} \frac{1}{2} \left\| \boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j \right\|^2_{\boldsymbol{\Sigma}_{i,j}}$$

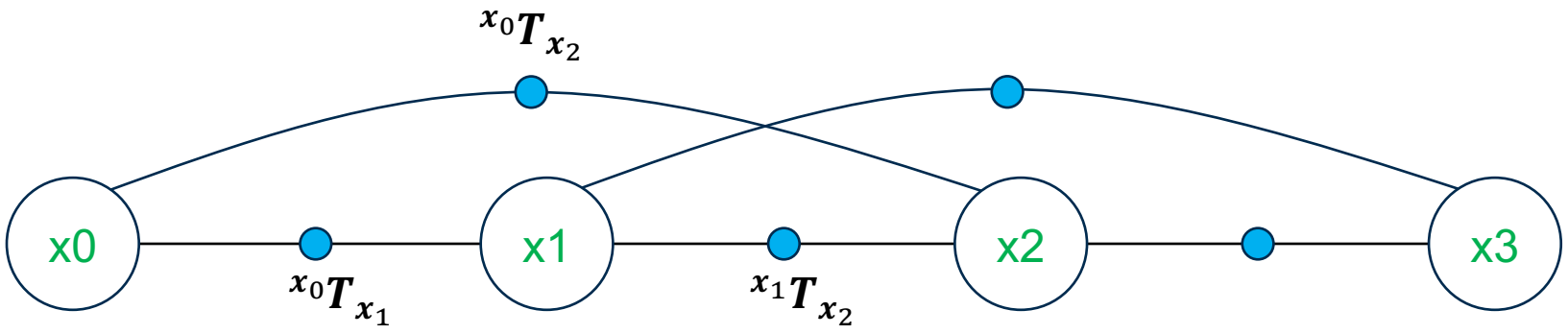
We can even drop the factor of ½ as it does not change where the optimum **x** occurs.

CRICOS No.00213J

Time to put it all together!

And there we have it: this is a (weighted) nonlinear least squares problem.

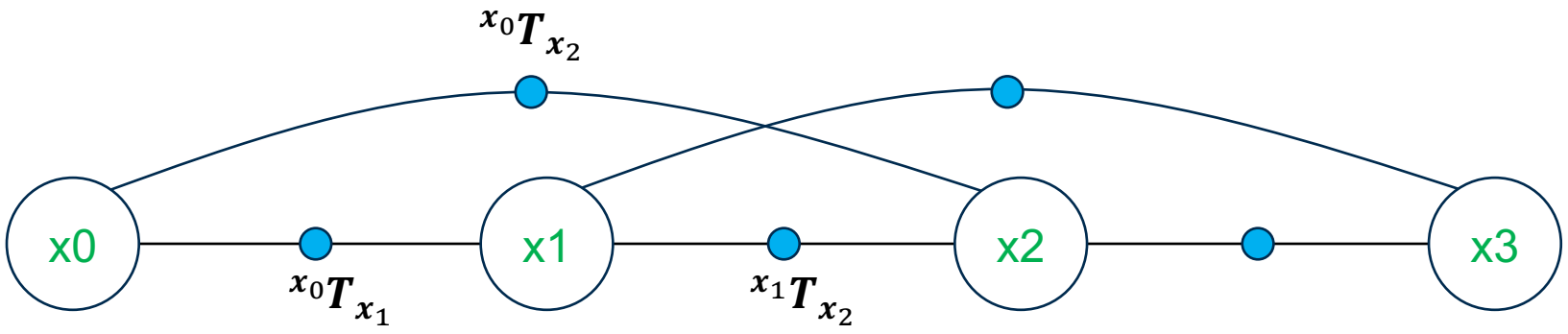
$$\mathbf{X}^* = \underset{\boldsymbol{x}}{\operatorname{argmin}} \sum_{i,j} \left\| \boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j \right\|^2_{\boldsymbol{\Sigma}_{i,j}}$$



CRICOS No.00213J

What did we just do?

- We started from a probabilistic view of the pose graph problem, which can be represented as a factor.
- We found a way to find the MAP-Estimate by solving a least squares problem.
- That is great news – We know how to do this! $\mathbf{X}^* = \underset{\boldsymbol{x}}{\operatorname{argmin}} \sum_{i,j} \left\| \boldsymbol{x}_i - {}^{i}\boldsymbol{T}_j \cdot \boldsymbol{x}_j \right\|^2_{\boldsymbol{\Sigma}_{i,j}}$

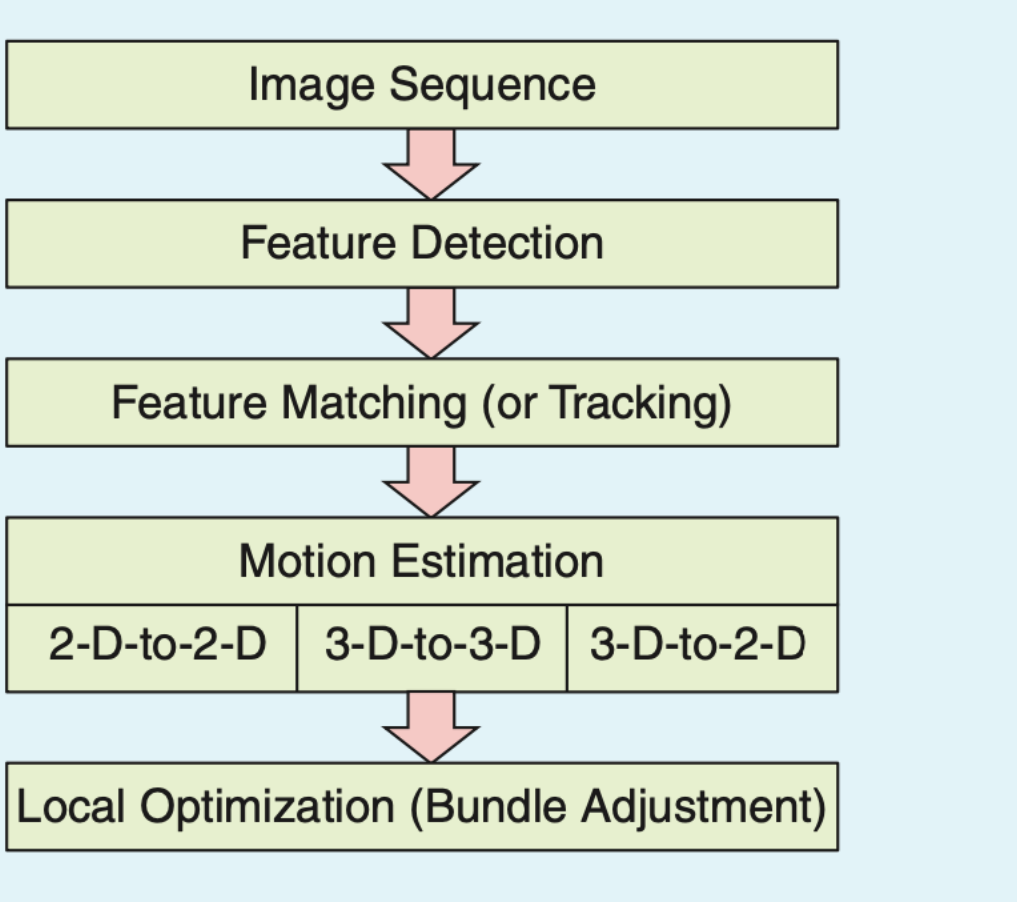


CRICOS No.00213J



Recap

Recap – What we learned so far



CRICOS No.00213J

Next Week

- How to evaluate Visual Odometry: Absolute Trajectory Error, Relative Pose Error
- Visual Inertial Odometry
- Dense and direct methods
- Beyond Visual Odometry: Visual SLAM
- Q&A for Part 1 of the semester

- Week 8 onwards: Deep learning for Robotic Vision

CRICOS No.00213J